

UNIVERZITET U BANJALUCI
ELEKTROTEHNIČKI FAKULTET
STUDIJSKI PROGRAM RAČUNARSTVO I INFORMATIKA

PERFORMANSE RAČUNARSKIH SISTEMA

Skripta sa predavanja i vježbi

Banjaluka, mart 2011.

SADRŽAJ

1.	UVOD U PERFORMANSE RAČUNARSKIH SISTEMA	4
2.	MJERENJE PERFORMANSI	6
2.1.	Ciljevi mjerenja performansi	6
2.2.	Tehnike mjerenja performansi	7
2.3.	Faze studije evaluacije performansi	9
2.4.	Metrika performansi	9
2.4.1.	Karakteristike dobrih metrika	10
2.5.	Skale mjerenja	11
2.6.	Mjerne tehnike i alati	13
2.6.1.	Strategije mjerenja	13
2.7.	Workload	15
2.7.1.	Strategije benčmarkinga	17
3.	SIMULACIJA	20
3.1.	Vrste modela	20
3.1.1.	Neformalni i formalni modeli	21
3.2.	Računarska simulacija	23
3.2.1.	Modeliranje i simulacija	24
3.3.	Karakteristike simulacionog modeliranja	26
3.4.	Potreba za simulacijom	27
3.5.	Prednosti i nedostaci simulacije	28
3.6.	Simulacioni proces	29
3.7.	Podjele simulacionih modela	32
3.7.1.	Deterministički i stohastički modeli	32
3.7.2.	Diskretni i kontinualni modeli	33
3.8.	Vrste simulacionih modela	34
3.8.1.	Emulacija	34
3.8.2.	Monte Karlo simulacija	34
3.8.3.	Kontinualna simulacija	36
3.8.4.	Simulacija diskretnih događaja	37
3.8.5.	Mješovita simulacija	41
3.8.6.	Izbor tipa simulacionog modela	42
3.9.	Validacija i verifikacija	42
3.9.1.	Validacija simulacionih modela	44
3.9.2.	Verifikacija simulacionih modela	50
3.10.	Generisanje slučajnih brojeva	51
3.10.1.	Manuelne metode	52
3.10.2.	Tabele slučajnih brojeva	52
3.10.3.	Metode za generisanje SB na analognom računaru	52
3.10.4.	Metode za generisanje SB na digitalnom računaru	52
3.10.5.	Linearni kongruentni GSB	54
4.	ANALIZA REDOVA ČEKANJA	56
4.1.	Modeli mreža	56

4.2.	Osnovne pretpostavke i notacija	57
4.3.	Operacionalna analiza.....	59
4.3.1.	Utilizacioni zakon	59
4.3.2.	Little-ov zakon.....	61
4.4.	Stohastička analiza	62
4.4.1.	Kendall-ova notacija.....	63
4.4.2.	M/M/1 sistemi	64
5.	INTERPRETACIJA REZULTATA MJERENJA	70
5.1.	Srednje vrijednosti i varijabilnost	70
5.1.1.	Aritmetička sredina	71
5.1.2.	Harmonijska sredina	72
5.1.3.	Geometrijska sredina	73
5.1.4.	Kvantifikovanje varijabilnosti.....	75
5.2.	Greške u eksperimentalnim mjerenjima	76
5.2.1.	Kvantifikovanje grešaka	79
5.3.	Poređenje alternativa	85
5.3.1.	Poređenje dvije alternative	86
5.3.2.	Poređenje više od dvije alternative	91
5.4.	Linearni regresioni model	98
5.4.1.	Korelacija.....	100
5.4.2.	Verifikovanje linearnosti	102
5.4.3.	Nelinearni modeli.....	102
5.5.	Verifikacija slučajnih brojeva	105
5.5.1.	Goodness-of-fit testiranje.....	105
5.5.2.	Testovi nezavisnosti	108
6.	SAOPŠTAVANJE REZULTATA	109
6.1.	Tipovi varijabli	109
6.2.	Preporuke za pravljenje dobrih grafičkih dijagrama.....	109
6.3.	Česte greške na dijagramima.....	111
6.4.	Grafičke varke	112
	PRILOG A – Kritične vrijednosti Studentove t raspodjele	116
	PRILOG B.1 – Kritične vrijednosti F raspodjele (90%)	117
	PRILOG B.2 – Kritične vrijednosti F raspodjele (95%)	118
	PRILOG B.3 – Kritične vrijednosti F raspodjele (99%)	119
	PRILOG C – Kritične vrijednosti hi-kvadrat raspodjele.....	120
	LITERATURA	121

1. UVOD U PERFORMANSE RAČUNARSKIH SISTEMA

Većina naučnih i inženjerskih polja imaju definisane alate i tehnike za mjerenje i poređenje oblasti od interesa i za njihovo precizno saopštavanje. Međutim, u polju računarske nauke i inženjeringa, iznenađujuće, ne postoji dogovor kako da se mjeri jedna fundamentalna stvar kao što su performanse računarskih sistema. Na primjer, brzina automobila se može lako mjeriti pomoću standardnih jedinica, kao što je m/s. Korištenje standardizovanih jedinica dalje omogućava direktno poređenje brzine automobila sa, na primjer, brzinom aviona. Poređenje performansi različitih računarskih sistema nije uopšte tako jednostavno i direktno.

Problemi su se pojavili sa nepostojanjem dogovora čak i oko, na prvi pogled, najjednostavnijih ideja, kao što je određivanje najpogodnije metrike za mjerenje performansi. Sljedeći problem se pojavljuje kada mnogi istraživači skupljaju i objavljuju rezultate koristeći upitne i, u većini slučajeva, nepravilne metodologije.

Glavni cilj kursa je razumijevanje fundamentalnih koncepata iza alata i tehnika koji se koriste u analizi performansi računarskih sistema. Iako će se u nekim trenucima tokom kursa spomenuti neki specifični programi i alati, cilj kursa nije da se nauče njihove specifične implementacije, već osnovne ideje koje stoje u pozadini. Cilj namjerno nije da se studenti načine ekspertima u korištenju nekog softverskog benčmark alata, već da razumiju osnovne pretpostavke koje stoje iza tog alata, kao i iza statističkih tehnika koje se koriste u svrhu interpretiranja izmjerenih podataka. Prava stručnost u korištenju individualnih alata tek tada može da dođe do izražaja. Na taj način se najlakše može doći do zaključaka o potencijalnim prednostima prilikom njihovog korištenja, ali i hazarda koji se mogu pojaviti.

Performanse računarskih sistema predstavljaju termin koji se koristi kao indikator koliko dobro sistem radi, pod uslovom da radi korektno. Bitan aspekt svake inženjerske aktivnosti je vrednovanje sistema u odnosu na aktivnosti koje mu pripadaju. U zainteresovane strane spadaju projektanti, proizvođači, kupci, menadžeri i, konačno, krajnji korisnici računarskih sistema.

Mjerenje performansi je multidisciplinarno i podrazumijeva korištenje tehnika iz arhitekture računara, operativnih sistema, programiranja, diskretne matematike, statistike i teorije vjerovatnoće.

Specifikacija performansi mora biti prihvatljiva za korisnike i ona daje odgovor na pitanje kako sistem izvodi svoje funkcije. Obično se daje u kvantitativnim terminima.

Računarski sistem je agregacija hardvera i softvera računara. Ove komponente se nazivaju resursima. Svaki resurs ima svoje vlastite attribute koji su opisani sa sistemskim parametrima, čak i kada ne može da bude izražen brojučano (npr. algoritam po kojem komponenta radi).

Na nekim sličnim kursevima iz ove oblasti se pristupa matematički rigoroznije¹. Za razliku od tog pristupa, ovdje je ideja da se prisustvo matematike svede na minimum. Međutim, nemoguće je zaobići neke matematičke tehnike (uglavnom iz statistike), koje se izučavaju na svakom sličnom kursu u svijetu.

¹ Na primjer, pogledati vježbe iz istoimenog predmeta na ETF-u u Beogradu

2. MJERENJE PERFORMANSI

Za razliku od izračunavanja troškova za nabavljanje računarskih sistema, mjerenje njegovih performansi je krajnje složeno. Iznenadjujuće, jednu od glavnih teškoća predstavlja činjenica da se ljudi ne mogu složiti na koji način treba da se performanse mjere i interpretiraju. Teškoće dalje idu do toga da se ljudi ne mogu složiti ni šta riječ performanse zapravo označava.

Def. Analiza performansi u polju računarske nauke i inženjeringa predstavlja kombinaciju mjerenja, interpretacije i saopštavanja “brzine” ili “veliĉine” računarskog sistema.

Termini brzina i veličina su svjesno stavljeni pod navodnike da bi označili da njihova definicija isključivo zavisi od specifičnosti situacija u kojima se mjere. Nažalost, performanse nekog dijela računarskog sistema u većini sluĉaja zavise od interakcije sa ostalim komponentama, tako da eksperti na ovom polju moraju da steknu vještinu pomoću koje će na što efikasniji način filtrirati te dodatne uticaje na mjerenje performansi sistema.

Jedan od najzanimljivijih zadataka analitiĉara performansi računarskih sistema može da bude proces donošenja odluke kako da se mjere odreĉene performanse. Potrebna je velika doza kreativnosti da bi se razvile pogodne tehnike za mjerenje koje će minimalno da poremete sistem koji se mjeri, a da istovremeno pružaju taĉne i reproducibilne rezultate. Nakon što su sakupljeni neophodni podaci, analitiĉar mora da interpretira rezultate koristeći odgovarajuće statistiĉke tehnike. Konaĉno, ĉak i izvanredni rezultati mjerenja interpretirani na odgovarajući statistiĉki naĉin nemaju nikakvo znaĉenje ako se ne iznesu na jasan i konzistentan naĉin.

Glave 2, 3 i 4 obraĉuju tri glavne tehnike koje se koriste za mjerenje performansi računarskih sistema – direktno mjerenje, simulaciju i analitiĉko modelovanje. Glava 5 je posvećena interpretaciji dobijenih rezultata, a glava 6 njihovom saopštavanju.

2.1. Ciljevi mjerenja performansi

Iako ciljevi svake analize performansi zavise od specifiĉne situacije, te vještine, interesa i mogućnosti analitiĉara performansi, ipak se mogu filtrirati zajedniĉki ciljevi svake analize performansi, kako za dizajnere računarskih sistema, tako i za krajnje korisnike. U takve ciljeve spadaju:

- poreĉenje alternativa – pošto se računarski sistemi sastoje od velikog broja komponentata koje su meĉusobno u interakciji, zadatak analitiĉara je da pruži kvantitativnu informaciju o tome koja konfiguracija je najbolja pod specifiĉnim uslovima,

- utvrđivanje uticaja nove komponente na postojeći sistem – ovaj pristup se uglavnom radi pomoću *before-and-after* statističke analize,
- *tuning* sistema – zadatak analitičara je da odredi set parametara pod kojima sistem ima najbolje ukupne performanse,
- identifikovanje relativnih performansi – performanse računarskih sistema uglavnom dobijaju smisao tek kada se uporede sa nekim drugim sistemom i njegovim specifičnim performansama,
- *debugging* performansi – zadatak ovog cilja je da se pronađe razlog zbog čega performanse nisu na željenom nivou,
- postavljanje očekivanja – zadatak analitičara performansi je da odredi koja su realna očekivanja od nove linije nekog sistema ili njegove komponente.

Pri analizi performansi računarskih sistema treba voditi računa o posljedici donošenja pogrešne odluke! Rizik donošenja pogrešne odluke mora da bude proporcionalan dubini izvršene analize.

2.2. Tehnike mjerenja performansi

Postoje tri fundamentalne tehnike mjerenja performansi. To su:

1. direktno mjerenje postojećih sistema,
2. simulacija,
3. analitičko modelovanje.

Direktna mjerenja pružaju najbolje rezultate pošto nije potrebno vršiti pojednostavljajuće pretpostavke o sistemu. Ova karakteristika čini tehniku direktnog mjerenja najpouzdanijom od sve tri tehnike. Međutim, direktna mjerenja nisu fleksibilna i pružaju informacije samo o specifičnom sistemu koji se mjeri. Nedostatak fleksibilnosti se odnosi na činjenicu da je skoro nemoguće mijenjati parametre sistema u realnom vremenu, a često je cilj da se izmjeri ponašanje sistema prilikom neke promjene. Navedeni nedostaci su doveli do drugih tehnika mjerenja.

Simulacija računarskog sistema se odnosi na program napisan sa svrhom da modeluje bitne detalje sistema koji se analizira. Pošto je simulator ništa drugo nego program, lako ga je modifikovati da prati promjene bilo koje simulirane komponente. Troškovi simulacije uključuju vrijeme i napor potrebne da se napiše i *debug*-uje simulacioni program, kao i vrijeme potrebno za izvršavanje potrebnih simulacija. Zavisno od kompleksnosti sistema koji se modeluje, kao i od nivoa detalja koji se modeluju, ovi troškovi mogu biti relativno niski u odnosu na troškove nabavke prave mašine na kojoj bi se izvodili isti eksperimenti. Glavni nedostatak simulacije je što je

nemoguće modelovati svaki detalj sistema, pa je potrebno vršiti određene apstrakcije. Te apstrakcije limitiraju tačnost dobijenih rezultata.

Analitički model predstavlja matematički opis sistema. U odnosu na prethodne dvije tehnike, rezultati analitičkog modelovanja su najmanje povjerljivi i najmanje tačni. Međutim, ova tehnika može da pruži brzi grubi pogled na ukupno ponašanje sistema ili neke od njegovih komponenata. Ako ništa drugo, analitički modeli se mogu koristiti kao sredstvo validacije jedne od prethodno dvije opisane tehnike. Validacija znači da se može provjeriti da li se rezultati dobijeni pomoću drugih tehnika mogu smatrati razumnim.

Sumiranje osobina sve tri tehnike je prikazano u tabeli 2.1.

Tabela 2.1 – Tehnike mjerenja performansi

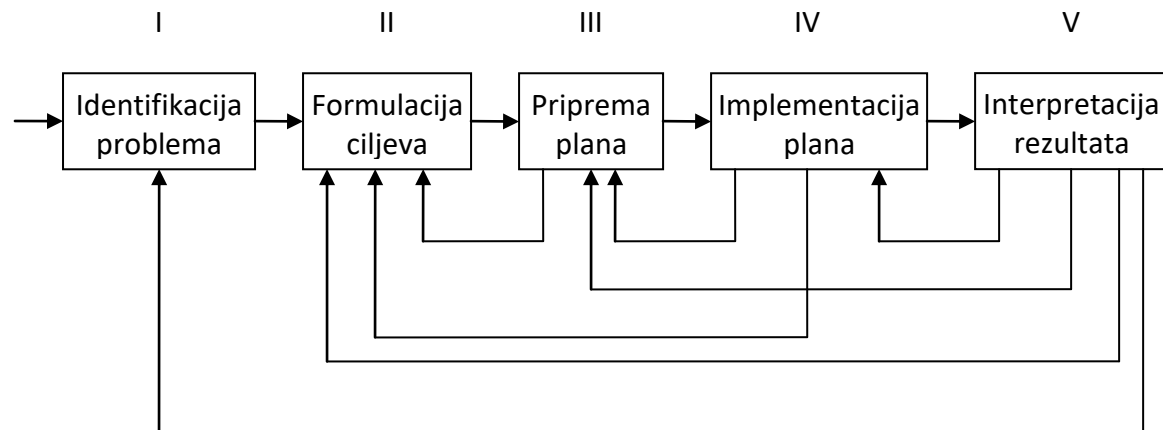
	Tehnika		
Karakteristika	Analitički model	Simulacija	Mjerenje
<i>Fleksibilnost</i>	Velika	Velika	Mala
<i>Cijena</i>	Mala	Srednja	Velika
<i>Povjerljivost</i>	Mala	Srednja	Velika
<i>Tačnost</i>	Mala	Srednja	Velika

Fleksibilnost tehnike se odnosi na to koliko je lako mijenjati konfiguraciju sistema tokom njegovog ispitivanja. Cijena se odnosi na vrijeme, napor i novac koji su potrebni da se izvedu odgovarajući eksperimenti korištenjem svake od tehnika. Povjerljivost tehnike je visoka ako upućena osoba ima visok nivo pouzdanosti da su dobijeni rezultati korektni. Konačno, tačnost indicira koliko su dobijeni rezultati bliski rezultatima dobijenim na realnom sistemu.

Izbor odgovarajuće tehnike zavisi od problema koji se rješava. Ovaj izbor se nekada više svodi na umjetnost nego na nauku.

2.3. Faze studije evaluacije performansi

Za razliku od monitoringa računarskog sistema, čija je glavna karakteristika kontinuirano praćenje realnog sistema, kod studije računarski sistem uopšte ne mora da postoji. Aktivnosti u ovoj studiji su grupisane u pet faza prikazanih na slici 2.1.



Slika 2.1 – Faze studije evaluacije performansi

Druga i treća faza uključuju pažljivu procjenu troškova i doprinos studije. U drugoj fazi se dodatno definišu metrike (indeksi) performansi.

Treća, četvrta i peta faza se mogu posmatrati kao komponente iterativne procedure (hipoteza se formuliše, testira i ako je test nezadovoljavajući vrši se vraćanje u neku od prethodnih faza radi modifikacije).

2.4. Metrika performansi

Osnovne karakteristike sistema koje tipično treba da se izmjere su:

- broj koliko puta se desio neki događaj,
- trajanje nekog vremenskog intervala,
- veličina nekog parametra.

Metrika performansi je vrijednost koja se koristi da se opišu performanse sistema, a dobija se mjerenjem jedne od navedenih fundamentalnih karakteristika.

Metrike performansi su deskriptori koji se upotrebljavaju da reprezentuju sistemske performanse ili neke od njihovih aspekata.

Različiti ljudi upotrebljavaju različite metrike performansi. To predstavlja problem pošto se zbog upotrebe individualnih i subjektivnih indeksa dešava da performanse budu

neuporedive i nekonzistentne. To predstavlja glavni razlog zbog kojeg metrike performansi treba da budu objektivne i kvantitabilne da bi mogle da budu objektivna mjera performansi mjerenih aspekata sistema.

Međutim, većina metrika su teške za kvantifikovanje. Primjeri takvih metrika su lakoća upotrebe sistema, strukturalnost programskog jezika i jačina instrukcionog seta. Neke od klasa kvantifikovanja performansi prikazane su u tabeli 2.2.

Tabela 2.2 – Klase metrika performansi

Metrika – klasa	Primjer indeksa	Opšta definicija
Produktivnost (volumen / vrijeme)	Propusnost, produkcion ciklus, kapacitet, maksimalan broj instrukcija...	Volumen informacija procesuiran u jedinici vremena
Odziv (vrijeme)	Vrijeme odziva, propusno vrijeme, vrijeme reakcije	Vrijeme između inputa i outputa
Iskoristivost (bez dimenzije)	Hardverski moduli (CPU, I/O uređaji), operativni sistem, sistemski softver, DBMS	Odnos vremena koje sistem provodi u nekoj specifičnoj namjeni i ukupnog vremena

Vrijeme odziva se definiše kao interval između korisničkog zahtjeva i sistemskog odgovora. Ova definicija je dosta pojednostavljena pošto zahtjevi, kao i odgovori, predstavljaju procese i ne dešavaju se trenutno.

Propusnost se definiše kao broj zahtjeva u jedinici vremena. Ovo zapravo predstavlja brzinu kojom sistem radi.

Nijedno od navedenih mjerenja nije nezavisno. Zavisnost postoji i od *workload*-a i od sistema koji se mjeri.

2.4.1. Karakteristike dobrih metrika

Iako karakteristike koje su navedene u nastavku teksta ne predstavljaju apsolutne zahtjeve koje metrika performansi mora da ispunjava, vremenom se pokazalo da metrike koje ne zadovoljavaju navedene uslove često vode do izvođenja pogrešnih zaključaka. Karakteristike dobrih metrika su:

1. *linearnost*

Pošto ljudi intuitivno razmišljaju u linearnim okvirima, vrijednost metrike treba da bude linearno proporcionalna stvarnim performansama sistema koji se mjeri. Jednostavnije rečeno, ako se performanse sistema promijene nekoliko puta i metrika mora da se promijeni za isti taj odnos. Karakteristika

proporcionalnosti čini metriku intuitivno privlačnom za mnoge ljude. Na primjer, logaritamska skala kojom se mjeri jačina zvuka ne zadovoljava ovu osobinu. To ne znači da nešto nije u redu sa metrikom, ali je poželjno da metrike koje se koriste u računarskim sistemima ispunjavaju osobinu linearnosti.

2. *pouzdanost*

Metrika se smatra pouzdanom ako sistem koji ima bolju izmjerenu metriku uvijek radi bolje od sistema sa slabijom metrikom. Iako se ova osobina čini toliko očiglednom da nije potrebno da se eksplicitno naglašava, nekoliko često korištenih metrika ne zadovoljava ovaj uslov. Jedna od takvih metrika je i MIPS (eng. *Million Instructions Per Second*).

3. *ponovljivost*

Metrika performansi je ponavljajuća ako se pri svakom izvođenju eksperimenta dobija ista vrijednost. Ova osobina implicira i da je dobra metrika deterministička.

4. *lakoća upotrebe*

Ako metriku nije lako izmjeriti, mala je vjerovatnoća da će je bilo ko i koristiti. Dalje, što je metrika teža za dobijanje, veća je vjerovatnoća da će se pogrešno i utvrditi. Jedina gora stvar od loše metrike je metrika čija vrijednost je izmjerena pogrešno!

5. *konzistentnost*

Metrika performansi je konzistentna ako je njena jedinica i precizna definicija ista u različitim sistemima, ali i na različitim konfiguracijama na istom sistemu. Ako jedinice metrike nisu konzistentne, metriku nije moguće koristiti za poređenje performansi različitih sistema. Iako se i ova karakteristika čini očiglednom, nju ne podržavaju neke često korištene metrike kao što su MIPS i MFLOPS.

6. *nezavisnost od vanjskih uticaja*

Mnogi nabavljači računarskih sistema donose odluke o tome koji sistem da kupe na osnovu poređenja rezultata koji se dobijaju korištenjem često korištenih metrika. Na osnovu toga, postoji veliki pritisak na proizvođače da dizajniraju svoje mašine tako da se dobije optimizovana vrijednost upotrebom neke određene metrike. Da bi se spriječile slične zloupotrebe, dobra metrika mora da bude nezavisna od takvih vanjskih uticaja.

2.5. **Skale mjerenja**

Skale mjerenja se dijele u četiri velike kategorije:

1. *Nominalna* (eng. *nominal*)

Spada u najniži nivo mjerenja i često se koristi sa kvalitativnim (kategoričkim) varijablama umjesto sa kvantitativnim. Kada se koristi ova skala, mjerenim objektima se određuje kojoj kategoriji pripadaju. Na osnovu klasifikacije objekata u odgovarajuću kategoriju, ova skala je i dobila naziv pošto se objektu praktično daje ime svrstavanjem. Ne postoji kvantitativan odnos između različitih kategorija. Fundamentalna osobina ove skale je ekvivalencija, pošto su svi objekti iz iste kategorije identični sa gledišta ove skale. Najčešća operacija koja se vrši unutar ove skale mjerenja je brojanje objekata u nekoj od kategorija.

Dozvoljene statističke metode su: *mode* srednja vrijednost i hi-kvadrat test.

2. *Ordinalna* (eng. *ordinal*)

Predstavlja viši nivo mjerenja u odnosu na nominalnu skalu, ali i nizak nivo izražavanja razlika između različitih kategorija. Objekti se rangiraju na osnovu toga da li posjeduju više, manje ili jednaku količinu svojstva koje se mjeri, tj. omogućava utvrđivanje da li je $A > B$, $A < B$ ili $A = B$. Međutim, ova skala ne daje odgovor na pitanje kolika je razlika između objekata ako postoji. Takođe, ova skala ne nudi referentnu vrijednost u odnosu na koju se porede ostali objekti (na primjer, svi mogu biti visoki, ali i svi mogu biti niski).

Dozvoljena statistička metoda je *median* srednja vrijednost.

3. *Intervalska* (eng. *interval*)

Ova skala predstavlja viši nivo u odnosu na ordinalnu skalu pošto posjeduje osobinu da se količinski može izraziti razlika između objekata. Naime, za razliku od ordinalne skale, na ovoj skali mjerenja postoje jednaki razmaci između susjednih podioka na skali. Na ovaj način se može osim osnovnih poređenja dodatno utvrditi i da li je $A - B = C - D$, $A - B > C - D$ ili $A - B < C - D$.

Dozvoljene statističke metode su: klasična srednja vrijednost, standardna devijacija, korelacija, regresija i ANOVA (analiza varijanse).

4. *Odnosna* (eng. *ratio*)

Ova skala predstavlja skalu mjerenja sa najvišim nivoom suptilnosti. Posjeduje sve osobine intervalske skale i, dodatno, postoji tačka apsolutne nule. Intervalska skala zbog neposjedovanja ove zadnje osobine ne dozvoljava izračunavanje odnosa (na primjer, ne može se reći da je 20 stepeni na Celsusovoj skali dva puta više od 10 stepeni). Ovo je moguće na odnosnoj skali (na primjer, na Kelvinovoj skali 20 stepeni jeste duplo više od 10 stepeni).

Dozvoljene statističke metode su sve metode podržane intervalskom skalom plus geometrijska sredina, harmonijska sredina, koeficijent varijacije i logaritmi.

2.6. Mjerne tehnike i alati

Postoji više različitih tipova metrika performansi koje su predmet mjerenja. Različite strategije za mjerenje vrijednosti tih metrika su tipično bazirane oko ideje događaja. Događaj se definiše kao predefinisana promjena stanja sistema. Primjeri događaja su referenciranje memorije, pristup disku, mrežna komunikacija, promjena u internom stanju procesora i slično.

Klasifikacija tipa događaja:

1. *event-count metrike*

Metrike koje spadaju u ovu kategoriju jednostavno broje koliko se puta određen događaj desio.

2. *secondary-event metrike*

Ovi tipovi metrika bilježe vrijednosti nekih sekundarnih parametara kada se desi određen događaj.

3. *profili*

Profil je agregaciona metrika koja se koristi za karakterizaciju sveukupnog ponašanja programa ili čitavog sistema. Tipično se koristi za identifikovanje gdje program ili sistem provodi svoje vrijeme izvršavanja.

2.6.1. Strategije mjerenja

Navedena klasifikacija događaja može da bude korisna pri utvrđivanju specifične strategije za mjerenje željene metrike pošto su različiti tipovi alata za mjerenje pogodni za različite tipove događaja. Alati za mjerenje se mogu kategorizovati na bazi fundamentalne strategije koja se koristi za utvrđivanje vrijednosti metrike koja se mjeri. Jedan od bitnih aspekata svake strategije mjerenja je koliko to mjerenje utiče na sistem koji se mjeri. U strategije mjerenja spadaju:

1. *event-driven*

Ova strategija bilježi informacije neophodne za izračunavanje metrike performansi kad god se desi određen događaj. Najjednostavniji alat za *event-driven* mjerenja je brojač koji direktno broji koliko puta se desio specifični događaj. Ova mjerenja su pogodna za događaje koji se rijetko pojavljuju jer bi u suprotnom znatno uticala na sistem koji se mjeri.

2. *tracing*

Strategija bilježenja traga je slična event-driven strategiji, ali ovdje se umjesto bilježenja da se događaj desio, snima dio stanja sistema u svrhu jedinstvenog identifikovanja događaja. Ova strategija očigledno zahtijeva mnogo veće

kapacitete za bilježenje podataka nego *event-driven* strategija. Dodatno, vrijeme potrebno da se snimi stanje sistema može značajno izmijeniti standardno izvršavanje programa koji se mjeri.

3. *sampling*

Za razliku od *event-driven* strategije, ova strategija bilježi stanje sistema nakon fiksnih vremenskih intervala bez obzira da li se neki događaj desio ili ne. Cilj je na osnovu skupljenih podataka odrediti metriku od interesa. Opisana logika dovodi do činjenice da je opterećenje sistema mjerenjem nezavisno od toga koliko puta se desi neki događaj. Pošto je vjerovatno da neće svaki događaj biti zabilježen, ova strategija kao rezultat nudi statistički pogled na ponašanje sistema.

4. *indirektna mjerenja*

Ova strategija se mora primijeniti kada metrika koja se želi utvrditi nije direktno dostupna. Tada se mora pronaći neka druga, dostupna, metrika iz koje se može izvesti željena metrika. Uspjeh indirektnih mjerenja se u većini slučajeva zasniva na kreativnosti analitičara koji vrši mjerenje.

Sumiranje osobina prve tri tehnike dato je u tabeli 2.3.

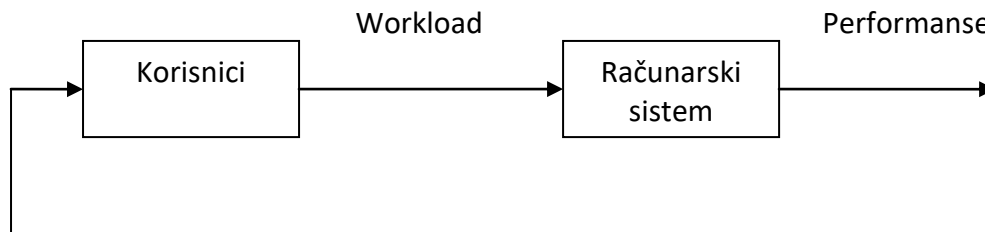
Tabela 2.3 – Strategije mjerenja

	Brojanje događaja	<i>Tracing</i>	<i>Sampling</i>
Rezolucija	Tačan broj	Detaljne informacije	Statistički sažetak
Opterećenje	Malo	Visoko	Konstantno
Uznemiravanje	Zavisí od broja događaja	Visoko	Fiksno

Za skupljanje više informacija o programu, potrebno je što više tačaka u kojima se vrši mjerenje. Međutim, problem nastaje pošto mjerenje mijenja sistem koji se mjeri pa samim tim i podaci koji se dobiju gube na pouzdanosti. Da bi situacija bila gora, uznemiravanja sistema usljed mjerenja su nelinearna i neaditivna. Nelinearnost znači da udvostručavanje mjernih tačaka u programu ne znači neminovno dvostruki uticaj na njegove performanse. Neaditivnost znači da dodavanje novih mjernih tačaka može da poništi uznemiravanje dosadašnjih, ali i da ga poveća. Činjenica je da mjerenje mijenja sistem i da to više nije sistem koji se želi mjeriti. Uvijek se mora voditi računa o tome da ta uznemiravanja sistema nisu prevelika.

2.7. Workload

Workload predstavljaju program, podaci i komande na osnovu kojih sistem produkuje output sa određenim nivoom performansi. Atributi *workload*-a se nazivaju *workload* parametri. Bilo koja modifikacija *workload*-a u principu produkuje različite performanse. Odnos *workload*-a, korisnika, računarskog sistema i performansi prikazan je na slici 2.2.



Slika 2.2 – Odnos *workload*-a, korisnika, rač. sistema i performansi

Workload se definiše kao set ulaza (programi, podaci, komande) koje sistem prima iz okruženja. Mjerenje performansi ima značaj samo ako je *workload* pravilno specifikovan. U glavne karakteristike *workload*-a spadaju: reprezentativnost, fleksibilnost, jednostavnost konstrukcije, kompaktnost, troškovi upotrebe, systemska nezavisnost, mogućnost reprodukcije i kompatibilnost sa sistemom ili modelom.

Workload se dijeli na pravi i sintetički. Pravi *workload* je onaj koji se posmatra na sistemu koji se koristi za normalne operacije. Nije ponovljiv i samim tim nije pogodan da se koristi kao testni *workload*. Za razliku od njega, sintetički *workload* ima slične karakteristike kao pravi, ali se dodatno može ponavljati na kontrolisan način. On se većinom koristi za testiranja.

Glavni razlog za korištenje sintetičkog *workload*-a je činjenica da on predstavlja reprezentaciju modela pravog *workload*-a. Dodatni razlozi za njegovu upotrebu su:

- nisu potrebni podaci iz realnih sistema, koji mogu biti ogromnih količina i sadržavati osjetljive podatke,
- može lako da se modifikuje bez uticaja na operisanje sistema koji se testira,
- portabilan je, tj. lako se prenosi na druge sisteme pošto je male veličine,
- ima ugrađene mogućnosti za mjerenja.

Sljedeći tipovi *workload*-a se koriste za poređenje računarskih sistema:

1. instrukcije sabiranja

U početku je CPU na računarima bio najskuplja komponenta, a najčešća operacija koju su obavljali bila je sabiranje. Mjerenje performansi računara svelo se na mjerenje performansi CPU-a. Za procesore koji brže sabiraju

smatralo se da imaju bolje performanse. Ideja je bila da se kao *workload* upotrebi što je više moguće operacija sabiranja. Naravno, daljim razvijanjem računarskih sistema i pojavom novih instrukcija, ovaj sistem se počeo smatrati prevaziđenim.

2. instrukcioni miksevi

Instrukcioni miks predstavlja specifikaciju različitih instrukcija koje procesor izvršava uparenih sa frekvencijom njihovog korištenja. Sa poznavanjem vremena izvršavanja instrukcija, moguće je izračunati prosječno vrijeme izvršavanja odgovarajućeg miksa i onda iskoristiti ta vremena u svrhu poređenja različitih procesora. Jedinice za poređenje su MIPS (eng. *Millions of Instructions Per Second*) i MFLOPS (eng. *Millions of Floating-point Ops Per Second*). Najpoznatiji instrukcioni miks, prikazan na slici 2.3, napravio je Gibson 1959. godine.

1. Load and Store	31.2
2. Fixed-Point Add and Subtract	6.1
3. Compares	3.8
4. Branches	16.6
5. Floating Add and Subtract	6.9
6. Floating Multiply	3.8
7. Floating Divide	1.5
8. Fixed-Point Multiply	0.6
9. Fixed-Point Divide	0.2
10. Shifting	4.4
11. Logical, And, Or	1.6
12. Instructions not using registers	5.3
13. Indexing	<u>18.0</u>
	100.0

Slika 2.3 – Gibsonov instrukcioni miks

Današnji računari imaju dosta kompleksnije klase instrukcija koje nisu predstavljene u miksevima. Takođe, vrijeme izvođenja instrukcija uveliko zavisi od moda adresiranja, procenta pogađanja keša, efikasnosti *pipeline*-a i interferencije sa drugim uređajima. Uprkos navedenim nedostacima, instrukcioni miksevi ipak pružaju jedan broj koji se može koristiti u poređenju sa drugim računarima slične arhitekture. Bitno je naglasiti da ovaj način benčmarkinga karakterizuje samo procesor, a ne cijeli računarski sistem.

3. *kerneli*

Sa pojavom *pipeline*-a, keširanja instrukcija i raznih mehanizama za prevođenje adresa, vremena izvršavanja instrukcija su postala krajnje varijabilna. Jedna instrukcija više nije mogla da se posmatra u izolaciji. Umjesto toga, postalo je pogodnije da se posmatra skup instrukcija koje predstavljaju neku uslugu koju nudi procesor. Pokušao se filtrirati skup takvih usluga sa idejom da se najčešća uzme kao *workload*. Najčešća usluga je dobila naziv kernel. Očigledno je da kerneli predstavljaju generalizaciju instrukcionog miksa. Neki od najpoznatijih kernela su traženje inverzne matrice, sortiranje, pretraživanje stabla i sl. Glavni nedostatak kernela je da, kao i instrukcioni miksevi, ne predstavljaju cijeli računarski sistem, već samo procesor.

4. *sintetički programi*

Ulazno/izlazne operacije i usluge operativnih sistema su postali važan dio pravih *workload*-a, a kerneli ih nisu podržavali. Početni pokušaji mjerenja I/O performansi su doveli do kreiranja jednostavnih testnih petlji koje iniciraju određeni broj sistemskih poziva ili I/O usluga. Na ovaj način se moglo izračunati prosječno CPU vrijeme i potrebno vrijeme za svaki od sistemskih poziva. Ovakvi programi su pisani u jezicima višeg nivoa. Prvu takvu testnu petlju je napravio Buchholz 1969. godine i nazvao je sintetičkim programom. Glavna prednost ovakvih *workload*-a je njihova mogućnost brzog razvijanja i isporuke različitim proizvođačima. Razvijeni programi se mogu lako modifikovati i pokrenuti na različitim sistemskim platformama. Takođe, oni imaju ugrađene mogućnosti mjerenja. Kada se proces mjerenja jednom razvije, on je automatizovan i može se ponavljati više puta. Glavni nedostatak sintetičkih programa je njihova mala veličina, pa oni ne prave reprezentativna referenciranja memorije i diska.

5. *benčmark programi*

Benčmarking predstavlja proces poređenja dva ili više sistema korištenjem *workload*-a. Korištenjem benčmark programa mjere se skoro svi resursi u sistemu uključujući procesore, I/O uređaje, mreže i baze podataka. Takođe, termin benčmark je skoro postao sinonim sa terminom *workload*.

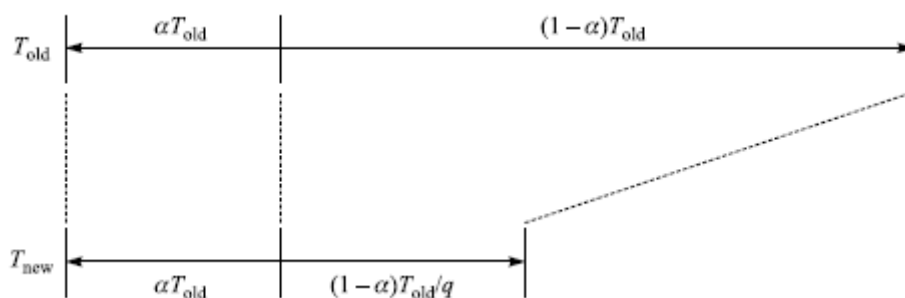
2.7.1. Strategije benčmarkinga

Većina benčmark programa baziraju mjerenje performansi na vremenu potrebnom za njegovo izvršavanje. Međutim, postoje i druge strategije. Tri strategije korištenja benčmark programa u svrhu mjerenja performansi računarskih sistema su:

1. mjerenje vremena potrebnog za izvođenje fiksne količine operacija,
2. mjerenje količine operacija koje su izvedene za fiksnu količinu vremena,
3. dozvoljavanje da i vrijeme i količina operacija variraju.

Koncept fiksiranja količine operacija koje se izvode vodi do gornje granice koja govori koliko se sveukupne performanse bilo kojeg računarskog sistema mogu poboljšati promjenama samo jedne komponente sistema. Ideju je izložio Gene Amdahl 1967. godine. Iako njegov rad nije sadržavao formalnu analizu, koncept je postao poznat kao Amdahl-ov zakon.

Amdahl-ov zakon pokazuje da je poboljšanje ukupnih performansi sistema ograničeno dijelom sistema koji nije zahvaćen promjenama. Na slici 2.4 su prikazani staro vrijeme izvršavanja T_{old} i poboljšano vrijeme izvršavanja T_{new} .



Slika 2.4 – Amdahl-ov zakon

Pošto su promjene u sistemu poboljšale performanse samo nekih njegovih dijelova, ostalo je mnogo dijelova koji nisu zahvaćeni uticajem ovih promjena. Ako se sa α označi procenat dijelova koji nisu zahvaćeni promjenama, onda se T_{new} može podijeliti u dvije komponente. Prva komponenta, αT_{old} , predstavlja vrijeme izvršavanja dijela programa koji nije zahvaćen promjenama. Druga komponenta predstavlja vrijeme izvršavanja komponenti čije su performanse poboljšane faktorom q . To vrijeme je jednako $(1 - \alpha) T_{old} / q$. Na osnovu ovog razmatranja može se izvesti formula za ukupno ubrzanje sistema:

$$S = \frac{T_{old}}{T_{new}} = \frac{T_{old}}{\alpha T_{old} + (1 - \alpha) T_{old} / q} = \frac{1}{1/q + \alpha(1 - 1/q)}.$$

Ova jednačina je zanimljiva pošto se njenim korištenjem može izračunati ukupno ubrzanje sistema prouzrokovano nekom promjenom, pod pretpostavkom da su q i α poznati. Dalje, može se izračunati šta bi se desilo da promjene prouzrokuju ogromno poboljšanje, tako da $q \rightarrow \infty$.

Lako je pokazati da, ako $q \rightarrow \infty$, u tom slučaju $(1 - \alpha) T_{old} / q \rightarrow 0$. Na osnovu toga slijedi da je ukupno ubrzanje ograničeno sa $1/\alpha$:

$$\lim_{q \rightarrow \infty} S = \lim_{q \rightarrow \infty} \frac{1}{1/q + \alpha(1 - 1/q)} = \frac{1}{\alpha}.$$

Ovaj rezultat pokazuje da, bez obzira koliko performanse neke komponente sistema budu poboljšane, ukupne performanse su ograničene operacijama koje moraju da se

izvedu a nisu pod uticajem promjena koje su donijele poboljšanje. Na primjer, idealno ubrzanje koje može da se ostvari u paralelnim računarskim sistemima sa p procesora je p . Međutim, ako 10% programa ne može da se izvrši paralelno, maksimalno ukupno ubrzanje može da bude $1/0.1 = 10$ puta, čak i ako je dostupan beskonačan broj procesora. Ograničenje u vidu činjenice da se 10% programa mora izvršiti sekvencijalno limitira ukupno poboljšanje performansi koje može da se ostvari.

3. SIMULACIJA

Često postoji želja da se predvide performanse nekog računarskog sistema prije nego što se on zapravo izgradi. Pošto prava mašina još uvijek ne postoji, performanse takvog sistema se očigledno ne mogu mjeriti direktno. Najbolje što se u tom trenutku može uraditi je simuliranje nekih važnih aspekata sistema. Zatim se može probati izvesti zaključak o tome kako će se sistem ponašati kada se zapravo i izgradi. Simulacija takođe može biti pogodna za ispitivanje performansi postojećeg sistema koje ne mogu da se izmjere ni direktno, ali ni indirektno.

Kada sistem još ne postoji, neophodno je napraviti mnoge pretpostavke o sistemu prije nego što se on može simulirati. Pojednostavljjuće pretpostavke je takođe neophodno utvrditi i kada se simulira postojeći sistem pošto je nemoguće modelovati svaki postojeći detalj sistema.

Prednost simulacije je što je mnogo manje košta nego da se izgradi prava mašina. Simulacija je takođe mnogo fleksibilnija pošto se lako mogu mijenjati razni parametri sistema tokom izvođenja simulacije, što je nemoguće u realnom sistemu.

Nedostatak simulacije je naravno činjenica da se ne mjeri realan sistem. Uvijek postoji *trade-off* između tačnosti simulacije i vremena potrebnog da se napravi simulator i izvrše željene simulacije. Utvrđivanje nivoa detalja sistema koji se simulira predstavlja veoma izazovan zadatak.

3.1. Vrste modela

Za predstavljanje realnog sistema koriste se različiti modeli, kao što su: mentalni (misaoni), verbalni, strukturni, fizički, analogni, matematički, simulacioni, računarski i razni drugi modeli. Često se dijele na materijalne (model hemijske strukture molekula ili model aviona) i simboličke modele (matematički, konceptualni, računarski, simulacioni i dr).

Mentalni modeli su strukture koje ljudski mozak neprekidno konstruiše kako bi bio u stanju da poveže niz činjenica sa kojima se čovjek susreće, a potom na osnovu toga da djeluje. Takvi modeli omogućavaju, na primjer, razumijevanje fizičkog svijeta, komunikaciju među ljudima i planiranje akcija. Istraživanje prirode i razvoj tehnologije doveli su do stvaranja opipljivijih i formalnijih vrsta modela, prvenstveno zbog potrebe da se omogući opisivanje složenih fenomena kako bi se oni mogli preciznije rješavati. U mentalnim procesima, različiti koncepti koje jedinka posjeduje postavljaju se u različite nove odnose. Koncept koji jedinka nosi nisu realan svet, a zaključci koje stvara na osnovu njih su zaključci formulisani pomoću modela. Verbalni modeli su direktna posljedica mentalnih modela i predstavljaju njihov izraz u govornom jeziku, a uobičajeno se predstavljaju u pisanom obliku. Verbalni modeli spadaju u klasu neformalnih modela.

Fizički modeli predstavljaju umanjene modele realnog sistema, koji se ponašaju na isti način kao i njihovi originali. Uglavnom se prave na osnovu teorije sličnosti ili, u boljem slučaju, na osnovu fizičkih zakona sličnosti.

Ukoliko su veze između objekata modela opisane matematičkim (numeričkim) relacijama, tada se radi o matematičkim modelima. Matematički model je iz klase apstraktnih. Kod formulisanja matematičkog modela polazi se od verbalnog modela koji se transformisanjem dovodi u stanje koje se može opisati matematičkim jezikom. Ova klasa modela ima široku primjenu, naročito u nauci i inženjerskim disciplinama.

Odavno je uočena činjenica da se različiti fizički objekti mogu opisivati istim matematičkim modelom. Između dva fizička modela koji imaju iste matematičke modele, kaže se da postoji matematička analogija. Istovjetnost matematičkih modela dva objekta pruža mogućnost da jedan od fizičkih objekata bude korišten za analizu matematičkog modela drugog objekta. Fizički objekat koji se koristi za analizu matematičkog modela drugog objekta, a sa kojim ima isti ili sličan matematički model, naziva se analogni model. Prema tome, između fizičkog objekta koji se ispituje i analognog modela postoji matematička analogija (analogija u ponašanju).

Konceptualni modeli se stvaraju na osnovu predstave o strukturi i logici rada sistema ili problema koji se modelira i prikazuju se u obliku čije je značenje precizno definisano (na primjer, dijagrami sa tačno definisanim simbolima). Njihova posebna važnost proističe iz činjenice da oni predstavljaju osnovu za izradu računarskih modela. Ovi se modeli često nazivaju i strukturni, pošto u grafičkom obliku ukazuju na strukturu (relativno stabilan odnos elemenata) posmatranog sistema. Takav prikaz omogućuje da se modeli vizualizuju i na taj način postaju zgodno sredstvo za komunikaciju među ljudima koji sa njima rade. Pored toga, grafički prikaz modela obezbjeđuje relativno jednostavan prikaz složenih sistema i nesumnjivo znači važan korak ka boljem razumijevanju sistema koji se modelira.

Računarski (simulacioni) modeli su prikaz konceptualnih modela u obliku programa za računar. U tom obliku modeli postaju sredstvo kojim se može efikasno analizirati rad modela u različitim spoljnim uslovima i sa različitim unutrašnjim parametrima i tako dobiti uvid u ponašanje sistema koji model opisuje. Kao sredstvo za izražavanje, računarski modeli koriste programske jezike i stoga su blisko vezani za razvoj računarskih nauka.

3.1.1. Neformalni i formalni modeli

Neformalni opis modela daje osnovne pojmove o modelu i, mada se teži njegovoj potpunosti i preciznosti, on to najčešće nije. Prilikom izgradnje neformalnog opisa, upravo radi eliminisanja pomenutih nedostataka, vrši se podjela na objekte, opisne promjenljive i pravila interakcije objekata.

Objekti su dijelovi iz kojih je model izgrađen; opisne promjenljive (preko vrijednosti koje uzimaju) opisuju stanja u kojima se objekti nalaze u određenim vremenskim trenucima (parametri pomoću kojih se opisuju konstantne karakteristike modela su takođe uključeni u opisne promjenljive); pravila interakcije objekata definišu kako objekti modela utiču jedan na drugi u cilju promjene njihovog stanja. Treba naglasiti da ne postoji pravilo za izbor objekata, opisnih promjenljivih i pravila interakcije. Taj izbor je prepušten onome ko opisuje i izgrađuje model i treba ga vršiti tako da se dobije validan model.

Neformalni opis modela priprema se dosta brzo i lako, ali on najčešće nije konzistentan i jasan, naročito kada su u pitanju složeni modeli. Anomalije koje se javljaju prilikom neformalnog opisa modela najčešće se mogu opisati na sljedeći način:

- nekompletan opis modela,
- nekonzistentan opis modela,
- nejasan opis modela.

Ukoliko model ne sadrži sve situacije koje mogu da nastupe, tada je opis nekompletan. Ukoliko su u opisu modela za istu situaciju predviđena dva ili više pravila čijom se primjenom dobijaju kontradiktorne akcije, tada je opis nekonzistentan. Na kraju, ako u jednoj situaciji treba obaviti dvije ili više akcija, a pri tome nije definisan njihov redoslijed, tada je opis modela nejasan.

S druge strane, savremena metodologija modeliranja u velikoj mjeri se oslanja na određene konvencije u komuniciranju, zvane formalizmima. Formalizam specificira klasu posmatranih objekata na nedvosmislen i generalan način, korištenjem konvencija i pravila.

Formalni opis modela treba da obezbjedi veću preciznost i potpunost u opisivanju modela, a ponekad omogućava i da se formalizuje postupak ispitivanja nekompletnosti, nekonzistentnosti i nejasnosti. Ono što je, ipak, najznačajnije jeste činjenica da uvođenje formalizama u metodologiju modeliranja omogućava da se sva pažnja posmatrača usmjeri na one karakteristike objekata koje su od najvećeg značaja za njegovo istraživanje, tj. da se koriste apstrakcije.

Ako se zasebno posmatra oblast naučnog rada, može se lako zapaziti da se ona u velikoj mjeri, a vjerovatno i u cjelini, sastoji od formalizacija i izgradnje modela. Formalni model ili apstraktno predstavljanje često se spominje kao ključ uspjeha interakcije čovjek - realni svijet koja se zasniva na "naučno-inžinjskom" pristupu. Važno je napomenuti da ovako posmatrani tip interakcije čovjek-realni sistem obuhvata dva različita koraka. Prvo, postoji uvijek faza izgradnje modela ili formalizacija, gde se kao rezultat javlja model realnog sistema. U drugoj fazi, formalni model se analizira i koristi, a na osnovu dobijenih rezultata donose se odluke koje obezbjeđuju efikasnije upravljanje samim realnim sistemom.

Na kraju, treba naglasiti i činjenicu da ne postoje striktna pravila vezana za proces izrade modela, već veliko značenje ima zdrav razum, sposobnost apstrakcije, sistematičnost i iskustvo. Dugo iskustvo velikog broja ljudi koji su se bavili modeliranjem dovelo je do nekih opštih preporuka pri izradi modela:

1. Granica sistema sa okolinom mora biti odabrana tako da sistem, odnosno njegov model, obuhvata samo fenomene od interesa. Okolina sistema modelira se tako da se ne opisuju detalji fenomena i uzročna veza među njima, već se daje samo njihov sažeti prikaz.
2. Modeli ne smiju biti suviše složeni niti detaljni, već treba da sadrže samo relevantne elemente sistema – suviše složene i detaljne modele gotovo nije moguće vrednovati ni razumjeti. Model ne smije suviše da pojednostavi problem, npr. izbacivanjem važnih promjenljivih potrebnih za adekvatan opis sistema ili suviše velikim stepenom agregacije komponenti sistema.
3. Model je razumno rastaviti na više dobro definisanih i jednostavnih modula sa tačno određenom funkcijom, koje je lakše i izgraditi i provjeriti.
4. U razvoju modela preporučuje se korištenje neke od provjerenih metoda za razvoj algoritama i programa. Tako se, na primjer, u metodi *top-down* polazi od prve verzije modela koja se sastoji od nekoliko modula sa više funkcija, zatim se u sljedećem koraku ti moduli rastavljaju na nekoliko jednostavnijih modula, itd., sve dok se ne dostigne željeni stepen detaljnosti modela. Pri tome je moguće razumjeti model i njegove module u svim fazama razvoja modela.
5. Potrebna je provjera logičke i kvantitativne ispravnosti modela, i to kako pojedinačnih modula tako i cijelog modela. Kod modela koji uključuju slučajne promjenljive to znači i primjenu odgovarajućih statističkih tehnika.

3.2. Računarska simulacija

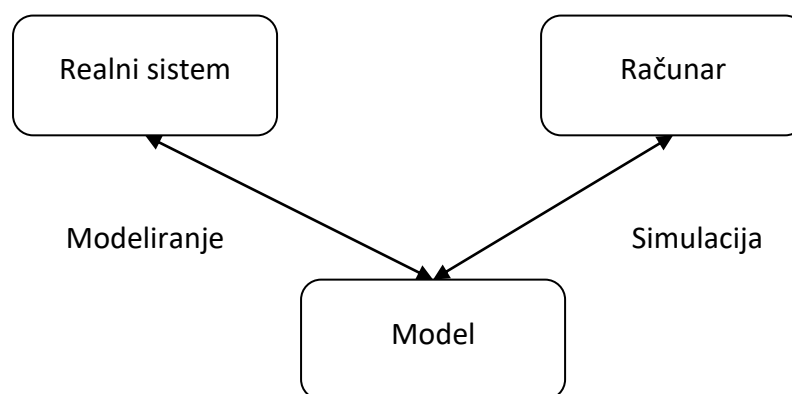
Da bi model bio koristan, od suštinske je važnosti to da se za dati ograničeni skup njegovih opisnih promjenljivih, njegovo ponašanje može odrediti na praktičan način: analitički, numerički ili putem eksperimenta, gdje se za izvjesne, uglavnom slučajne ulaze, posmatraju odgovarajući izlazi. Ovaj posljednji proces naziva se simulacija.

Riječ simulacija u svakodnevnoj upotrebi može da označi veći broj različitih aktivnosti, kao na primjer: složene video igre, ispitivanje uticaja brojnih faktora na let novih modela aviona, dio eksperimenta u socio-psihološkim istraživanjima itd. Kada riječ koriste računarski stručnjaci, organizatori, menadžeri ili statističari, obično pod simulacijom podrazumijevaju proces izgradnje apstraktnih modela za neke sisteme ili podsisteme realnog svijeta i obavljanje većeg broja eksperimenata nad njima. Posebno je interesantan slučaj kada se ti eksperimenti odvijaju na računaru. Tada je riječ o računarskom modeliranju i simulaciji.

3.2.1. Modeliranje i simulacija

Savremeno modeliranje nezamislivo je bez računara. U modeliranju računari se koriste u dvije svrhe: u razvoju modela i u izvođenju proračuna na osnovu stvorenog modela (npr. proračuna ponašanja modela u vremenu kod simulacionih modela). Na taj način, modeliranje pomoću računara postaje disciplina kojom se mogu adekvatno i efikasno prikazivati složeni sistemi i oblikovati i ispitivati njihovo ponašanje.

Izraz modeliranje i simulacija izražava složenu aktivnost koja uključuje tri elementa: realni sistem, model i računar. Ova se aktivnost na uprošten način može predstaviti dijagramom na slici 3.1.



Slika 3.1 – Relacije modeliranja i simulacije

Pod realnim sistemom podrazumijeva se uređen, međuzavisan skup elemenata koji formiraju jedinstvenu cjelinu i djeluju zajednički kako bi ostvarili zadati cilj ili funkciju, bez obzira da li se radi o prirodnom ili vještačkom sistemu, i takođe, bez obzira da li taj sistem u posmatranom trenutku postoji ili se njegovo postojanje planira u budućnosti. Realni sistem je izvor podataka o ponašanju, a ovi se podaci javljaju u obliku zavisnosti $X(t)$, gdje je X bilo koja promjenljiva koja interesuje istraživača, a t je vrijeme mjereno u odgovarajućim jedinicama. Drugim riječima, realni sistem se može posmatrati kao izvor podataka za specifikaciju modela.

Model, kao i svaki realni sistem, ima svoje objekte koji se opisuju atributima ili promjenljivama. On je apstraktni prikaz sistema i daje njegovu strukturu, njegove komponente i njihovo uzajamno djelovanje. S obzirom da se za simulaciju najčešće koristi računar (razmatra se samo taj slučaj), to se pod modelom može podrazumijevati skup instrukcija (program) koji služi da se generiše ponašanje simuliranog sistema (vremenska serija vrijednosti promjenljivih simuliranog sistema). Ponašanje modela ne mora da bude u potpunosti jednako ponašanju simuliranog sistema, već samo u onom domenu koji je od interesa.

Računar, kao treća komponenta ove aktivnosti, predstavlja uređaj sposoban za izvršenje instrukcija modela, koje na bazi ulaznih podataka generišu razvoj modela u

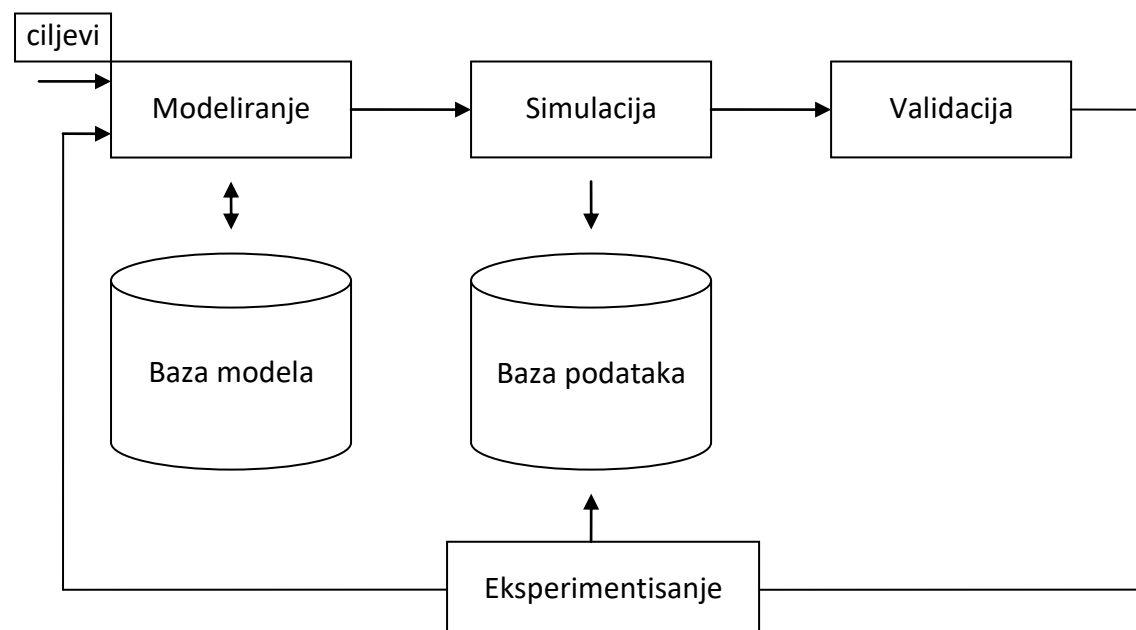
vremenu. Računari, uz različite metode i programske alate, omogućavaju pogodan ambijent za stvaranje složenih modela i efikasan rad nad njima.

Međutim, pored ovih elemenata, pažnju treba usmjeriti i na otkrivanje i definisanje relacija koje postoje između njih. Modeliranje je proces kojim se uspostavlja veza između realnog sistema i modela, dok je simulacija proces koji uspostavlja relaciju između modela i računara.

Relacija modeliranja odnosi se na validnost modela. Validnost ili valjanost modela opisuje koliko vjerno jedan model predstavlja simulirani sistem. Proces utvrđivanja stepena slaganja podataka o realnom sistemu sa podacima modela naziva se validacija modela. Proces validacije je veoma značajan, jer se na osnovu njega donose odluke o upotrebljivosti rezultata simulacije, izmjeni modela, izmjeni podataka (ulaznih promjenljivih, parametara), daljem nastavku simulacije, ponavljanju simulacije, itd.

Relacija simulacije odnosi se na provjeru da li simulacioni program vjerno prenosi model na računar kao i na tačnost kojom računar izvršava instrukcije modela. Prije poređenja stvarnih podataka sa podacima koje generiše računar (simulator), mora se utvrditi tačnost, odnosno korektnost simulatora. Proces procjene korektnosti simulatora naziva se verifikacija.

Na slici 3.2 prikazane su aktivnosti procesa modeliranja i simulacije sa bazom modela kao centralnim objektom. Procesom modeliranja se upravlja na osnovu ciljeva koji se generišu van granica sistema. Svaki novi cilj inicira aktivnost sinteze modela. Pri sintezi modela se koristi raspoloživo znanje iz baze modela i baze podataka. Ove baze čuvaju i organizuju prikupljene podatke o realnom sistemu. Faze simulacije (eksperimentisanje sa modelom) i validacije slijede fazu izgradnje modela.



Slika 3.2 – Modeliranje i simulacija sa bazom podataka

Validacija vodi novom eksperimentisanju nad realnim sistemom i može da zahtijeva dodatne modifikacije ili čak odbacivanje i reinicijalizaciju prvobitnog modela. U tom procesu, kao rezultat nedostatka podataka u bazi znanja mogu se formulisati novi ili izmijeniti postojeći ciljevi. Na kraju, kao rezultat javlja se jedan ili više modela koji vode ka ispunjenju eksternih ciljeva (ukoliko proces ne "upadne" u petlju iz koje ne može da izađe). Kreirane modele koristi donosilac odluke. Pored toga, oni se mogu memorisati u bazi modela i koristiti u nekoj narednoj fazi aktivnosti.

3.3. Karakteristike simulacionog modeliranja

Računarska simulacija je proces rješavanja problema koji se tiče predviđanja i određivanja budućih stanja realnog sistema na osnovu proučavanja računarskog modela tog sistema. Drugim riječima, računarska simulacija se zasniva na ideji eksperimentisanja sa modelom realnog sistema na računaru, tokom vremena.

Simulacioni eksperimenti najčešće se izvode sa ciljem da se prikupe određene informacije, čije bi dobijanje putem eksperimenta nad samim realnim sistemom bilo nepraktično ili suviše skupo. Te informacije kasnije se transformišu u odluke značajne za upravljanje realnim sistemom koji je predmet simulacionog modeliranja. Cilj simulacije je proučavanje ponašanja sistema koji se simulira, ali i utvrđivanje kako bi se isti sistem ponašao kada bi na njega delovao neki drugi skup promjenljivih okolnosti (ulaznih veličina i parametara).

Za razliku od analitičkih modela koji sveukupno ponašanje sistema tretiraju direktno, simulacioni modeli prikupljaju podatke o promjenama stanja sistema i izlaza, fokusirajući se na ponašanje individualnih komponenti sistema. Značaj simulacionih modela proističe iz činjenice da se samo mali broj kompleksnih realnih sistema može adekvatno opisati preko analitičkih jednačina.

Kod primjene simulacionog modeliranja, ne može se dobiti rješenje u analitičkom obliku, u kojem su zavisne promjenljive funkcije nezavisnih promjenljivih, već se rješenje problema dobija eksperimentisanjem nad modelom. Pri tome, simulacioni eksperimenti daju kao rezultat skup tačaka, tj. vrijednosti zavisnih promjenljivih za pojedine vrijednosti nezavisnih promjenljivih (vrijeme). Zbog slučajnog karaktera promjenljivih modela, dobija se čak i više različitih vrijednosti zavisnih promjenljivih za istu vrijednost nezavisnih promjenljivih, tj. eksperimenti daju određeni uzorak vrijednosti zavisnih promjenljivih. Pri tome, planiranje i analiza simulacionih eksperimenata zahtijevaju statistički pristup.

Simulacioni modeli najčešće su modeli dinamičkih sistema, tj. sistema koji se mijenjaju u vremenu, s obzirom na to da su istraživači, u većini slučajeva, zainteresovani za simulaciju modela dinamičkih sistema. Ovi su modeli uglavnom dati u obliku konceptualnih (strukturnih) i računarskih modela. Postoji, međutim, i grupa stacionarnih problema za čije je rješavanje simulacija takođe interesantna. Najčešće, ali ne i

generalno, simulacioni modeli su modeli sistema koji se ne mogu opisati niti rješavati matematičkim sredstvima. Njihove tipične primjene su u oblastima inženjerstva, menadžmenta i ekonomije, ali sve više i u medicini, biologiji i drugim prirodnim naukama.

Izgradnja i korištenje simulacionih modela je proces koji zahtijeva znatnu vještinu i dobro poznavanje brojnih naučnih disciplina. Po svojoj prirodi on je blisko vezan za alate i tehnike računarskih nauka i systemske analize. Takođe, proces simulacije se oslanja i na metode operacionih istraživanja i numeričke analize. Zbog postojanja slučajnih promjenljivih u simulacionim modelima, često se koriste i pristupi teorije vjerovatnoće i statistike. Eksperimentalna priroda metoda rješavanja problema u simulacionom modeliranju ima analogije i sa empirijskom prirodom istraživanja u prirodnim naukama, budući da se u oba područja koristi planiranje, izvođenje i analiza eksperimenata radi razumijevanja rada sistema koji se ispituje, odnosno provjere postavljenih hipoteza. No, bez obzira na svoj multidisciplinarni pristup, simulacija danas predstavlja koherentnu i dobro razvijenu oblast i s pravom nosi naziv zasebne naučne discipline.

3.4. Potreba za simulacijom

Može se postaviti pitanje zbog čega se uopšte jedan sistem (simulirani sistem) zamjenjuje modelom, a zatim vrši simulacija. Postoji više razloga, ali su najvažniji sljedeći:

- Eksperiment nad realnim sistemom može da bude skup ili čak nemoguć (na primjer, u ekonomskim sistemima ili hemijskim postrojenjima).
- Analitički model nema analitičko rješenje (npr. složeniji modeli masovnog opsluživanja).
- Sistem može da bude suviše složen da bi se opisao analitički (na primjer, sistemom diferencijalnih jednačina).

Pored ovih, postoji još nekoliko značajnih razloga:

- Eksperimentisanje sa realnim sistemom, čak i ako se zanemare drugi aspekti, uglavnom je neisplativo ili suviše složeno. Modeliranje, s druge strane, može da ukaže na to da li je dalje ulaganje u eksperiment ekonomski opravdano ili ne.
- Izgradnja modela i simulacija ponekad imaju za cilj da se shvati funkcionisanje postojećeg sistema čija je struktura nepoznata i ne može joj se prići.
- Prilikom iznalaženja optimalnog funkcionisanja nekog sistema, uobičajeno je da se mijenjaju razni parametri. Često je to neizvodljivo sa realnim sistemom, bilo zato što takvog sistema uopšte nema (tek ga treba graditi) ili zato što bi takav eksperiment bio preskup. Tada su gradnja modela i njegova simulacija moguće rješenje.

- Ponekad treba simulirati uslove pod kojima nastupa razaranje sistema. Naravno, razaranje realnog sistema najčešće nije dopustivo. U takvim prilikama, simulacija predstavlja pravo rješenje.
- Vrijeme može da bude vrlo jak razlog da se pribjegne simulaciji. Pri simulaciji, vrijeme se može sažeti. To je značajno kod simulacije dugotrajnih procesa. U drugim slučajevima, vrijeme se može znatno produžiti. Na primjer, sa ciljem da se prati postepeno odvijanje vrlo brzih procesa, što je u realnom sistemu nemoguće.
- Kada se vrši realni eksperiment, uvijek postoji izvjesna greška pri mjerenju usljed nesavršenosti mjernih uređaja. Pri simulaciji ove greške nema. Postoji samo greška "zaokruživanja" usljed konačne dužine riječi u računaru, ali se ona sa malo truda može učiniti zanemarljivom.
- Ponekad je poželjno zaustaviti dalje odvijanje eksperimenta, kako bi se ispitale vrijednosti svih promjenljivih u tom trenutku. Ovo je teško moguće u realnom sistemu.

3.5. Prednosti i nedostaci simulacije

Bavljenje simulacionim metodama i tehnikama zahtjeva, između ostalog, i poznavanje njenih prednosti i nedostataka. Kao osnovne prednosti korištenja simulacije navode se sljedeće:

- Jednom izgrađeni model može se višestruko koristiti za analizu predloženih planova ili politika.
- Simulacione metode mogu se koristiti kao pomoć kod analize, čak i ako su ulazni podaci na neki način nepotpuni.
- Čest je slučaj da se simulacioni podaci mogu mnogo jeftinije dobiti od sličnih podataka iz realnog sistema.
- Simulacione metode lakše je primijeniti nego analitičke metode. Stoga je krug potencijalnih korisnika simulacionih metoda znatno širi.
- Analitički modeli uglavnom zahtijevaju više pojednostavljujućih pretpostavki koje ih čine matematički prilagodljivim. Simulacioni modeli takva ograničenja nemaju. Sa analitičkim modelima, najčešće se može izračunati jedino ograničeni broj mjerljivih karakteristika sistema, dok kod simulacionih modela generisani podaci mogu da se koriste za procjenu bilo koje shvatljive i mjerljive karakteristike.
- U nekim slučajevima, simulacija je jedino sredstvo za rješavanje odgovarajućeg problema.

- Moguće je opisati i rješavati složene dinamičke probleme sa slučajnim promjenljivima koji su nedostupni matematičkom modeliranju.

U osnovne nedostatke korištenja simulacije ubrajaju se sljedeći:

- Simulacioni modeli za digitalne računare mogu biti skupi i mogu zahtijevati značajno vreme za izgradnju i validaciju.
- Zbog statističkog karaktera simulacije potrebno je izvođenje većeg broja simulacionih eksperimenata kako bi se dobio odgovarajući uzorak rezultata simulacije, a već i pojedinačno izvođenje eksperimenta može zahtijevati dosta vremena i memorije računara.
- Ne dobijaju se zavisnosti izlaznih promjenljivih od ulaznih promjenljivih modela niti optimalna rješenja.
- Za ispravno korištenje simulacionog modeliranja potrebno je poznavanje više različitih metoda i alata.
- Vrednovanje modela je dosta složeno i zahtijeva dodatne eksperimente.

3.6. Simulacioni proces

Simulacioni proces je struktura rješavanja stvarnih problema pomoću simulacionog modeliranja. On se može prikazati u obliku niza koraka koji opisuju pojedine faze rješavanja problema ovom metodom (životni ciklus simulacije). Struktura simulacionog procesa nije strogo sekvencijalna, već je moguć i povratak na prethodne korake procesa, zavisno od rezultata dobijenih u pojedinim fazama procesa. Broj faza i redoslijed njihovog obavljanja zavisi od svake konkretne situacije, ali je ipak moguće navesti jedan opšti, uređen skup procedura (slika 3.3).

Osnovni koraci simulacionog procesa su sljedeći:

1. Definicija cilja simulacione studije

Definicija željenog cilja i svrhe studije: problem koji treba riješiti (oblikovanje sistema, analiza sistema i sl.), granice sistem/okolina, nivo detaljnosti.

2. Identifikacija sistema

Opis komponenti sistema, interakcija komponenti, način rada, veze s okolinom, formalni prikaz sistema.

3. Prikupljanje podataka o sistemu i njihova analiza

Prikupljanje i mjerenje relevantnih podataka o sistemu, analiza tih podataka (izbor raspodjela nezavisnih slučajnih promjenljivih, ocjena vrijednosti parametara raspodjela).

4. *Izgradnja simulacionog modela*

Stvaranje konceptualnog modela koji adekvatno opisuje sistem i omogućava rješavanje zadatog problema.

5. *Izgradnja simulacionog programa*

Izbor programskog jezika ili paketa i stvaranje simulacionog programa bilo pisanjem programa, bilo automatskim generisanjem programa na osnovu konceptualnog modela.

6. *Verifikacija simulacionog programa*

Testiranje simulacionog programa prema postavkama simulacionog modela (pojedinačne procedure, generisanje slučajnih promjenljivih, povezivanje procedura). Ukoliko verifikacija programa nije dala zadovoljavajuće rezultate, potreban je povratak na korak 5.

7. *Vrednovanje (validacija) simulacionog modela*

Ispitivanje da li simulacioni model adekvatno predstavlja stvarni sistem (ispitivanjem podudarnosti izlaza modela i realnog sistema, analizom rezultata od strane eksperata, analizom osjetljivosti). Ukoliko vrednovanje modela nije uspješno, potrebno je vratiti se na tačku 4 (izmjene u simulacionom modelu).

8. *Planiranje simulacionih eksperimenata i njihovo izvođenje*

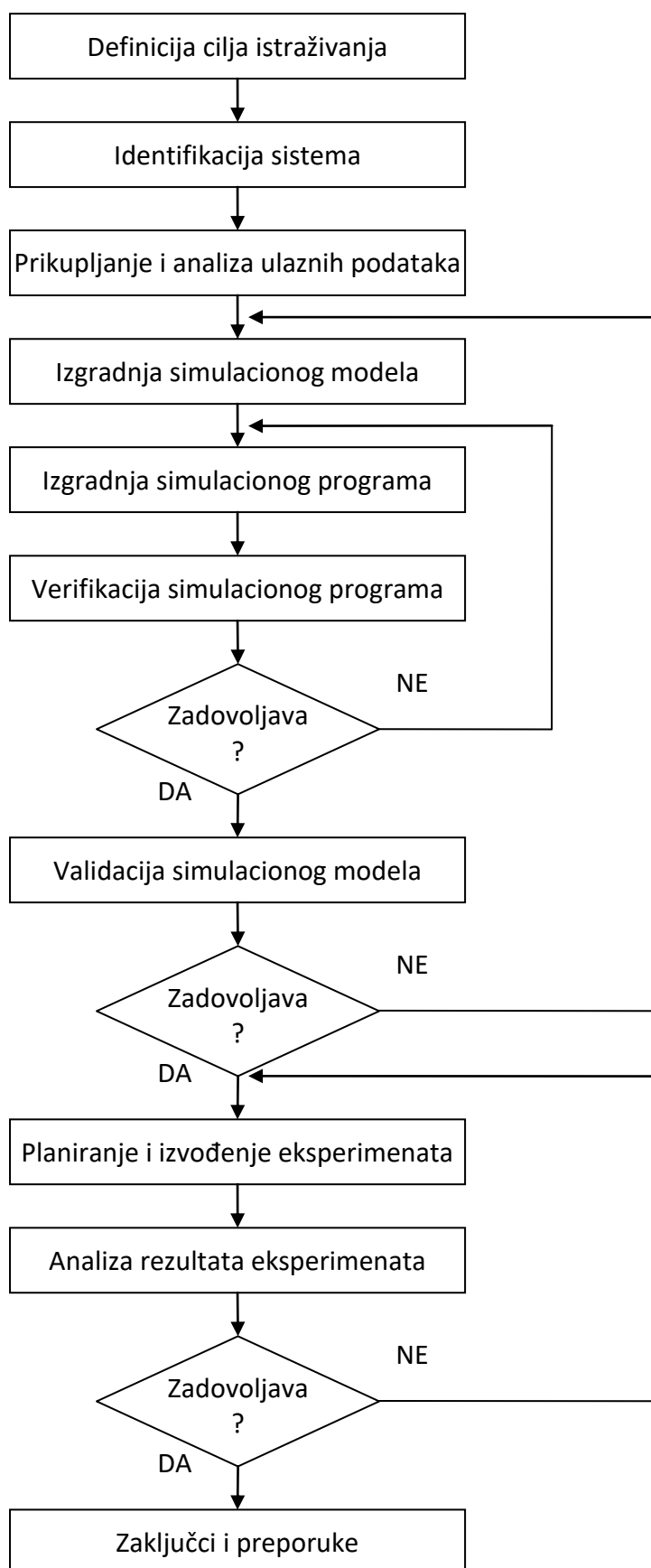
Planiranje simulacionih eksperimenata koji omogućuju ispunjenje cilja studije (plan promjene parametara modela, ponavljanje eksperimenta zbog analize uticaja slučajnih promjenljivih, i dr). Izvođenje simulacionih eksperimenata prema usvojenom planu.

9. *Analiza rezultata eksperimenata*

Statistička analiza rezultata simulacionih eksperimenata. Tokom analize rezultata može se pokazati potreba za dopunom koraka 8 (izvođenje dodatnih eksperimenata).

10. *Zaključci i preporuke*

Prezentacija relevantnih rezultata na osnovu kojih se mogu donijeti odgovarajuće odluke (izbor konfiguracije sistema, izmjene u sistemu, i dr.).



Slika 3.3 – Dijagram toka simulacionog procesa

3.7. Podjele simulacionih modela

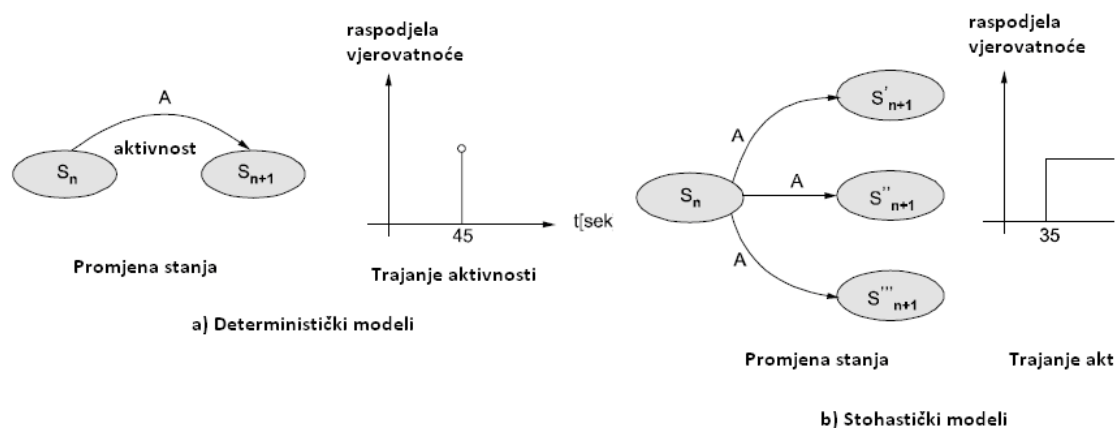
Razlikuju se dva osnovna načina podjele simulacionih modela: prvi, prema vrsti promjenljivih u modelu i drugi, prema načinu na koji se stanje u modelu mijenja u vremenu.

3.7.1. Deterministički i stohastički modeli

Deterministički modeli su oni čije se ponašanje može predvidjeti, odnosno u kojima je ovo stanje sistema u potpunosti određeno prethodnim stanjem. Na slici 3.4-a prikazan je deterministički model u kojem se stanje sistema S_n mijenja pod uticajem aktivnosti A u stanje S_{n+1} . Prikazana je takođe i aktivnost A sa determinističkim trajanjem od 45 sekundi, npr. trajanje pregleda jedne komponente određenog proizvoda.

Stohastički modeli su oni čije se ponašanje ne može unaprijed predvidjeti, ali se mogu odrediti vjerovatnoće promjena stanja sistema. Dakle, za stohastičke modele je karakteristično slučajno ponašanje, odnosno postojanje slučajnih promjenljivih u sistemu. Na slici 3.4-b prikazan je stohastički model u kome se stanje sistema S_n može promijeniti u jedno od stanja S'_{n+1} , S''_{n+1} ili S'''_{n+1} pod uticajem aktivnosti A. Tako na primjer, nakon izvršenog pregleda, komponenta može biti odbačena ili se može ugraditi u finalni proizvod, pri čemu svako od tih stanja ima određenu vjerovatnoću.

Drugi izvor slučajnosti, prikazan na istoj slici, je aktivnost sa slučajnim trajanjem, odnosno trajanjem koje se određuje iz uniformne raspodjele vjerovatnoće. Primjer takve aktivnosti je točenje goriva na benzinskoj pumpi, gde se mjerenjem određenog uzorka točenja goriva vozilima dobijaju podaci iz kojih se može odrediti raspodjela vjerovatnoća trajanja te aktivnosti.



Slika 3.4 – Deterministički i stohastički modeli

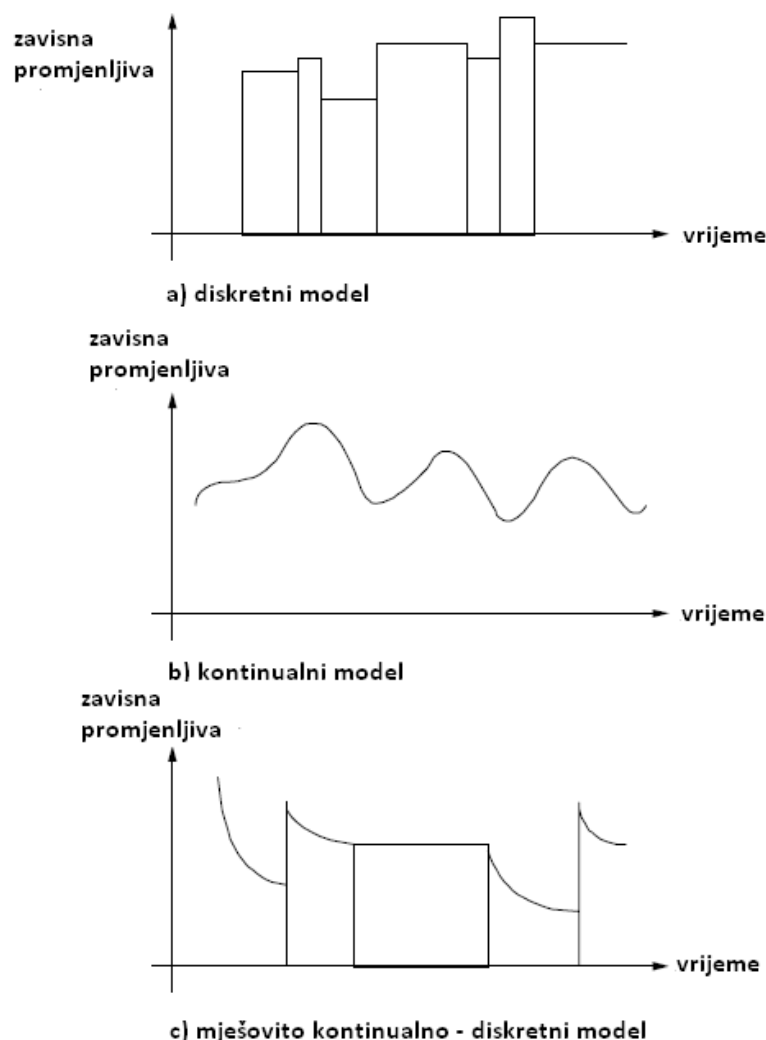
3.7.2. Diskretni i kontinualni modeli

U diskretnim modelima stanje sistema se mijenja samo u pojedinim tačkama u vremenu (nema kontinualne promjene stanja). Takve promjene se nazivaju događaji. Diskretna (diskontinualna) promjena stanja prikazana je na slici 3.5-a.

Na primjer, u modelu samoposluge mogu postojati promjenljive koje opisuju broj kupaca u redovima pred kasama, a taj se broj može mijenjati samo u trenutku dolaska kupaca u red i u trenutku početka opsluživanja na kasi.

U kontinualnim modelima promjenljive stanja se mijenjaju kontinualno u vremenu, kao što je prikazano na slici 3.5-b. Primjer kontinualne promjene je let aviona čiji se položaj i brzina mijenjaju kontinualno u vremenu. Treba imati u vidu da se na digitalnim računarima ne mogu izvoditi kontinualne promjene veličina, već se one moraju aproksimirati skupom diskretnih vrijednosti.

Mogući su i mješoviti, kontinualno-diskretni modeli koji sadrže i kontinualne i diskretne promjenljive. Promjena stanja takvog modela prikazana je na slici 3.5-c.



Slika 3.5 – Diskretni, kontinualni i mješoviti modeli

3.8. Vrste simulacionih modela

Prikazane podjele simulacionih modela dovele su do formulisanja četiri osnovne vrste simulacionih modela, koji se razlikuju, s jedne strane, po pristupu modeliranju i klasi problema koji se rješava, i s druge strane, po tehnikama modeliranja i simulacije koje su za njih razvijene. To su:

- emulacija,
- Monte Karlo (eng. *Monte Carlo*) simulacija,
- kontinualna simulacija,
- simulacija diskretnih događaja,
- mješovita, kontinualno-diskretna simulacija.

Osim emulacije i Monte Karlo simulacije, koje su statičke, sve ostale nabrojane vrste su dinamičke. U statičkim simulacijama ne postoji vremenski parametar. Simulacija se vrši dok sistem ne dođe u neko stanje ekvilibrijuma, tj. dok dalje izvođenje simulacije nije korisno ili nije moguće. Monte Karlo simulacija spomenuta je ovdje iz razloga što su u njoj tretman slučajnih događaja i generisanje slučajnih vrijednosti bliski onom iz simulacije diskretnih događaja.

U nastavku su ukratko opisane sve nabrojane vrste simulacionih modela, dok su modeli kontinualne simulacije i modeli sa diskretnim događajima, detaljnije razmatrani u posebnim poglavljima.

3.8.1. Emulacija

Emulator je simulacioni program koji se pokreće na određenom sistemu sa svrhom da sistem izgleda kao neki drugi. Na ovaj način emulator žrtvuje performanse u korist fleksibilnosti. JVM (eng. *Java Virtual Machine*) je primjer emulacije procesora. U tom slučaju se simulira izvršavanje virtuelnog procesora čiji je instrukcioni set definisan u JVM specifikaciji.

3.8.2. Monte Karlo simulacija

Monte Karlo simulacija (statistička simulacija), kao što joj i ime kaže, povezana je sa slučajnim fenomenima. Ova metoda je razvijena u SAD tokom Drugog svjetskog rata, ali i danas još uvijek nema potpune saglasnosti o korištenju tog termina. Neki autori Monte Karlo simulacijama nazivaju bilo koju vrstu programa koji koriste slučajne brojeve. S druge strane, u većini literature o simulacionom modeliranju, ovaj termin se upotrebljava samo za statičke tipove simulacije kod kojih se u rješavanju problema

koristi stvaranje uzoraka iz raspodjela slučajnih promjenljivih. Pri tome, problemi mogu biti bilo determinističkog, bilo stohastičkog karaktera.

Razlikuju se sljedeći tipovi primjene Monte Karlo simulacije:

1. *Deterministički problemi koje je teško ili skupo rješavati*

Tipičan primjer ovoga tipa je računanje vrijednosti određenih integrala koji se ne mogu riješiti analitički, tj. čija je podintegralna funkcija takva da se ne može naći rješenje u obliku analitičkog izraza. Monte Karlo simulacija pristupa proračunu integrala tako što se generiše niz slučajnih tačaka (x_j, y_j) , sa jednakim vjerovatnoćama unutar određenog pravougaonika i potom ispituje koliko je generisanih tačaka unutar površine koja odgovara integralu. Ovakav pristup, koji se zasniva na generisanju slučajnih brojeva, analogan je onom koji se koristi kod simulacije sistema sa diskretnim događajima.

2. *Složeni fenomeni koji nisu dovoljno poznati*

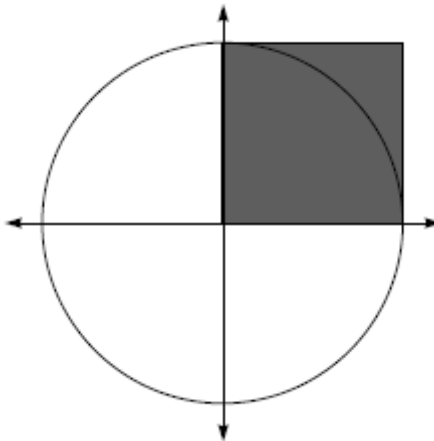
Ovo je druga klasa problema koji se rješavaju Monte Karlo simulacijom. Za njih je karakteristično da nije poznat način uzajamnog djelovanja između elemenata već su poznate samo vjerovatnoće njegovog ishoda, koje se koriste za izvođenje niza eksperimenata koji daju uzorke mogućih stanja zavisnih promjenljivih. Statističkom analizom takvih uzoraka dobija se raspodjela vjerovatnoća zavisnih promjenljivih koje su od interesa. Ovakav pristup se najčešće primjenjuje kod analiziranja društvenih ili ekonomskih fenomena.

3. *Statistički problemi koji nemaju analitička rješenja*

Statistički problemi bez analitičkog rješenja (npr. procjene kritičnih vrijednosti ili testiranje novih hipoteza) su jedna specifična klasa problema koji se rješavaju Monte Karlo simulacijom. Prilikom rješavanja takvih problema takođe se koristi generisanje slučajnih brojeva i promjenljivih.

Kao primjer koji će pokazati kako radi Monte Karlo simulacija, može poslužiti numeričko izračunavanje vrijednosti broja π . Izračunavanje se vrši na osnovu slike 3.6. Ako se pretpostavi da je poluprečnik kruga sa slike 1, slijedi da je površina cijelog kruga $\pi (1)^2 = \pi$. Samim tim, površina označene četvrtine kruga je $\pi/4$. Pošto je poluprečnik 1, površina označenog kvadrata na slici (unutar kojeg je i četvrtina kruga) je 1. Samim tim, odnos površina kruga i kvadrata, označena sa R , je $\pi/4$. Na osnovu toga, slijedi da je numerička vrijednost broja π jednaka $4R$.

Gornjim razmatranjem je problem izračunavanja numeričke vrijednosti broja π transformisan u izračunavanje odnosa površina četvrtine kruga i kvadrata. Tu se može iskoristiti Monte Karlo simulacija. Ako se slučajno pogađaju tačke unutar kvadrata, može se brojati broj tačaka koje dodatno padnu i unutar kruga, n_{circ} i ukupan broj tačaka, n_{total} . Kada ukupan broj tačaka postane dovoljno velik, željeni odnos površina postaje $R = n_{\text{circ}} / n_{\text{total}}$.



Slika 3.6 – Izračunavanje numeričke vrijednosti broja π pomoću Monte Karlo simulacije

Eksperiment generisanja tačaka se može uraditi generisanjem dvije slučajne promjenljive u_1 i u_2 koje prate uniformnu raspodjelu između 0 i 1. Ako je dobijenoj tački udaljenost od koordinatnog početka ($\sqrt{u_1^2 + u_2^2}$) manja od 1, tada je jasno da je ta tačka pala i unutar posmatrane četvrtine kruga. Što se više tačaka izgeneriše, preciznost dobijene vrijednosti postaje sve veća.

3.8.3. Kontinualna simulacija

Kontinualna simulacija se koristi za dinamičke probleme kod kojih se promjenljive stanja mijenjaju kontinualno u vremenu. Postoje dvije osnovne klase problema koji se rješavaju ovom metodom.

U prvoj klasi su relativno jednostavni problemi koji su opisani detaljno i kod kojih su promjene "glatke" i prirodno se opisuju diferencijalnim jednačinama. To su tipično problemi iz fizike, biologije i inženjerstva. U drugoj klasi su problemi koji nastaju opisom veoma složenih sistema u agregiranom obliku, u kojem se niz elemenata sistema redukuje na manji broj komponenti i u kojima se promjene u sistemu aproksimiraju konstantnim brzinama promjene. To su najčešće problemi iz područja ekonomije i društvenih nauka.

Razlikujemo tri osnovna tipa kontinualnih simulacionih modela:

1. *Modeli koji se opisuju običnim diferencijalnim jednačinama (sistemi običnih diferencijalnih jednačina)*

Problemi u vezi sa raznim procesima kao što su, na primjer, razni oblici kretanja i mnogi fizički, hemijski, biološki i drugi procesi gdje se radi o jednoj nepoznatoj funkciji ($y = y(t)$) jedne nezavisne promjenljive (t), izražavaju se matematičkim jednačinama u kojima se, pored nezavisne promjenljive i nepoznate funkcije, javljaju (obavezno) i izvodi te funkcije (dy/dt). Tim jednačinama se u svakom konkretnom slučaju uspostavlja veza između nepoznate funkcije (y), njene

realne promjenljive (t) i njenih izvoda (y') i takve se jednačine nazivaju obične diferencijalne jednačine. Modeli koji se opisuju običnim diferencijalnim jednačinama ili njihovim sistemima mogu se u određenim slučajevima riješiti analitički, ali je to dosta rijetko kod stvarnih problema (uobičajeno je da ove jednačine uzimaju oblik složenih nelinearnih diferencijalnih jednačina koje se po pravilu ne mogu riješiti analitički). Zbog toga se za takve jednačine moraju koristiti numeričke metode. Numeričke metode mogu se primijeniti i na analitički rješive jednačine, ali se to u praksi izbjegava, jer je u većini slučajeva za takve jednačine jednostavnije analitičko rešenje. Numeričke metode koje služe za rješavanje diferencijalnih jednačina kod kojih se kao nezavisna promjenljiva javlja vrijeme, uobičajeno se nazivaju metode kontinualne simulacije.

2. *Modeli koji se opisuju sistemima parcijalnih diferencijalnih jednačina*

Parcijalne diferencijalne jednačine sadrže više od jedne nezavisne promjenljive (x_j) po kojima se traže izvodi zavisne promjenljive. Tipični primjeri problema koji se mogu opisati parcijalnim diferencijalnim jednačinama su problemi aerodinamike, hidrodinamike i meteorologije.

3. *Modeli dinamike sistema (eng. System Dynamics)*

Dinamika sistema je metodologija istraživanja, modeliranja i simulacije složenih dinamičkih sistema. Sistemi sa povratnom vezom su osnovni tip sistema koji se modeliraju dinamikom sistema. Povratna veza može biti pozitivna ili negativna. Modeli sa povratnom vezom koriste se najčešće za modeliranje inženjerskih, bioloških, društvenih i ekonomskih sistema. Dinamika sistema prikazuje sisteme kao povezane upravljačke petlje. Pri tome su pojedinačni događaji jako agregirani, a posledica toga je da se mogu opisivati kao kontinualni tokovi opisani diferencnim jednačinama (tj. konačnim razlikama, a ne beskonačno malim veličinama). Novo stanje sistema u narednom trenutku vremena računa se na osnovu stanja u prethodnom trenutku vremena i razlike ulaznih i izlaznih tokova za to stanje u prethodnom intervalu vremena. Da bi proračun mogao početi, neophodno je da budu zadate početne vrijednosti veličina kao i vrijednosti svih konstanti i parametara koje model sadrži.

3.8.4. Simulacija diskretnih događaja

Simulacija diskretnih događaja je metoda simulacionog modeliranja sistema kod kojih se diskretne promjene stanja u sistemu ili njegovom okruženju događaju diskontinualno u vremenu. Uglavnom se koristi za analizu dinamičkih sistema sa stohastičkim karakteristikama. S obzirom da se pod događajem podrazumijeva diskretna promjena stanja entiteta sistema (tj. promjenljive koja opisuje to stanje), događaj

nastupa u određenom trenutku vremena. Kako su veličine u ovakvim sistemima najčešće slučajne prirode, govorimo o diskretnim stohastičkim sistemima.

Kod modela sistema sa diskretnim događajima, pored koncepata koji opisuju strukturu, kao što su objekti, relacije između njih i njihovi atributi, uvedeni su i koncepti za opis dinamike. To su:

- događaj,
- aktivnost i
- proces.

Događaj predstavlja diskretnu promjenu stanja entiteta u sistemu ili njegovom okruženju. Između dva uzastopna događaja stanje sistema se ne mijenja. On može nastupiti zbog:

- ulaska/izlaska privremenog entiteta u odnosno iz sistema (dolazak novog klijenta, odlazak klijenta), ili
- promjene atributa pojedinih objekata sistema (opslužilac u sistemu postaje slobodan ili zauzet).

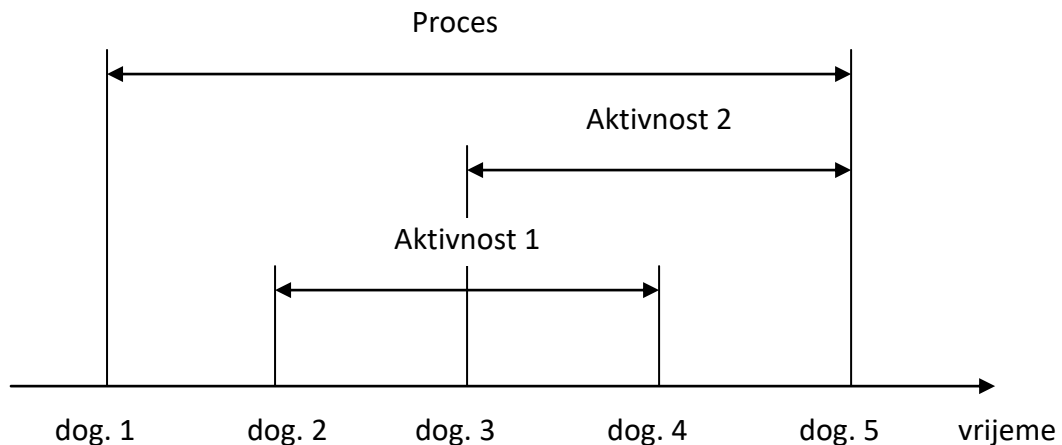
Uslovni događaji mogu da nastupe tek kada je ispunjen uslov njihovog nastupanja. Obično su povezani sa zauzimanjem nekog resursa, odnosno početkom aktivnosti.

Bezuslovni događaji su oni čiji je jedini uslov da tekuće vrijeme simulacije bude jednako vremenu njegovog nastupanja. To je, po pravilu, planirano vrijeme završetka neke aktivnosti. Obično su povezani sa oslobađanjem resursa koji su bili aktivirani u toku trajanja aktivnosti.

Aktivnost je skup događaja koji mijenjaju stanje jednog ili više entiteta. Trajanje aktivnosti se može unaprijed definisati, ali može da zavisi i od ispunjenja nekih uslova u modelu, tako da je vrijeme završetka takve aktivnosti nepoznato. Za aktivnost sa nepoznatim vremenom završetka u literaturi se može naći i izraz "zadržavanje" a uglavnom se odnosi na zadržavanje u redovima čekanja.

Proces je niz uzastopnih, logički povezanih događaja kroz koje prolazi neki privremeni objekat. Drugim riječima, proces je hronološki uređena sekvenca događaja koja opisuje jednu pojavu od nastajanja do terminiranja. U simulaciji, proces može da obuhvata dio ili cjelokupan "život" privremenog entiteta.

Odnos događaja, aktivnosti i procesa može se grafički predstaviti kao na slici 3.7.



Slika 3.7 – Odnos događaja, aktivnosti i procesa

Koristeći se prethodno definisanim pojmovima, može se opisati simulacija diskretnih događaja na sljedeći način (Shannon, 1975):

Def. *Entiteti* koji se opisuju *atributima* i uzajamno djeluju učestvujući u *aktivnostima* pod određenim *uslovima* stvaraju *događaje* koji mijenjaju *stanja sistema*.

Simulacija diskretnih događaja je specifična metodologija simulacije koja se bavi modeliranjem sistema koji se mogu predstaviti skupom događaja. Pod događajem ovdje se podrazumijeva diskretna promjena stanja entiteta sistema. Događaj nastupa u određenom trenutku vremena, odnosno promjene stanja entiteta se dešavaju diskontinualno u vremenu, tj. samo u nekim vremenskim trenucima (kada nastupi događaj). Simulacija opisuje svaki diskretni događaj, krećući se od jednog događaja do drugog pri čemu nastaje pomak (prirast) vremena simulacije. Između dva uzastopna događaja, stanje sistema se ne mijenja. Sistemi koji se modeliraju na ovaj način su dinamički i gotovo redovno stohastički.

Simulator diskretnih događaja se koristi za modelovanje sistema čije se globalno stanje mijenja kao funkcija vremena. Glavna ideja je da se globalno stanje sistema pravilno ažurira svaki put kada se desi neki događaj. Iako su specifični detalji svakog simulatora jedinstveni, ipak svi simulatori diskretnih događaja dijele zajedničku opštu strukturu. Svaki ovakav simulator zahtijeva neku od sljedećih komponentata:

- *raspoređivač događaja* (eng. *event scheduler*)

Raspoređivač događaja je srce simulatora diskretnih događaja. On održava listu svih događaja u njihovom redu po globalnom vremenu. Njegov zadatak je da sljedeći događaj sa liste skloni sa liste i rasporedi ga u odgovarajuću rutinu za procesiranje događaja. On takođe smješta nove događaje u listu na odgovarajuće mjesto u skladu sa vremenom kada on treba da se izvrši. Njegov zadatak takođe predstavlja i ažuriranje varijable koja predstavlja globalno vrijeme.

- *varijabla koja predstavlja globalno vrijeme i način na koji se ona ažurira*

Ova komponenta bilježi trenutno vrijeme izvođenja simulacije. Može se ažurirati od strane raspoređivača na dva načina. U prvom se uvećava za neki fiksni iznos. Zatim se provjeravaju događaji na čekanju iz liste. Svi događaji koji se podudaraju sa tim trenutkom šalju se na izvršavanje. Nakon završetka njihovog procesiranja, raspoređivač opet inkrementuje ovu varijablu. U alternativnom pristupu varijabla koja predstavlja globalno vrijeme prilagođava se sljedećem događaju na čekanju iz liste. U ovom slučaju vrijednost ove varijable se mijenja neuniformno pošto će se nekad dešavati da se nekad mijenja za veliki iznos, a nekad se neće promijeniti uopšte. Ovaj pristup se češće susreće u računarskim sistemima.

- *rutine za procesiranje događaja*

Svaki događaj u sistemu tipično ima svoju procesnu rutinu koja simulira šta bi se desilo da se događaj nalazi u realnom sistemu. Ove rutine mogu da ažuriraju globalno stanje ili čak da generišu dodatne događaje koji se smještaju u listu za čekanje kontrolisanu od strane raspoređivača. Primjer za ovaj slučaj je pristup memoriji od strane procesora. U slučaju da tražena vrijednost ne postoji u kešu, generiše se novi događaj koji predstavlja vrijeme potrebno da se izvrši rutina za *cache miss*.

- *mehanizmi za generisanje događaja*

Simulatori diskretnih događaja se često klasifikuju na osnovu tehnike koju koriste za generisanje događaja. Tehnike za generisanje događaja su: *execution driven*, *trace driven* i *distribution driven*.

- *Execution driven*

Ova vrsta simulacije je slična emulaciji po tome što simulator zapravo izvršava benčmark program. Na osnovu toga, simulator produkuje isti izlaz iz benčmark programa koji bi bio produkovan da se taj benčmark izvršava na pravoj mašini. Glavna razlika između ova dva pristupa je u tome što simulator, osim što izvršava program, dodatno modeluje potrebne detalje sistema koji se testira. Emulacija, sa druge strane, samo izvodi program zanemarujući kako se on izvršava i jedini njen zadatak je produkovanje odgovarajućeg izlaza iz benčmark programa. *Execution driven* simulator obraća pažnju i na to kako se dolazi do željenog izlaza. Ovi modeli se smatraju najtačnijim tipom simulacije pošto modeluju sve detalje potrebne da bi se program izvršio. Međutim, ova prednost se može smatrati i nedostatkom pošto modelovanje svakog detalja može biti prilično skupo u smislu vremena potrebnog za odvijanje simulacije i vremena potrebnog za razvijanje *execution driven* simulatora.

- *Trace driven*

Trace (trag) je sekvenca događaja koji se odvijaju dok se sistem prati. U ovoj sekvenci događaji su uglavnom poredani po vremenu kada treba da

se izvrše. Jedna od prednosti ovakvog pristupa je što simulator ne mora da pruži sve funkcionalnosti potrebne od strane programa, kao što je potrebno kod *execution driven* simulatora. Sa druge strane, *trace* predstavlja samo jedno moguće izvršavanje programa. Taj *trace* uopšte ne mora da predstavlja tipičan *workload*, pa je nedostatak ovakvog pristupa nedostatak generalnosti. Drugi nedostatak je teškoća rukovanja sa velikim *trace* fajlovima. Ogromna količina informacija koje se generišu tokom *tracing*-a tipično limitiraju *trace* na nekoliko sekundi od ukupnog vremena izvršavanja programa. Zbog navedenih nedostataka, *execution driven* simulacija je uprkos svojoj složenosti popularnija od *trace driven* simulacije.

- *Distribution driven*

Ovaj tip simulacije je sličan *trace driven* simulaciji uz razliku što ulazne događaje generiše sam simulator tako da oni prate neku predefinisanu funkciju vjerovatnoće. Generiše se sekvenca slučajnih brojeva radi modeliranja očekivanog statističkog ponašanja ulaza u sistem koji se simulira. Pošto ne postoji garancija da će pravi program imati ovakvu sekvencu ulaza, simulacija treba da se odvija više puta sa nekoliko različitih ulaznih sekvenci. Produkovani izlazi zatim mogu da se sumiraju na odgovarajući način radi dobijanja statističkog pogleda na performanse posmatranog sistema. Ovakav pristup nudi veću fleksibilnost od *trace driven* simulacije gdje je niz događaja u *trace*-u fiksiran i teško je izmijeniti njihov redoslijed.

- *rutine za bilježenje podataka i njihovo sumiranje*

Osim održavanja varijabli stanja neophodnih za simuliranje sistema, simulator mora da održava odgovarajuće brojače događaja i mjerače vremena. Ove vrijednosti se na kraju koriste da bi se izračunali odgovarajući statistički podaci kojima se sumiraju rezultati simulacije.

3.8.5. Mješovita simulacija

Kod pojedinih vrsta sistema, kontinualna simulacija, kao i simulacija diskretnih događaja, ne može u potpunosti da opiše način rada sistema. To su oni sistemi koji sadrže procese koji teku kontinualno i događaje koji dovode do diskontinuiteta u ponašanju sistema. Da bi se takvi sistemi modelirali i simulirali, razvijena je mješovita simulacija koja omogućava integrisanje kontinualnih i diskretnih elemenata sistema.

Veza između diskretnog i kontinualnog pristupa postiže se uvođenjem dva tipa događaja. Vremenski događaji su događaji koje generiše mehanizam upravljanja događajima, kakav postoji u simulaciji diskretnih događaja. Oni mogu da izazovu trenutnu promjenu stanja kontinualne promjenljive. S druge strane, događaji stanja su

oni događaji koje aktivira mehanizam pomaka vremena sa konstantnim prirastom, čiji je vremenski interval mali, a koji je karakterističan za kontinualnu simulaciju. Ovi događaji mogu da aktiviraju događaje diskretnog dijela modela. Poznatiji simulacioni jezici za mješovitu simulaciju su GASP i SLAM.

3.8.6. Izbor tipa simulacionog modela

Tip simulacionog modela najčešće se bira tako da bude jednak tipu originalnog sistema. To ipak ne znači da se uvijek diskretni sistem mora prikazati diskretnim modelom, odnosno kontinualni sistem kontinualnim modelom. Izbor tipa simulacionog modela zavisi najviše od cilja simulacione studije. Pored toga, važnu ulogu ima i sposobnost modelara da pronađe adekvatnu apstrakciju sistema za rješavanje određenog problema.

Najvažnije pri izboru simulacionog modela je da model bude što jednostavniji i razumljiviji, kako zbog njegovog lakšeg razvoja i modifikacije, tako i zbog potrebe da ga korisnik što lakše razumije. Osim toga, značajan faktor pri izboru modela uvijek mora da bude i njegova cijena, kao i efikasnost u pogledu utroška resursa računara, kako bi se mogao efikasno koristiti za rješavanje problema.

3.9. Validacija i verifikacija

Jedan od najznačajnijih i najtežih zadataka sa kojim se sreće modelar u postupku izgradnje modela su verifikacija i validacija simulacionih modela. Korisnici modela, inženjeri i analitičari koji koriste izlaze (rezultate) modela kao pomoć za svoje prijedloge pri projektovanju i menadžeri koji donose odluke zasnovane na tim prijedlozima, uvijek gledaju na model sa izvjesnom dozom skepticizma u pogledu njegove validnosti. Da bi smanjio postojeći skepticizam i povećao povjerenje u svoj model, modelar mora da sarađuje sa krajnjim korisnicima u procesu izgradnje i validacije modela.

Problem validacije modela proističe iz činjenice da je model uvijek uproštena slika realnog sistema koji predstavlja, odnosno pojednostavljeni pogled na realni sistem koji se istražuje. Pored toga, u model se gotovo uvijek unose određene aproksimacije realnosti, što proces validacije čini još važnijim. Iz svega toga proističe da se model mora testirati kako bi se ustanovilo da li je on pouzdan, bez grešaka i da li je dovoljno uvjerljiv za sve one koji će ga koristiti. Postupak kojim se ispituje koliko vjerno i precizno jedan model predstavlja realni sistem, najčešće se može posmatrati kroz dva povezana koraka:

1. Verifikacija.

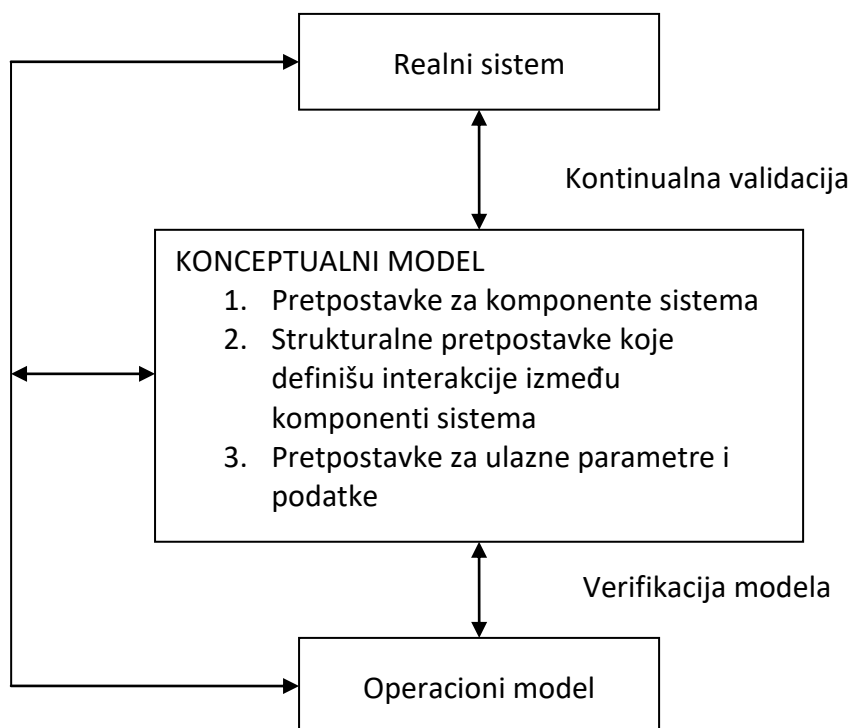
Odnosi se na provjeru da li je simulacioni program, kojim se implementira model, bez grešaka i konzistentan sa modelom. To je u stvari poređenje

konceptualnog modela sa računarskim kodom kojim se takva koncepcija implementira.

2. Validacija.

Odnosi se na postupak određivanja da li je model precizna reprezentacija realnog sistema. Najčešće je to jedna iterativna procedura u kojoj se ponašanje modela poredi sa ponašanjem realnog sistema i uočena neslaganja i razlike koriste za dogradnju i ispravku modela. Postupak poboljšanja modela se nastavlja, sve dok se ne odluči da dobijena tačnost modela zadovoljava.

Iako se verifikacija i validacija modela konceptualno razlikuju, najčešće se simultano sprovode od strane modelara. Razlog za to je što se ova dva procesa u praksi dobrim dijelom poklapaju. Naime, ukoliko jedan simulacioni program proizvodi besmislice, nije uvijek jasno koliko su krive greške u konceptualnom modelu, koliko greške u programiranju, a koliko korištenje pogrešnih podataka. Zato se često kaže da su izgradnja modela, verifikacija i validacija u dinamičkoj povratnoj sprezi. Na to ukazuje i slika 3.8.



Slika 3.8 – Izgradnja modela, validacija i verifikacija

Prvi korak u izgradnji modela obuhvata posmatranje realnog sistema i interakcija između njegovih različitih komponenti, kao i prikupljanje podataka o ponašanju sistema. Međutim, u tom postupku modelar ne smije da se osloni samo na svoja zapažanja, pošto to često vodi pogrešnom shvatanju samog sistema i njegovog ponašanja, već je neophodno angažovanje stručnjaka koji dobro poznaju sistem ili neke njegove dijelove.

Specifična znanja o pojedinim aspektima sistema, koja ovi stručnjaci različitih profila posjeduju, od velikog su značaja za modelara i budući tok izgradnje modela.

Drugi korak u izgradnji modela je formiranje konceptualnog modela, skupa pretpostavki za komponente modela i strukturu sistema, kao i hipoteza za vrijednosti parametara modela. Kao što se može vidjeti sa slike 3.8, konceptualna validacija je poređenje realnog sistema i konceptualnog modela.

Treći korak u izgradnji modela je pisanje računarskih programa za simulacione modele.

Treba napomenuti da se ove tri faze izgradnje modela višestruko prepliću i da to nije jednostavan linearni proces koji se sastoji iz tri koraka; naime, modelar se vraća na svaku od ove tri faze više puta u toku izgradnje, verifikacije i validacije modela.

3.9.1. Validacija simulacionih modela

Problem validacije modela nastaje pošto se u fazi izgradnje modela unose mnoge aproksimacije realnog sistema. Na taj način se ograničava model (uspostavljaju se granice u odnosu na okruženje), ignorišući sve ono izvan njega što nije eksplicitni ulaz i odbacivajući faktore za koje se smatra da su nevažni. Aproksimacije koje se najčešće primjenjuju su:

1. Funkcionalna aproksimacija

Izrazito nelinearne funkcije često se aproksimiraju nekim jednostavnijim funkcijama, na primjer linearnim. Važna pretpostavka je da jednostavnija funkcija treba da bude približna "originalnoj" funkciji u oblasti gde će sistem vjerovatno funkcionisati. Ako program mora da funkcionise i u oblasti gdje je poklapanje funkcija slabo, potrebno je obezbijediti da on štampa poruku upozorenja.

2. Aproksimacija raspodjele

Realne vjerovatnoće raspodjela koje su i same poznate jedino aproksimativno, često se aproksimiraju jednostavnijim raspodjelama, kao što su normalna ili eksponencijalna. Najekstremniji primjer aproksimacije je onaj kada se slučajna promjenljiva zamijeni konstantom.

3. Aproksimacija nezavisnosti

Model se često pojednostavljuje tako što se pretpostavlja da su različite komponente (opisane slučajnim promjenljivima) statistički nezavisne.

4. Aproksimacija agregacije

Veoma čest tip aproksimacije je agregacija. Pod agregacijom se podrazumijeva situacija kada se više elemenata posmatraju kao jedna cjelina. Neki tipični primjeri agregacija su:

a. *Vremenska agregacija*

Svi diskretno-vremenski modeli vrše vremensku agregaciju. Interval vremena kao što je dan, tretira se kao jedan pojedinačni period. Za sve događaje koji se dese tokom dana, pretpostavlja se da su se desili istovremeno.

b. *Među-sektorska agregacija*

Više odjeljenja, firmi, proizvodnih linija, itd. posmatra se kao jedna cjelina.

c. *Agregacija pomoćnih sredstava*

Ovdje se više pomoćnih sredstava posmatra kao jedno. Na primjer, ako se posjeduje računarski sistem sa dva centralna procesora u paralelnoj vezi, nakon agregacije će se smatrati da se posjeduje jedan, dva puta brži centralni procesor.

5. *Aproksimacija stacionarnosti*

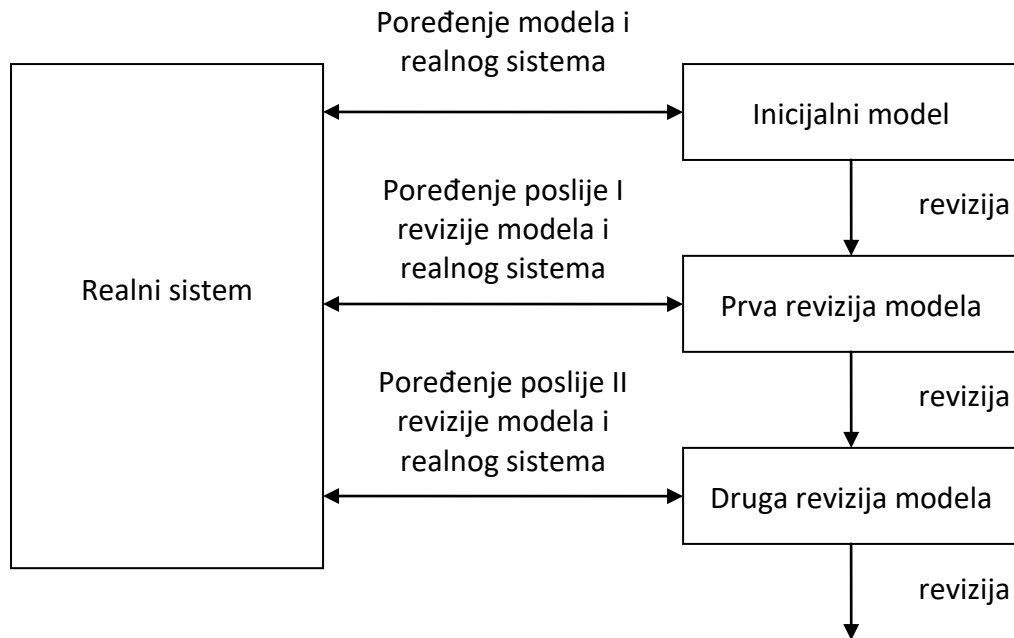
Stvar pojednostavljuje činjenica da se parametri i druge karakteristike sistema ne mijenjaju u vremenu. Za izvjesne fizičke procese, kao što su neke astronomske pojave, ovo može biti prihvatljiva početna aproksimacija. Međutim, u političkim, ekonomskim, organizacionim i socijalnim sistemima, svakodnevno iskustvo ukazuje da je većina pojava nestacionarne prirode i da je kao posljedica toga pretpostavka postojane strukture neodrživa.

Sve pretpostavke i aproksimacije koje se unose u model mogu, na izvjestan način, da dovedu u pitanje validnost tog modela. Iz tog razloga se i pristupa određivanju validnosti modela, odnosno poređenju modela i njegovog ponašanja sa realnim sistemom i njegovim ponašanjem.

Cilj procesa validacije je dvojak:

- da proizvede model koji predstavlja ponašanje realnog sistema i koji je dovoljno blizak realnom sistemu, tako da se može koristiti za obavljanje eksperimenta,
- da pouzdanost modela poveća na prihvatljiv nivo, tako da model mogu koristiti različiti donosioci odluka.

Na slici 3.9 prikazan je iterativni postupak poređenja modela i realnog sistema u postupku validacije.



Slika 3.9 – Iterativni proces validacije modela

Proces validacije modela ne treba posmatrati kao izolovani skup mjera i procedura koje slijede nakon izgradnje modela, već kao integralni i nezamjenjivi dio razvoja modela. Već je pomenuto da je proces validacije iterativni proces. Konceptualni model se poredi sa realnim sistemom i ukoliko postoje neslaganja i razlike za koje se smatra da nisu opravdane, vrši se ispravka (ili čak značajnija promjena) na modelu, zatim se tako dobijeni model ponovo poredi sa realnim sistemom i vrše nove ispravke, ukoliko je to potrebno itd. Iterativni proces se nastavlja sve dok se ne postigne zadovoljavajuća tačnost modela.

Poređenje modela sa realnim sistemom vrši se pomoću više različitih testova; neki od njih su subjektivne prirode, dok su drugi objektivni. Subjektivni testovi najčešće podrazumijevaju učešće ljudi koji dobro poznaju sistem ili neke njegove aspekte i koji, primjenjujući posebna znanja o sistemu, donose zaključke o modelu i njegovom izlazu. Objektivni testovi uvek zahtijevaju neke podatke o ponašanju sistema, kao i podatke koje proizvodi model. Prikupljeni podaci se obrađuju različitim statističkim testovima koji porede određene aspekte skupa podataka sistema i skupa podataka modela. Najčešće, jedan skup podataka nije dovoljan za konačnu validaciju modela, već se "finalna" validacija vrši uz pomoć nekog drugog, dodatnog skupa podataka.

Praktični pristup procesu validacije

Naylor i Finger (1976) formulisali su praktičan pristup problemu validacije, koji se sastoji se iz tri faze:

1. izgraditi model koji vjerno predstavlja realni sistem,
2. potvrditi pretpostavke modela,

3. uporediti ulazno-izlazne transformacije modela sa odgovarajućim ulazno-izlaznim transformacijama realnog sistema.

Procjena validnosti modela

Prvi cilj modelara je da izgradi model koji će izgledati razumno korisnicima ili drugima koji posjeduju znanja o realnom sistemu koji se simulira. Potrebno je da potencijalni korisnici modela budu uključeni u proces izgradnje modela od početka, dakle od konceptualizacije, pa sve do implementacije modela, kako bi se osiguralo da se u model "ugradi" visok stepen realizma kroz razumne pretpostavke koje se odnose na strukturu sistema i pouzdane podatke. Takođe je posebno važno da potencijalni korisnici, ali i drugi "ljudi od znanja", budu uključeni u proces iterativnog poboljšavanja modela, koji se sprovodi na osnovu uočenih neslaganja i odstupanja u postupku poređenja modela i realnog sistema. Posebna prednost uključivanja korisnika ogleda se u tome što se na taj način povećava povjerenje u model, bez koga menadžeri i drugi donosioci odluka sigurno ne bi bili voljni da vjeruju rezultatima simulacije, u procesu donošenja odluka.

Za provjeru validnosti modela, može se koristiti i analiza osjetljivosti. To je postupak testiranja osjetljivosti modela na različite pretpostavke i promjene ulaznih veličina. Promjene izlaznog ponašanja modela, kao posljedica promena ulaznih veličina, mogu pružiti korisne informacije o modelu i korisniku i modelaru. Kod većine simulacionih modela, postoji veliki broj ulaznih veličina, pa samim tim i veliki broj mogućih testova osjetljivosti. Zato je potrebno da modelar odabere one ulazne veličine za koje smatra da su "najkritičnije" i njih testira, pošto je vrlo vjerovatno da vrijeme i finansijska sredstva neće dozvoliti da se ispita osjetljivost modela na sve ulazne promjenljive. Ukoliko podaci o realnom sistemu omogućavaju bar dvije izmjene ulaznih parametara, testovi osjetljivosti se mogu dopuniti korištenjem odgovarajućih statističkih tehnika.

Validacija pretpostavki modela

Pretpostavke modela je moguće svrstati u dvije generalne kategorije: pretpostavke o strukturi i pretpostavke o podacima. Pretpostavke o strukturi se tiču pitanja kako funkcioniše sistem i često obuhvataju pojednostavljenja i apstrakcije realnog sistema. Pretpostavke o podacima treba da budu zasnovane na skupu pouzdanih podataka i ispravnoj statističkoj analizi tih podataka. Statistička analiza se uglavnom sastoji iz tri koraka:

1. identifikacija odgovarajućih raspodjela dobijenih podataka,
2. procjena parametara izabrane raspodjele,
3. validacija pretpostavljenog statističkog modela nekim testom (npr. hi-kvadrat ili Kolmogorov-Smirnov test).

Validacija ulazno - izlaznih transformacija

Jedini objektivni test modela kao cjeline je provjera sposobnosti modela da predvidi buduće ponašanje realnog sistema, kada ulazni podaci modela odgovaraju ulaznim podacima realnog sistema i kada je "politika" koja je implementirana u modelu, implementirana negdje i u realnom sistemu. Osim toga, ako se nivo neke ulazne veličine povećava ili smanjuje, model treba precizno da predvidi šta će se desiti u realnom sistemu pod istim okolnostima. Drugim riječima, struktura modela treba da bude dovoljno precizna da model daje dobra predviđanja i to ne samo za jedan skup ulaznih podataka već za cijeli opseg skupova ulaznih podataka koji su od interesa.

U ovoj fazi validacije, model se posmatra kao ulazno-izlazna transformacija, tj. model prihvata vrijednosti ulaza i transformiše ove ulaze u izlaze koji ukazuju na ponašanje modela.

Umjesto validacije ulazno-izlazne transformacije modela preko predviđanja budućnosti, modelar može koristiti i stare istorijske podatke, koji su sačuvani isključivo za proces validacije (podaci koji nisu korišteni u fazi izgradnje i dogradnje modela).

Da bi se izvršila validacija ulazno-izlazne transformacije, neophodan uslov je postojanje sistema koji se proučava, kako bi bilo moguće prikupiti podatke o sistemu za bar jedan skup ulaznih podataka. Ukoliko je sistem u fazi planiranja i nije moguće prikupiti podatke, potpuna validacija ulazno-izlazne transformacije nije moguća. U nekim slučajevima mogu postojati određeni podsistemi planiranog sistema, pa je moguće sprovesti samo djelimičnu validaciju modela.

Validacija simulacionih modela je bitan i odgovoran posao, jer se brojne i važne odluke zasnivaju na rezultatima eksperimentisanja sa modelom. Zato je obimna i svestrana analiza modela i rezultata dobijenih simulacijom tog modela od velike važnosti za kvalitet simulacije i mogućnosti primjene dobijenih rezultata u procesu odlučivanja.

Najčešće je suviše teško, suviše skupo ili vremenski nemoguće koristiti sve moguće tehnike validacije za svaki model koji se razvija. Stoga je veoma važan posao svakog modelara da odabere one tehnike validacije koje najviše odgovaraju u konkretnom slučaju i da na taj način obezbjedi tačnost i pouzdanost modela.

Formalni kriterijum za utvrđivanje validnosti modela

U skladu sa iznesenim postavkama, realan sistem treba posmatrati kao sistem opisan na višem nivou, dok model za koji treba utvrditi validnost treba da bude na nižem nivou opisa. U ovom poglavlju prikazan je pregled već ranije izložene materije sa stanovišta utvrđivanja validnosti modela.

Homomorfizam (grč. *homo*-sličan, *morph*-struktura) predstavlja formalni kriterijum za utvrđivanje validnosti uprošćenog modela za date eksperimentalne uslove, u odnosu na osnovni model čije su karakteristike objašnjene ranije. Radi jednostavnijeg izlaganja, ograničenje je na dokazivanje validnosti diskretnih vremenskih modela.

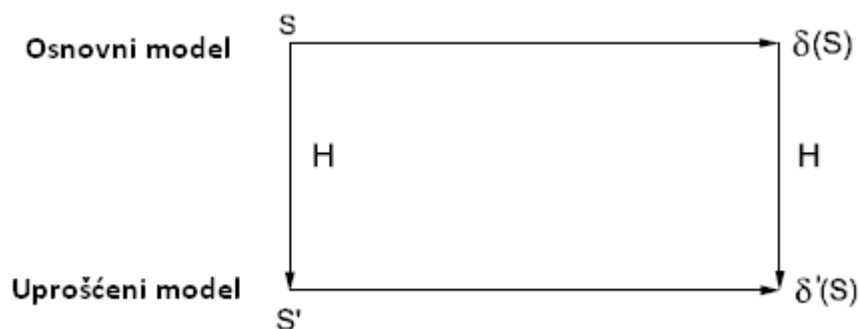
Suština formalnog postupka za provjeru validnosti je u tome da se nađe preslikavanje H pomoću kojeg je moguće iz svakog stanja osnovnog modela (npr. s) preći u odgovarajuće stanje uprošćenog modela (npr. s'). Ako ovakvo preslikavanje postoji, tada se zaključuje da su ulazno-izlazna ponašanja osnovnog i uprošćenog modela ista za date eksperimentalne uslove. Da bi se utvrdio homomorfizam, moraju biti ispunjeni sljedeći uslovi:

1. *Očuvanje funkcije nastupanja vremena*

Stanje u osnovnom modelu (s) i njemu odgovarajuće stanje u uprošćenom modelu (s') moraju imati istu funkciju nastupanja vremena. To znači da će se stanja osnovnog modela i stanja uprošćenog modela mijenjati u istim trenucima vremena.

2. *Očuvanje funkcije prelaza stanja*

Ako stanju s osnovnog modela odgovara stanje s' uprošćenog modela (tj. primjenom preslikavanja H na stanje s dobija se stanje s'), tada i sljedeća stanja u koja će model preći u narednom trenutku posmatranja moraju odgovarati jedno drugom. Sljedeće stanje osnovnog modela će biti $\delta(s)$, a uprošćenog modela $\delta'(s')$, što znači da primjenom preslikavanja H na stanje $\delta(s)$ treba da se dobije stanje $\delta'(s')$. Ovo se grafički može prikazati preko tzv. komutativnih dijagrama (slika 3.10).



Slika 3.10 – Očuvanje funkcije prelaza stanja

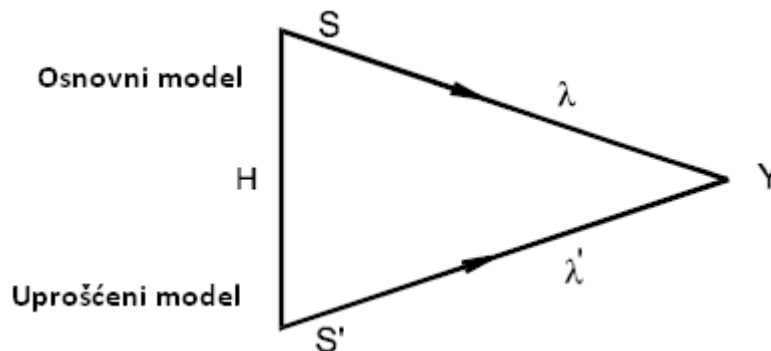
Dakle, polazeći od stanja s osnovnog modela može se prvo vršiti preslikavanje H da bi se došlo u stanje s' uprošćenog modela, a zatim primijeniti funkcija prelaza stanja uprošćenog modela da bi se došlo u stanje $\delta'(s')$; isto tako moguće je prvo primijeniti funkciju prelaza stanja osnovnog modela (prelaz iz stanja s u $\delta(s)$), a zatim preslikavanje H . U oba slučaja dolazi se u isto stanje. Na osnovu izloženog može se zaključiti da preslikavanje H koje važi između stanja osnovnog i uprošćenog modela mora važiti i između njihovih budućih stanja koja se dobijaju na osnovu funkcije prelaza stanja.

3. *Očuvanje izlazne funkcije*

Neka je s stanje osnovnog modela i neka je moguće primjenom preslikavanja H preći u stanje s' uprošćenog modela. Primjena izlazne funkcije osnovnog

modela (za one eksperimentalne uslove za koje je izvršeno uprošćavanje (λ) da bi se dobio uprošćeni model za stanje s), daje isti izlaz Y kao i primjena izlazne funkcije uprošćenog modela (λ') na stanje s' .

Odgovarajući komutativni dijagram dat je na slici 3.11. Ovde se jasno uočava da se validnost uprošćenog modela utvrđuje i u odnosu na zadate eksperimentalne uslove. Ukoliko je preslikavanje H tipa 1:1, u pitanju je izomorfizam.



Slika 3.11 – Očuvanje izlazne funkcije

Ako su zadovoljeni svi prethodno definisani uslovi, onda je uprošteni model validan u odnosu na osnovni model, za date eksperimentalne uslove. Dalje, ako je ranije već bilo tvrdjeno da je osnovni model validan u odnosu na realni sistem (simulirani sistem), tada je i uprošteni model validan u odnosu na realni sistem.

Cjelokupan proces validacije zahtijeva određeno vreme, trud i novčana sredstva. Ovi se faktori svakako moraju uzeti u obzir u postupku validacije i donošenju odluke koliko će se daleko otići u procesu revizije modela. Najčešće, modelar određuje maksimalni prihvatljivi nivo odstupanja modela od realnog sistema. Ukoliko se ovaj nivo odstupanja ne može postići uz ulaganje ograničenih budžetskih sredstava, namijenjenih procesu validacije, modelar ili smanjuje očekivanu preciznost modela ili odbacuje dati model.

3.9.2. Verifikacija simulacionih modela

Iako u dosadašnjim razmatranjima nije bilo riječi o konkretnoj implementaciji simulacionog modela nekim programskim jezikom, tj. o računarskom kodiranju modela, ovdje će ukratko biti ukazano na neke najvažnije karakteristike procesa verifikacije modela, koji, kao što je poznato, polazi baš od računarskog koda modela. Tim prije što proces testiranja ispravnosti jednog modela najčešće prepliće procese verifikacije i validacije, odnosno modelar, kao što je već napomenuto, ta dva ispitivanja najčešće vrši simultano. Dakle, proces verifikacije treba da pokaže da li je i u kojoj mjeri, konceptualni model na odgovarajući način predstavljen računarskim kodom, odnosno u kojoj se mjeri

slažu konceptualni (pretpostavke za komponente sistema i strukturu; vrijednosti parametara; apstrakcije i pojednostavljenja u modelu) i računarski kod.

U postupku verifikacije nema standardnog recepta. Zato je potrebno izvršiti više različitih provjera:

- Ručna verifikacija logičke ispravnosti: model se izvjesno vreme propušta na računaru i ručno, a potom porede dobijeni rezultati.
- Modularno testiranje: pojedinačno testiranje svakog modula kako bi se ustanovilo da li daje razumne izlaze za sve moguće ulaze.
- Provjera u odnosu na poznata rješenja: model se podešava tako da predstavlja sistem čija su rješenja poznata i upoređuju se sa rezultatima modela.
- Testiranje osjetljivosti: varira se jedan parametar, dok ostali ostaju nepromijenjeni i provjerava se da li je ponašanje modela osjetljivo na promjenu tog parametra.
- Testiranje na poremećaje: parametri modela se postavljaju na neprirodne vrijednosti i provjerava se da li se model ponaša na neshvatljiv način. Na taj način se mogu otkriti greške u programu koje je vrlo teško uočiti na drugi način.

Osim navedenog, mogu se primjeniti i neki drugi poznati postupci koji se koriste pri otklanjanju grešaka u računarskim programima.

3.10. Generisanje slučajnih brojeva

U toku specifikacije stohastičkih modela za promjenljive koje opisuju promjenu vremena, ulaza i stanja, definišu se statističke raspodjele koje ih opisuju sa najmanjom greškom u odnosu na realni sistem. Pri simulaciji tako definisanih modela javlja se potreba za računarskim algoritmima koji će generisati slučajne promjenljive za opisivanje promjena vremena i stanja u modelu sa nekom zadatom raspodjelom.

U statističkoj teoriji se dokazuje da je slučajnu promjenljivu sa zadatom raspodjelom moguće generisati na osnovu jedne ili više nezavisnih slučajnih promjenljivih sa uniformnom raspodjelom. Na taj način se problem svodi na generisanje slučajne promjenljive sa uniformnom raspodjelom. Slučajnu promjenljivu sa uniformnom raspodjelom moguće je dobiti jednom od sljedećih metoda:

- manuelne metode,
- tabele slučajnih brojeva,
- metode za rad sa analognim računarima i
- metode za rad sa digitalnim računarima.

3.10.1. Manuelne metode

Manuelne metode su malog opsega i zbog toga su neprimjenljive u simulaciji. Eventualno se mogu koristiti za obučavanje kadrova. U ove metode spadaju: izvlačenje brojeva iz kutije, bacanje kockica itd.

3.10.2. Tabele slučajnih brojeva

Tabele slučajnih brojeva su serije slučajnih brojeva (SB) koji se proizvode na pogodan način, pomoću nekog fizičkog izvora ili nekom od metoda izvlačenja brojeva.

Najveća do sada publikovana tabela slučajnih brojeva, sadrži 1.000.000 cifara (eng. *a million random digits*, Rand Corporation, The Free Press, 1955). Brojevi su dobijeni na sljedeći način. Bio je konstruisan specijalan rulet koji se pokretao i zaustavljao uz pomoć elektronike. Pokretni disk se naglo zaustavljao i birala se cifra koju je pokazivala strelica.

Ovako dobijene cifre se skupljaju u četvorocifrene ili petocifrene brojeve i unose u tabelu slučajnih brojeva. Iako ovakav postupak posjeduje sva svojstva slučajnosti, ipak su se poslije dužeg vremena počele da dobijaju cifre koje, prema provjeri statističkim testom, nisu imale ravnomjernu raspodjelu.

3.10.3. Metode za generisanje SB na analognom računaru

Ove se metode baziraju na korištenju slučajnih fizičkih procesa koji se odvijaju u analognom računaru i daju "prave slučajne brojeve". Nepodesne su za rad, jer u slučaju ponovljene simulacije sa istim redoslijedom slučajnih brojeva, fizički proces daje nove slučajne brojeve.

3.10.4. Metode za generisanje SB na digitalnom računaru

Metode za generisanje slučajnih brojeva baziraju se na sljedećim postupcima:

1. korištenje fizičkih izvora + A/D konverzija,
2. korištenje tabela slučajnih brojeva na diskovima računara,
3. korištenje algoritama za dobijanje pseudoslučajnih brojeva.

Korištenje fizičkih izvora (bijeli šum, radiaktivni raspad) i konverzija fizičkih veličina u digitalne je skup i nepristupačan proces za svakodnevnu primjenu.

Tabele slučajnih brojeva na masovnim memorijama računara su dobar ali spor način za dobijanje slučajnih brojeva pri simulacijama sasvim prosječnih sistema koji u toku simulacije traže nekoliko miliona slučajnih brojeva.

Algoritmi za generisanje pseudoslučajnih brojeva su brz način za generisanje slučajnih brojeva. Međutim kvalitet takvih generatora zavisi od algoritma i računara na kome se implementira. Dužina sekvence brojeva bez ponavljanja je ograničena a uniformnost i rezolucija mogu da budu promjenljive. Zbog toga je potrebno prije upotrebe ovakvih generatora ispitati njihovu rezoluciju, uniformnost i dužinu sekvence slučajnih brojeva bez ponavljanja standardnim statističkim metodama (npr. χ^2 test) i utvrditi da li dobijene karakteristike odgovaraju zahtjevima modela. Međutim, bez obzira na navedene nedostatke algoritamskih generatora slučajnih brojeva (GSB), danas su pretežno u upotrebi ovakvi generatori. Od izbora algoritma zavisi i kvalitet generatora. Postoje mnogi algoritmi za generisanje slučajnih brojeva, ali se uglavnom koriste kongruentni generatori. Njih karakteriše jednostavnost i brzina.

Ako postoji potreba da se testiranje simulacije ponovi sa identičnim vrijednostima, pravi slučajni brojevi nisu potrebni. Umjesto njih, razvijeno je mnoštvo tehnika za generisanje pseudoslučajnih sekvenci brojeva pomoću determinističkih algoritama. Pseudoslučajni brojevi samo izgledaju da su slučajni. Međutim, kod slučajnih brojeva nemoguće je predvidjeti sljedeću vrijednost. Kod pseudoslučajnih sekvenci je unaprijed poznata cijela sekvenca.

Prvi korak u generisanju sekvence slučajnih brojeva koji dolaze iz bilo koje raspodjele je mogućnost generisanja sekvence koja se čini uniformno raspoređenom na intervalu $[0, 1)$. To znači da je vjerovatnoća dobijanja bilo koje vrijednosti iz intervala konstantna. Dobar generator pseudoslučajnih sekvenci treba da posjeduje sljedeće karakteristike:

- *efikasnost*

Pošto će tipična simulacija zahtijevati veliki broj slučajnih vrijednosti, generator treba da lako ostvari efikasno računanje.

- *dugačak period*

Sekvenca slučajnih vrijednosti generisanih od strane konačnog algoritma mora da bude konačna. To znači da će sekvenca neminovno morati nakon nekog parametra k početi da se ponavlja, tako da je $x_{n+k} = x_n$, $x_{n+k+1} = x_{n+1}$... Što je vrijednost k veća, sekvenca će se činiti slučajnijom.

- *nezavisno i uniformno raspoređene vrijednosti*

Dobijene vrijednosti treba da budu uniformno raspoređene na intervalu $[0, 1)$. To znači da svaka od vrijednosti mora imati istu vjerovatnoću pojavljivanja u sekvenci. Dalje, red u kojem se vrijednosti pojavljuju treba da bude nezavisan u statističkom smislu.

- *ponavljajući*

Radi potreba testiranja, poželjno je da generator može da reprodukuje identičnu sekvencu kao što je proizvodivao prethodni put.

3.10.5. Linearni kongruentni GSB

LCG je jedan od najjednostavnijih generatora koji ispunjava sve navedene poželjne karakteristike. Njegovo funkcionisanje se može izraziti pomoću formule:

$$z_i = (a z_{i-1} + c) \bmod m$$

gdje su a , c i m konstante pažljivo izabrane tako da generator ispunjava navedene karakteristike, a z_i predstavlja željenu sekvencu slučajnih vrijednosti. Ako se konstante izaberu na odgovarajući način, generator može da generiše sve vrijednosti iz skupa $\{0, 1, 2, \dots, m-1\}$ u nekom permutovanom redoslijedu. Pošto je $0 \leq z_i < m$, sekvenca vrijednosti $u_i = z_i/m$ će izgledati kao da je uniformno raspoređena na intervalu $[0, 1)$.

Ovakav generator ima nekoliko svojstava koja čine sekvencu vrijednosti koju on generiše drugačijom od prave slučajne sekvence. Na primjer, svaka od dobijenih m vrijednosti se pojavljuje tačno jednom u sekvenci. Na osnovu toga, vjerovatnoća dobijanja bilo koje vrijednosti je $1/m$, čime je zadovoljen uslov uniformnosti. Međutim, pošto se z_i dobija pomoću z_{i-1} , kada se prva vrijednost ponovi, ponoviće se i cijela sekvenca. Posljedica je da je nemoguće da se jedna vrijednost pojavi dva puta zaredom u sekvenci generisanih vrijednosti.

Dalje, pošto se koristi izraz $u_i = z_i/m$ za aproksimiranje vrijednosti raspoređenih na intervalu $[0, 1)$, najmanja razlika između bilo koje dvije vrijednosti je $1/m$. Ova kvantizacijska greška se takođe ne bi pojavila u pravoj slučajnoj sekvenci. Ipak, i uz navedena ograničenja, ovakav tip linearnog kongruentnog generatora može proizvoditi uzorke koji statistički izgledaju kao da dolaze iz željene raspodjele.

Biranje konstanti

Pošto period generatora ne može biti veći od m , poželjno je da ta vrijednost bude što je moguće veća. Takođe, pogodno je da bude jednaka nekoj vrijednosti koja je stepen broja 2, tj. da m bude jednako $m = 2^j$, gdje je j pozitivan cijeli broj. Na osnovu toga se maksimalno pojednostavljuje operacija modulo.

Dodatno je pokazano da će, u slučaju da je $c \neq 0$, maksimalan period biti ostvaren ako i samo ako važi:

1. konstante m i c su uzajamno proste,
2. svaki prost broj koji je faktor od m je takođe faktor i od $a - 1$,
3. ako 4 dijeli m , onda mora da dijeli i $a - 1$.

Ova ograničenja su u potpunosti obuhvaćena ako se konstante izaberu tako da je $m = 2^j$, $a = 4d + 1$, c je neparan broj, a j i d su pozitivni cijeli brojevi.

Na primjer, mnoge implementacije UNIX operativnog sistema koriste sljedeći LCG:

$$z_i = (1\ 103\ 515\ 245\ z_{i-1} + 12\ 345) \bmod 2^{32}.$$

Kod nekih LCG-ova je vrijednost c jednaka nuli. Ovakav izbor vrijednosti za konstantu c eliminiše potrebu za operacijom sabiranja, čime se poboljšava efektivnost procesa generisanja. Međutim, u tom slučaju se period ovakvog multiplikativnog generatora smanjuje sa 2^j na 2^{j-2} . Ako se u tom slučaju j izabere tako da bude prost broj, onda se period generatora povećava na $2^j - 1$. Period kod multiplikativnog generatora nikada ne može da bude 2^j pošto ovaj generator nikada ne može da produkuje vrijednost 0. Primjer dobrog multiplikativnog generatora je:

$$z_i = (16\ 807\ z_{i-1}) \bmod (2^{31} - 1).$$

Upozorenja i preporuke

Pogledom na gornje formule za LC generatore, može se učiniti da je generisanje dobrog generatora slučajnih brojeva prilično jednostavno, što nije tačno. Takođe, ne treba pomiješati kompleksnost sa slučajnošću. Može se desiti da dugačka i kompleksna sekvenca operacija neće proizvoditi dobru sekvencu slučajnih vrijednosti. Ispostavilo se da su najbolji generatori oni koji imaju jednostavnu sekvencu matematičkih operacija i na taj način se njihova ispravnost lako može dokazati analitički. Bitno je još jednom naglasiti da sekvenca dobijena na ovaj način nije nepredvidljiva, već baš suprotno. Međutim, u simulacionim eksperimentima će u većini slučajeva biti potreban generator koji ispunjava uslove statističke, a ne prave slučajnosti.

Završna napomena: Puno je bolje koristiti generator koji se dokazao uspješnim kroz obuhvatna istraživanja nego pokušati razviti svoj generator. U prevodu, ovo ne pokušavajte kod kuće 😊.

Mnogi procesi u računarskim sistemima koji treba da se uključe u simulaciju mogu da se modeluju koristeći sekvencu slučajnih brojeva koja treba da prati neku neuniformnu raspodjelu (na primjer, eksponencijalnu raspodjelu). Generisanje neuniformnih raspodjela je proces iz dva koraka. Prvi korak je generisanje sekvence vrijednosti koja prati uniformnu raspodjelu na intervalu $[0, 1)$ koristeći neku od opisanih tehnika za generisanje uniformnih sekvenci. Drugi korak je transformisanje uniformne sekvence u sekvencu željene raspodjele. Tehnike za transformaciju su: inverzna transformacija, *alias* metoda, dekompozicija, specijalna karakterizacija i *accept-reject* metoda.

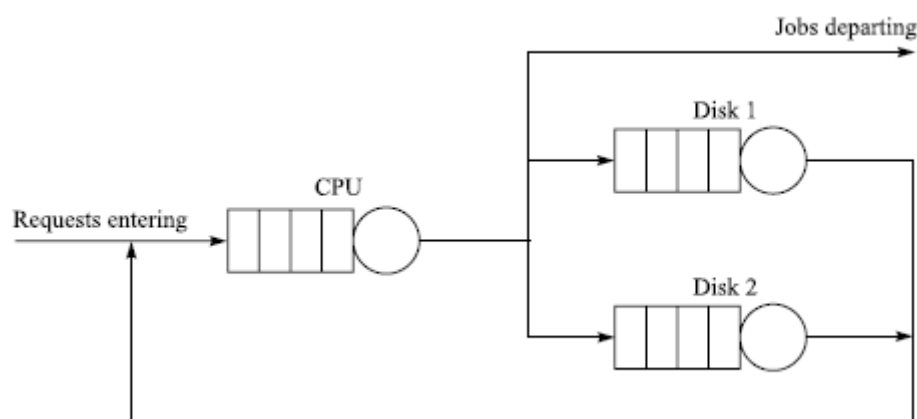
4. ANALIZA REDOVA ČEKANJA

Analiza redova čekanja je najvažnija analitička tehnika za modelovanje u računarskim sistemima. Postoji ogroman broj knjiga na tu temu a i svake godine se objavi na hiljade novih radova i istraživanja. Ovo poglavlje samo dotiče površinu ove oblasti fokusirajući se na fundamentalne rezultate koji formiraju bazu za kompleksnije tehnike analitičkog modelovanja. Srećom, ovi fundamentalni rezultati su u većini slučajeva dovoljni za davanje odgovora na pitanja koja se mogu dati o računarskim sistemima. Najbitnija primjena ovog modela je validacija rezultata dobijenih direktnim mjerenjem ili simulacijom.

4.1. Modeli mreža

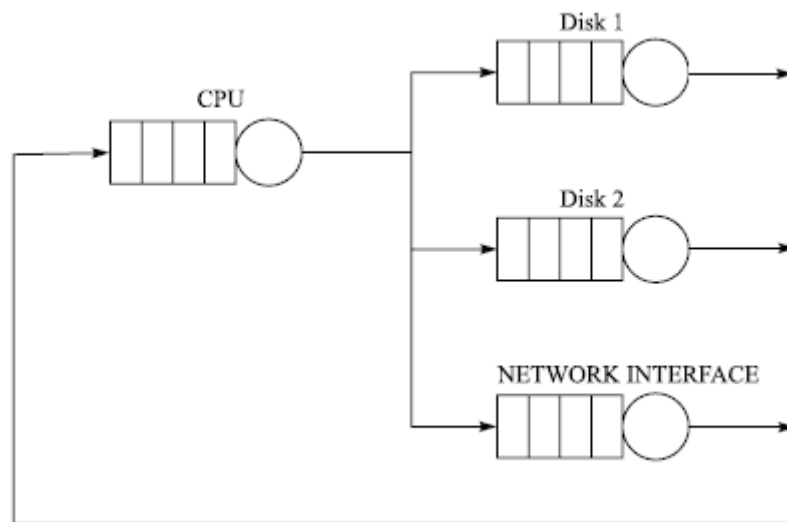
Glavna ideja koja stoji iza analize redova čekanja je da se razmisli o svim poslovima koji moraju da se izvrše u računarskom sistemu. Oni se posmatraju kao mušterije koje zahtijevaju uslugu od strane individualnih podsistema, kao što su memorija, procesor, I/O uređaji... Svaki od ovih resursa, popularno nazvan centar za usluživanje ili server, može procesirati samo jedan posao u jednom trenutku. Drugi poslovi koji žele da koriste zauzeti resurs moraju da čekaju u redu čekanja dok usluživanje trenutnog posla ne završi. Razni serveri i redovi čekanja se na dijagramima povezuju nizom strelica koje pokazuju kako poslovi mogu da putuju između servera u sistemu.

Postoje dva tipa modela mreža redova čekanja. U otvorenom mrežnom modelu, kao što je prikazano na slici 4.1, poslovi pristižu u sistem iz nekog vanjskog izvora. Konkretno, na slici poslovi pristižu sa lijeve strane i prvo se procesiraju od strane CPU-a. Ovo procesiranje može da rezultuje pristupom jednom ili više diskova. Nakon procesiranja zahtjeva, disk podsystemi će vratiti posao CPU-u. Nakon toga, CPU može da generiše nove zahtjeve za diskove bazirane na dobijenom odgovoru ili može da završi procesiranje slanjem posla na izlaznu liniju.



Slika 4.1 – Model otvorene mreže reda čekanja jednostavnog web servera

Iako će svi poslovi koji uđu u sistem na kraju i da izađu iz njega, neograničen broj poslova može da bude u sistemu u bilo kom trenutku. Za razliku od ove karakteristike otvorenih mrežnih modela, zatvoreni mrežni modeli imaju fiksni broj poslova koji cirkulišu između različitih redova čekanja i servera u sistemu. Pošto su zatvoreni mrežni sistemi po svojoj logici bliži ograničenoj prirodi realnih sistema, smatraju se tačnijim modelima od otvorenih mreža. Na nesreću, ostvarivanje matematičkih rješenja u zatvorenim mrežama je tipično mnogo teže nego u otvorenim. Primjer jedne zatvorene mreže je dat na slici 4.2.



Slika 4.2 – Model zatvorene mreže reda čekanja

Prije prelaska na analizu modela otvorenih mreža sa jednim redom čekanja bitno je naglasiti da ovi modeli samo aproksimiraju karakteristike realnih sistema. Samim tim, dobijeni rezultati su samo aproksimacije ponašanja realnih sistema. Ova analiza pruža prosječno ponašanje sistema i ne može se iskoristiti za analiziranje ponašanja individualnog posla.

4.2. Osnovne pretpostavke i notacija

Pretpostavke o sistemu koje je potrebno utvrditi prije same analize su:

- *Balansiranje toka poslova*

Broj poslova koji pristignu na server u dovoljno velikom vremenskom intervalu mora biti jednak broju poslova koji napuste server. Jednostavnije rečeno, ne postoji gubljenje poslova unutar servera ili reda čekanja. Takođe, server ne može samostalno da generiše novi posao. Bitno je napomenuti da ova pretpostavka ne važi za kraće vremenske intervale pošto poslovi moraju čekati u redu dok je server zauzet. Međutim, u dužim vremenskim intervalima broj poslova koji uđu na server mora da se poklapa sa brojem poslova koji budu usluženi.

- *One-step ponašanje*

U bilo kojoj instanci vremena samo jedan posao može da uđe ili da napusti server, što znači da se stanje sistema mijenja samo inkrementalno. Ova pretpostavka striktno ne dozvoljava poslovima da istovremeno vrše prelaze između različitih servera u sistemu.

- *Homogenost*

Homogenost označava da su prosječno pristizanje i usluživanje poslova nezavisni od stanja sistema. Dalje, vjerovatnoća prelaza poslova između dva servera ne zavisi od broja poslova na serverima između kojih se vrši prelaz.

- *Ekskluzivnost*

Posao može biti prisutan samo na jednom serveru, bez obzira da li je u redu čekanja ili se trenutno uslužuje. Na osnovu ove pretpostavke jedan posao ne može da istovremeno traži usluge od više servera. Slično tome, kada se posao uslužuje na serveru, ima ekskluzivno pravo na njega tako da svi drugi poslovi koji žele uslugu od tog servera moraju da čekaju u redu.

- *Nepostojanje blokiranja*

Uslugom koju pruža server ne može da upravlja nijedan drugi uređaj u sistemu. Kada se posao pojavi na vrhu reda čekanja, server mora momentalno početi da ga uslužuje.

- *Nezavisnost*

Poslovi ne smiju da utiču jedan na drugi ni na koji način. Jedini vid interakcije smije da bude činjenica da dijele isti red čekanja. Dalje, ne smije da postoji nikakva sinhronizacija između poslova.

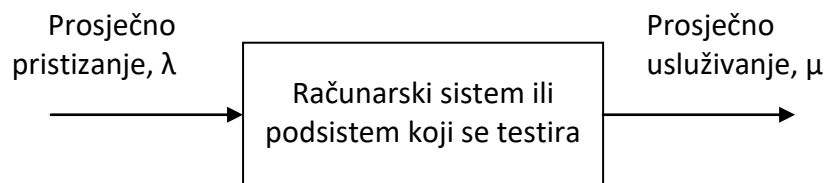
Notacija:

- s – prosječno vrijeme potrebno da bi se uslužio posao (eng. *service*),
- $\mu = 1/s$ – prosječno usluživanje, definiše se kao broj poslova uslužen u jedinici vremena,
- λ – prosječno pristizanje, definiše se kao ukupan broj poslova koji je pristigao u intervalu T kroz taj interval,
- $\rho = \lambda / \mu$ – intenzitet saobraćaja,
- r – prosječno vrijeme odziva, definiše se kao prosječno vrijeme koje posao provede u sistemu, uključujući vrijeme koje provede u redu čekanja plus vrijeme za koje se uslužuje (eng. *response*),
- w – prosječno vrijeme koje posao provede u redu čekanja (eng. *wait*),
- q – prosječan broj poslova u redu čekanja (eng. *queue*),

- n – broj poslova u sistemu, računajući i poslove u redu čekanja i poslove koji se uslužuju,
- U – utilizacija (iskoristivost) sistema, definiše se kao procenat ukupnog vremena u kojem je sistem zauzet (eng. *utilization*),
- a – broj pristizanja koja se dese u nekom intervalu T (eng. *arrival*),
- d – broj odlazaka koji se dese u nekom intervalu T (eng. *departure*),

4.3. Operacionalna analiza

U operacionalnoj analizi sistem se posmatra kao *black box* u kojem poslovi dolaze na jednom kraju sistema, procesiraju se određeno vrijeme, i onda izlaze na drugom kraju. Ovakav pristup dozvoljava analizu redova čekanja bez pretpostavki o statističkoj raspodjeli vremena između pristizanja dva posla u sistem i o raspodjeli vremena između dva usluživanja. Umjesto toga, vrši se direktno mjerenje operacionalnih karakteristika sistema koji se proučava. Na osnovu toga mogu se izvesti zakoni o sistemskom prosječnom ponašanju.



Slika 4.3 – Operacionalna analiza

4.3.1. Utilizacioni zakon

Pretpostavimo da se sistem posmatra određeni vremenski interval T , bilježi broj poslova koji stigne u sistem u tom vremenskom intervalu, kao i vrijeme usluživanja svakog posla. Na osnovu toga, može da se dobije prosječno pristizanje kao količnik broja pristiglih poslova (a) i vremenskog intervala T :

$$\lambda = a / T$$

Prosječno pristizanje je recipročno srednjem vremenu između pristizanja dva posla. Jedinica za prosječno pristizanje je broj poslova u jedinici vremena. Ekvivalentno, prosječno usluživanje (μ) je recipročno prosječnom vremenu usluživanja jednog posla, a mjerna jedinica je identična kao kod prosječnog pristizanja, iako se sada radi o broju usluženih poslova.

Prosječna utilizacija sistema (U) je procenat ukupnog vremena u kojem je sistem bio zauzet usluživanjem poslova. Ako se sistem posmatra u intervalu T i otkrije da je bio zauzet b vremenskih jedinica, onda slijedi da je prosječna utilizacija:

$$U = b / T.$$

Ako se uzme da je d broj usluženih poslova u vremenskom intervalu T , gornja formula može da se napiše na sljedeći način:

$$U = b / T = (b / d) * (d / T).$$

Količnik b/d očigledno predstavlja prosječnu količinu vremena u kojem je uređaj zauzet uslužujući svaki od d poslova. Ovo i predstavlja definiciju prosječnog vremena usluživanja po poslu ($s = b/d$). Dalje, d/T predstavlja prosječno usluživanje (broj odlazaka u jedinici vremena). Pošto vrijedi pretpostavka o balansu poslova, slijedi i da broj odlazaka mora biti jednak broju dolazaka u istom intervalu. Stoga, prosječno usluživanje mora biti jednako prosječnom pristizanju (λ). Ako se ove vrijednosti uvrste u gornju jednačinu, dobija se sljedeća jednakost:

$$U = \lambda s.$$

Prosječna utilizacija uređaja (procenat ukupnog vremena u kojem je sistem zauzet) jednaka je proizvodu prosječnog pristizanja i prosječnog vremena usluživanja jednog posla. Ova jednakost se naziva utilizacioni zakon.

Primjer. Posmatra se I/O sistem koji se sastoji od jednog diska. Ako prosječno vrijeme potrebno za servisiranje svakog zahtjeva iznosi $600 \mu s$, koji je maksimalan broj zahtjeva u vremenu koje sistem može da toleriše?

Maksimalan kapacitet sistema je kada je utilizacija 100% ili $U = 1$. Pošto je $U = \lambda s$, nalazi se da je maksimalan broj zahtjeva u vremenu:

$$\lambda_{\max} = U / s = 1 / (600 \times 10^{-6}) = 1,667 \text{ zahtjeva / s.}$$

Pošto je prosječno usluživanje recipročno prosječnom vremenu usluživanja, ovaj zakon može da se napiše i u sljedećem obliku:

$$U = \lambda / \mu.$$

Ovaj količnik se još naziva i intenzitet saobraćaja. Zbog svoje važnosti u sistemima redova čekanja, intenzitetu saobraćaja dat je i poseban simbol, $\rho = \lambda / \mu$.

Ako je $\rho > 1$, slijedi da je $\lambda > \mu$, tj. prosječno pristizanje je veće od prosječnog usluživanja. Intuitivno se može zaključiti da u tom slučaju poslovi pristižu brže u sistem nego što se mogu procesirati. Posljedica ovoga je da će broj poslova u sistemu rasti neograničeno, što će dalje dovesti do beskonačnih vremena čekanja. Na osnovu ovog razmatranja, jasno je da se u svrhu održavanja stabilnog sistema, prosječno pristizanje mora održavati manjim od prosječnog usluživanja. Drugim riječima, brzina usluživanja poslova mora biti veća od brzine njihovog pristizanja u sistem, tj. ρ mora biti manje od 1 i utilizacija bilo kog uređaja ne smije da bude veća od 100%.

4.3.2. Little-ov zakon

Ako se *black box* sistem sa slike 4.3 posmatra dovoljno velik interval vremena T , vidjeće se da u sistem ulazi a poslova. Ako je ukupno kašnjenje (čekanje) svih poslova u redu čekanja D , onda je prosječno vrijeme čekanja jednog posla (w) jednako:

$$w = D / a.$$

Pošto je $\lambda = a / T$, gornja jednačina se može napisati kao:

$$D = w \lambda T.$$

Sa druge strane, uzimajući u obzir činjenicu da poslovi čekaju samo kada su u redu čekanja, formula za ukupno kašnjenje se može dobiti i kao proizvod prosječnog broja poslova u redu čekanja i vremenskog intervala T u kojem se vrši posmatranje. Znači:

$$D = qT.$$

Izjednačavanjem prethodne dvije jednakosti dobija se:

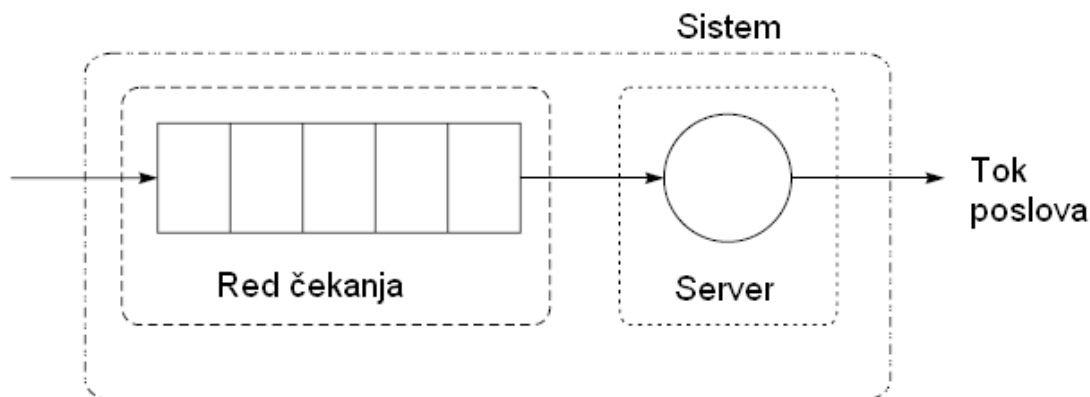
$$qT = w \lambda T.$$

Skraćivanjem sa T , dobija se konačna jednakost:

$$q = w \lambda.$$

Zadnji izraz, poznat kao Little-ov zakon, govori da se prosječan broj poslova u redu čekanja može dobiti kao proizvod prosječnog vremena jednog posla provedenog u redu čekanja i prosječnog pristizanja.

Little-ov zakon je generalan i može se primjeniti na bilo koji dio sistema, kao i na cjelokupan sistem. Jedini preduslov je da se poštuje pretpostavka o balansu poslova u sistemu. Pojedine primjene se mogu uočiti na osnovu slike 4.4.



Slika 4.4 – Little-ov zakon

Slijede tri različite formulacije Little-ovog zakona:

1. Prosječan broj poslova u redu čekanja jednak je proizvodu prosječnog pristizanja i prosječnog vremena koje jedan posao provede čekajući u redu:

$$q = w\lambda.$$

2. Ako se posmatra cijeli sistem, koji uključuje i server i red čekanja, dobija se *black box* pogled na sistem sa slike 4.3. Prosječan broj poslova u sistemu jednak je proizvodu prosječnog pristizanja i prosječnog odzivnog vremena sistema:

$$n = \lambda r.$$

3. Ako se posmatra samo server, isključujući red čekanja, dobija se da je prosječan broj poslova koji se uslužuju jednak proizvodu prosječnog pristizanja i prosječnog vremena usluživanja jednog posla. Na osnovu pretpostavke o ekskluzivnosti, slijedi da je prosječan broj poslova koji se uslužuju jednak utilizaciji servera. Znači:

$$U = \lambda s.$$

Primjer. Prosječno vrijeme odziva *web* servera je 11 ms kada prima 450 zahtjeva u sekundi. Koliki je prosječan broj zahtjeva koji se baferuju u sistemu?

Na osnovu formulacije Little-ovog zakona na sistemskom nivou, $n = \lambda r$, i vrijednosti $\lambda = 450$ zahtjeva/s i $r = 11 \times 10^{-3}$ s slijedi da je $n = 450 \times (11 \times 10^{-3}) = 4.95$ zahtjeva koji su baferovani u sistemu.

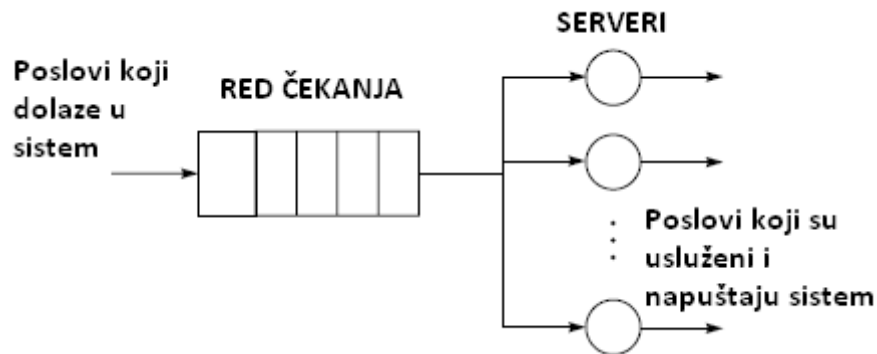
Važno je naglasiti da n i q predstavljaju samo prosječne vrijednosti, a da prave vrijednosti u nekom trenutku mogu da budu znatno veće. Kao posljedica toga, ove vrijednosti se ne mogu iskoristiti za planiranje količine memorije koja se mora rezervisati za smještanje nadolazećih zahtjeva (poslova). Utvrđivanje ovih vrijednosti zahtijeva više informacija o karakteristikama dolazećih zahtjeva. Operacionalna analiza u tom trenutku postaje nemoćna, i stohastička analiza je morala da uđe u igru.

4.4. Stohastička analiza

Utilizacioni i Little-ov zakon pružaju *high-level* pogled na tok poslova kroz sistem ne uzimajući u obzir *low-level* detalje o tome šta se događa unutar sistema. Međutim, ako se izvrše pretpostavke da vrijeme koje prođe između dolaska dva posla i vrijeme potrebno za usluživanje tih poslova prate određene stohastičke raspodjele, mogu se dobiti mnogo detaljniji odgovori nego što je mogla da ih pruži operacionalna analiza.

Fundamentalni model stohastičkog reda čekanja sastoji se od jednog ili više servera koji procesiraju poslove iz jednog reda, kao što je prikazano na slici 4.5. Po ovom modelu vrijeme između dolazaka dva posla u red čekanja je stohastički proces koji se opisuje nekom funkcijom raspodjele. Slično tome, i vrijeme potrebno za usluživanje poslova na serveru se modeluje kao stohastički proces. Ovakav model omogućava da se odrede neki bitni parametri performansi kao što su prosječno vrijeme koje posao provede u redu

čekanja, prosječan broj poslova u redu čekanja, vjerovatnoću postojanja određenog broja poslova u redu i prosječno odzivno vrijeme.



Slika 4.5 – Stohastički model sa jednim redom čekanja i više servera

4.4.1. Kendall-ova notacija

Stohastički model reda čekanja može da se potpuno specifikuje korištenjem šest parametara. Kendall-ova notacija je metoda za predstavljanje tih šest parametara u formi $A/S/c/B/N/D$. Nabrojani simboli imaju sljedeće značenje:

- A označava proces pristizanja poslova u red čekanja (eng. *arrival*). Ovaj stohastički proces opisuje kada poslovi pristižu u red. Još korisnije za analizu je poznavanje vremena između dolazaka različitih poslova. Ako poslovi pristižu u vremenima $T_0, T_1, \dots, T_i, \dots$, vremena $t_1 = T_1 - T_0, t_2 = T_2 - T_1, \dots, t_i = T_i - T_{i-1}$ se nazivaju vremena između dolazaka. Obično se pretpostavlja da ova vremena prate određenu funkciju raspodjele. Najčešći proces koji se koristi u analizi redova čekanja je Poisson-ov proces. U Poisson-ovom procesu vremena između dolazaka prate eksponencijalnu raspodjelu. Eksponencijalna raspodjela ima jednu zanimljivu matematičku osobinu koja uveliko olakšava cjelokupnu analizu koja se opisuje. Ta osobina je da pojava sljedećeg događaja uopšte ne zavisi od vremena pojavljivanja prethodnog događaja. Ovo svojstvo se naziva *memorylessness*². Srećom, pokazano je da mnogi stvarni procesi u računarskim sistemima stvarno prate eksponencijalnu raspodjelu. Eksponencijalna raspodjela se označava simbolom M u Kendall-ovoj notaciji po inicijalu osnovnog svojstva. Ostale raspodjele koje se rijeđe predstavljaju Kendall-ovom

² U matematičkom smislu svojstvo *memorylessness*-a označava sljedeće. U slučaju da eksponencijalna raspodjela ima srednju vrijednost vremena koje se čeka na pojavu sljedećeg događaja x i da je prošlo x vremenskih jedinica od pojave zadnjeg događaja, intuitivno bi se očekivalo da se sljedeći događaj treba pojaviti uskoro (pošto je x srednja vrijednost). Međutim, i poslije x vremenskih jedinica vrijeme očekivanja sljedećeg događaja ostaje x ! U svakom trenutku, bez obzira koliko je proteklo od pojave zadnjeg događaja, ova vrijednost se ne mijenja. Dokaz za ovo svojstvo eksponencijalne raspodjele može da se nađe u bilo kom udžbeniku iz vjerovatnoće i statistike. Na osnovu toga, lako je zaključiti da je vrijeme pojavljivanja sljedećeg očekivanog događaja u potpunosti nepredvidivo.

notacijom su: Erlang-ova (E), hiperekspencijalna (H), deterministička (D) i generalna slučajna raspodjela (G).

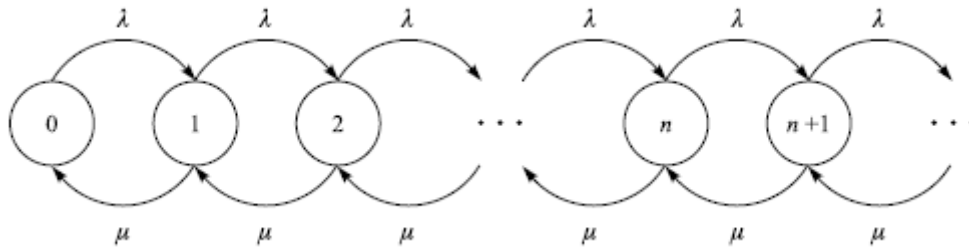
- S označava raspodjelu vremena potrebnih za usluživanje poslova kada napuste red čekanja i pređu na jedan od servera. Notacija je identična kao i kod raspodjele vremena između dolazaka poslova u red čekanja pošto se Poisson-ov proces i ovdje koristi kao najčešće korištena raspodjela.
- c označava broj servera u sistemu. Svaka analiza redova čekanja pretpostavlja da su svi serveri identični, tako da svaki ima istu raspodjelu vremena potrebnih za usluživanje poslova. Sistem u kojem nisu svi serveri identični mora da se rasparča u posebne podsisteme prije nego što se može analizirati tehnikama koje će biti opisane.
- B označava ukupan broj poslova koji mogu da stanu u sistem, računajući poslove u redu čekanja i poslove koji se uslužuju na serveru. Realni sistemi imaju fizičku granicu koliko njihov bafer može da primi poslova. Međutim, analizu je lakše vršiti ako se pretpostavi da ovdje ne postoje ograničenja i da je broj poslova koji može da stane u sistem beskonačan.
- N označava ukupan broj poslova koji bi ikad mogli da uđu u sistem. Kao i za prethodni parametar, analiza se znatno pojednostavljuje ako se i za ovaj parametar pretpostavi da je beskonačan.
- D označava način na koji se poslovi izmještaju iz reda čekanja i smještaju na server radi izvršavanja. Ovaj parametar se naziva disciplina usluživanja. Najčešća upotrebljavana disciplina je FCFS (eng. *First Come, First Served*). Mogu se upotrebljavati i neke druge discipline kao što su LCFS (eng. *Last Come, First Served*) i RR (eng. *Round Robin*).

U većini analiza se pretpostavljaju podrazumijevane vrijednosti za zadnja tri parametra (∞ , ∞ , FCFS) pa se i ne pišu u specifikaciji Kendall-ove notacije. Na osnovu toga, dvije najpopularnije notacije su M/M/1 i M/M/c. U obe navedene verzije vremena između dolazaka poslova u red čekanja i vremena usluživanja poslova na serveru prate ekspencijalnu raspodjelu, a razlikuju se samo po broju servera u sistemu. Na ovom kursu će se detaljno izučavati samo M/M/1 sistemi.

4.4.2. M/M/1 sistemi

Analiza ovih sistema se započinje sa pretpostavkom da poslovi pristižu u red čekanja jedan po jedan, tj. da ne može u jednom trenutku stići grupa poslova odjednom. Na osnovu ove pretpostavke stanje čitavog sistema se može opisati sa cijelim brojem n, koji predstavlja ukupan broj poslova u sistemu (poslovi u redu čekanja i poslovi koji se uslužuju na serveru).

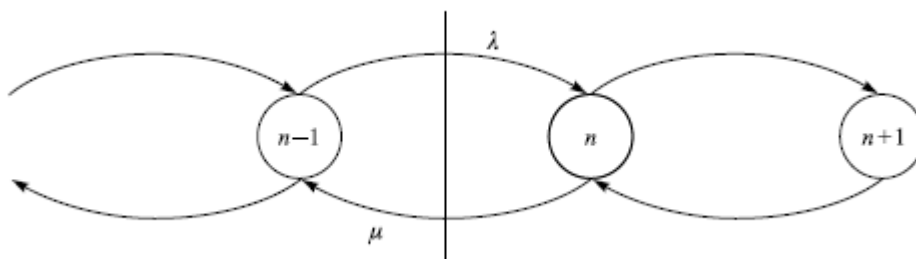
Pristizanje novog posla u sistem se naziva rođenje (eng. *birth*). Ovaj događaj prouzrokuje tranziciju sistema iz stanja n u stanje $n + 1$. Slično tome, kada server završi sa usluživanjem posla sistem prelazi iz stanja n u stanje $n - 1$. Ovaj događaj se popularno naziva smrt (eng. *death*). Čitav proces se naziva proces rođenja i smrti i može se predstaviti dijagramom sa slike 4.6.



Slika 4.6 – Proces rođenja i smrti

Jedna od glavnih pretpostavki u analizi je da sljedeće stanje zavisi isključivo od trenutnog stanja. To znači da sljedeće stanje ne zavisi od sekvence stanja kroz koju je sistem prošao prije dolaska u trenutno stanje. Dalje, sljedeće stanje je takođe potpuno nezavisno od vremenskog kvantuma koje je sistem proveo u trenutnom stanju. Opisana pretpostavka važi samo ako vremena između dolaska dva različita posla, kao i vremena njihovog usluživanja na serveru, prate eksponencijalnu raspodjelu. Konačno, kao što je ranije i navedeno, samo jedno rođenje ili jedna smrt može da se desi u jednom trenutku. Na osnovu toga, sistem vrši tranzicije samo između susjednih stanja. Proces koji ispunjava navedene pretpostavke naziva se Markovljev lanac.

U stabilnom stanju, prosječno prelaženje u stanje n mora biti jednako prosječnom izlaženju iz stanja n . Ako se navedena osobina posmatra na slici 4.7, tok između stanja $n - 1$ i n mora biti jednak u oba smjera. Bez ove pretpostavke, broj poslova u sistemu bi neograničeno rastao. Ponašanje stabilnog stanja se često naziva konzervacija toka (eng. *conservation of flow*).



Slika 4.7 – Jednačina toka

Na osnovu gornjeg razmatranja, mogu se napisati jednačine balansa direktno iz dijagrama sa slike 4.7. Ove jednačine se još nazivaju jednačine toka. Tok za strelicu koja je usmjerena udesno je vjerovatnoća da se sistem nalazi u stanju $n - 1$ pomnožena sa prosječnim pristizanjem poslova u sistem, λ . Slično tome, tok za strelicu koja je

usmjerena ulijevo je vjerovatnoća da se sistem nalazi u stanju n puta prosječno usluživanje poslova u sistemu, μ . Pošto u stabilnom stanju ove dvije vrijednosti moraju da budu jednake, dobija se jednačina:

$$\lambda P_{n-1} = \mu P_n.$$

Rearanžiranjem se dobija sljedeća rekurentna jednakost:

$$P_n = \frac{\lambda}{\mu} P_{n-1} = \rho P_{n-1}.$$

Iz operacionalne analize je poznato da je intenzitet saobraćaja:

$$\rho = \lambda / \mu$$

Takođe, iz rekurentne relacije dobija se da je:

$$P_1 = \rho P_0, P_2 = \rho P_1 = \rho^2 P_0, P_3 = \rho P_2 = \rho^3 P_0, \dots$$

Na osnovu toga može se zaključiti uopštena jednakost vjerovatnoće da se u sistemu nalazi n poslova:

$$P_n = \rho^n P_0.$$

Sljedeći korak je da se ova jednakost transformiše na način da se P_n može izračunati bez poznavanja neke druge vjerovatnoće. Pošto zbir svih vjerovatnoća mora biti jednak 1, dobija se da je:

$$\sum_{n=0}^{\infty} P_n = \sum_{n=0}^{\infty} \rho^n P_0 = 1.$$

Razvijanjem gornje sume dobija se:

$$\rho^0 P_0 + \rho^1 P_0 + \rho^2 P_0 + \dots = 1$$

Na osnovu zadnje jednačine može se dobiti jednakost za P_0 :

$$P_0 = \frac{1}{\rho^0 + \rho^1 + \rho^2 + \dots + \rho^{\infty}} = \sum_{n=0}^{\infty} \frac{1}{\rho^n}.$$

Ako je $\rho < 1$ (što mora da važi u ovim sistemima), može se pokazati da dobijena suma konvergira ka vrijednosti $1 - \rho$. Ako ovu vrijednost vratimo u uopštenu jednačinu za vjerovatnoću da se u sistemu nalazi n poslova, dobija se:

$$P_n = (1 - \rho) \rho^n, n = 0, 1, 2, \dots$$

Na osnovu dobijene jednakosti lako je zaključiti da vjerovatnoća da se u sistemu nalazi određeni broj poslova zavisi isključivo od intenziteta saobraćaja u sistemu, odnosno od odnosa prosječnog usluživanja i prosječnog pristizanja.

Na osnovu zadnje jednakosti mogu se izvesti i drugi parametri performansi za M/M/1 sisteme. Na primjer, prosječan broj poslova u sistemu se dobija izračunavanjem očekivane vrijednosti parametra n:

$$E[n] = \sum_{n=0}^{\infty} nP_n = \sum_{n=0}^{\infty} n(1-\rho)\rho^n = \frac{\rho}{1-\rho}.$$

Korespondirajuća varijansa za dobijeni broj poslova u sistemu je:

$$\begin{aligned} \text{Var}[n] &= E[(n - E[n])^2] = E[n^2] - (E[n])^2 \\ &= \sum_{n=0}^{\infty} n^2(1-\rho)\rho^n - \left(\sum_{n=0}^{\infty} n(1-\rho)\rho^n \right)^2 = \frac{\rho}{(1-\rho)^2}. \end{aligned}$$

Iz Little-ovog zakona je poznato da je prosječan broj poslova u sistemu proizvod prosječnog odzivnog vremena sistema i prosječnog pristizanja. Na osnovu toga može se dobiti prosječno vrijeme odziva:

$$r = \frac{E[n]}{\lambda} = \left(\frac{\rho}{1-\rho} \right) \left(\frac{1}{\lambda} \right) = \frac{1}{\mu(1-\rho)} = \frac{1}{\mu - \lambda}.$$

Pošto se najviše jedan posao može usluživati u bilo kom trenutku, prosječan broj poslova koji čekaju u redu može se izračunati na sljedeći način:

$$q = \sum_{n=1}^{\infty} (n-1)P_n = E[n] - \rho = \frac{\rho^2}{1-\rho}.$$

Korespondirajuće prosječno vrijeme koje posao provede čekajući u redu čekanja je jednostavno izračunati ako se primjeti da je ukupno vrijeme koje posao provede u sistemu zapravo vrijeme odziva. Na osnovu toga slijedi da je prosječno vrijeme koje posao provede u redu čekanja jednako razlici ukupnog vremena koje provede u sistemu i prosječnom vremenu za koje se posao usluži ($1/\mu$):

$$w = r - \frac{1}{\mu} = \frac{\rho}{\mu(1-\rho)}.$$

Konačno, vjerovatnoća da se u sistemu nalazi minimalno k poslova dobija se sumiranjem vjerovatnoća $P_k, P_{k+1}, P_{k+2}, \dots$:

$$\Pr(K \geq k) = \sum_{n=k}^{\infty} P_n = \sum_{n=k}^{\infty} (1-\rho)\rho^n = \rho^k$$

Korištenjem svih dobijenih rezultata, mogu se napraviti neki uopšteni komentari o performansama sistema sa jednim redom čekanja. Prvo, server je zauzet kada se u

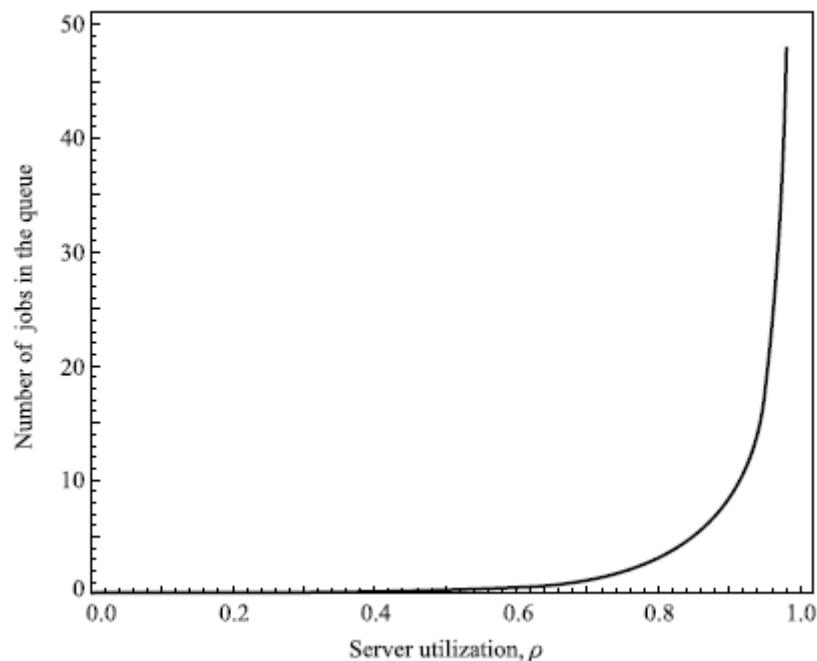
sistemu nalazi jedan ili više poslova. Na osnovu toga, problem izračunavanja utilizacije servera se svodi na problem računanja vjerovatnoće da se u sistemu nalazi minimalno jedan posao. Jedan od načina na koji se može izračunati utilizacija servera je:

$$U = 1 - P_0 = 1 - (1 - \rho) = \rho.$$

Zanimljivo je primjetiti da je identičan rezultat dobijen i u utilizacionom zakonu u operacionalnoj analizi.

Slika 4.8 pokazuje kako se prosječan broj poslova u redu čekanja mijenja kao funkcija utilizacije servera. Na početku se veličina reda sporo povećava sa povećanjem utilizacije servera. Međutim, kada utilizacija dostigne 80%, veličina reda čekanja počinje masovno da se povećava. Ako se utilizacija i dalje povećava, broj poslova u redu čekanja počinje da raste neograničeno. Na osnovu jednačine dobijene za vrijeme odziva sistema, r , primjetno je da će i to vrijeme početi neograničeno da raste kada se utilizacija bude približila 100%.

Na osnovu prethodnog razmatranja, može se zaključiti da je neophodno održavati utilizaciju servera ispod 70%.



Slika 4.8 – Rast broja poslova u redu čekanja kao funkcija od utilizacije servera

Primjer. Mjere se vremena između zahtjeva prema *web* serveru i zaključuje se da ona prate eksponencijalnu raspodjelu sa srednjim vremenom između dva zahtjeva od 8 ms. Vrijeme koje je potrebno serveru da procesira svaki zahtjev je takođe približno eksponencijalno raspoređeno sa prosječnim vremenom usluživanja od približno 5 ms.

- a) Koje je prosječno vrijeme odziva prema korisnicima koji šalju zahtjeve prema ovom serveru?

Pošto jedan zahtjev dolazi po svakih 8 ms, dobija se da je $\lambda = 0.125$ zahtjeva / ms. Na sličan način se zaključuje da je $\mu = 0.2$ posla / ms. Na osnovu zadnje dvije dobijene veličine dobija se da je utilizacija servera $\rho = 0.125 / 0.20 = 0.625$. Drugim riječima, server prosječno radi sa 62.5% od maksimalnog kapaciteta. Prosječno vrijeme odziva je onda:

$$R = 1 / [\mu (1 - \rho)] = 1 / [0.20 (1 - 0.625)] = 13.33 \text{ ms.}$$

- b) Koliko puta brže server mora da procesira zahtjeve da bi se prosječno vrijeme odziva prepolovilo?

Ako se pretpostavi da je broj dolazećih zahtjeva u vremenu ostao isti, želja je da se prosječno vrijeme odziva prepolovi na $13.33 / 2 = 6.67$ ms. Poznato je da je:

$$R = \frac{1}{\mu(1 - \rho)} = \frac{1}{\mu - \lambda}.$$

Rearanžiranjem prethodnog izraza nalazi se da je potrebno da prosječno usluživanje bude u broju poslova u milisekundi:

$$\mu = 1/R + \lambda = 1/6.67 + 0.125 = 0.275.$$

Dobijena vrijednost je 37.5% veća od brzine na postojećem serveru. To znači da se povećavanjem performansi servera za 37.5% prosječno vrijeme odziva duplo smanjuje.

5. INTERPRETACIJA REZULTATA MJERENJA

Nakon dobijenih rezultata mjerenja performansi računarskog sistema, neophodno je da se ti rezultati interpretiraju na odgovarajući način. Kao glavno sredstvo za ovu svrhu koriste se statističke tehnike. Dat je pregled srednjih vrijednosti, uticaj grešaka mjerenja na konačne rezultate, način poređenja dvije i više alternativa, kao i linearnog regresionog modela kao jednog od načina za predviđanje budućeg ponašanja sistema.

5.1. Srednje vrijednosti i varijabilnost

Performanse računarskih sistema su multidimenzionalne. Na osnovu toga, može se zaključiti da je sumiranje sveukupnih performansi sistema jednim brojem prilično nezahvalan posao. Na primjer, računarski sistem može biti optimizovan za izvršavanje nekih tipova programa. Međutim, ova specijalizacija može da prouzrokuje da se neki drugi tipovi programa izvršavaju sa veoma lošim performansama. Pošto će mjerenje vremena izvršavanja različitih programa na takvim sistemima rezultovati širokim spektrom vrijednosti, pokušaj sumiranja performansi jednom srednjom vrijednošću može da prouzrokuje zaključke koji vode u pogrešnom smjeru.

Ipak, bez obzira na sve argumente, u ljudskoj prirodi je da žele da porede različite računarske sisteme na jednostavan način. Na osnovu toga, postoji jak pritisak da se performanse računarskog sistema redukuju na jedan broj koji će na neki način obuhvatiti sve esencijalne aspekte sistema. Iako ovo predstavlja skoro nemoguć zadatak, srednje vrijednosti se mogu iskoristiti za neku vrstu grubog poređenja. Samim tim, bitno je razumjeti kako pravilno izračunati odgovarajuću vrstu srednje vrijednosti i prepoznati kada je srednja vrijednost pogrešno izračunata ili kada se koristi na neodgovarajući način.

Postoje tri različita indeksa centralne tendencije koji se koriste za sumiranje različitih mjerenja: *mean*, *median* i *mode* vrijednosti.

1. Aritmetička sredina (eng. *mean*)

Predstavlja najčešće korišten indeks centralne tendencije. Ovaj indeks je aproksimacija prave srednje vrijednosti slučajnog procesa koji se mjeri. Prava srednja vrijednost se nikada ne može direktno izračunati pošto bi za njeno izračunavanje bio potreban beskonačan broj mjerenja. Samim tim, aritmetička sredina predstavlja najbolji način njene aproksimacije. U sljedećoj sekciji će se opisati tehnike koje pokazuju koliko je dobijena sredina zapravo blizu prave srednje vrijednosti. Formula za izračunavanje aritmetičke sredine u slučaju n mjerenja je:

$$\bar{x}_A = \frac{1}{n} \sum_{i=1}^n x_i.$$

2. median

Jedna od karakteristika aritmetičke sredine je davanje jednake važnosti svakom od mjerenja. Kao posljedica te osobine, jedna izmjerena vrijednost koja značajno odstupa od ostalih, popularno nazvana *outlier*, može značajno da utiče na izračunatu vrijednost. Korištenje *median* vrijednosti sprječava ovu neželjenu pojavu. Naime, pri računanju ove vrijednosti se sve izmjerene vrijednosti prvo sortiraju i onda se uzima vrijednost u sredini. U slučaju da je izmjeren paran broj vrijednosti, uzima se srednja vrijednost centralne dvije.

3. mode

Ovaj indeks predstavlja vrijednost koja se najčešće pojavljuje. Zanimljivo je primjetiti da ova vrijednost ne mora uvijek da postoji, i to u slučajevima kada nijedna vrijednost nije izmjerena više puta od drugih. Mode vrijednost je pogodna kada se sumiraju kategorični podaci. U tom slučaju nema smisla tražiti neku kvantitativnu srednju vrijednost.

Opisani indeksi predstavljaju samo oblike aritmetičke sredine. Osim nje, postoje još i harmonijska i geometrijska sredina. Na određeni skup mjerenja se može primjeniti bilo koja od navedene tri sredine. Međutim, bitno je odabrati pravu sredinu u odgovarajućoj situaciji. Na primjer, ako se mjere vremena izvršavanja, onda rezultujuća srednja vrijednost treba da bude direktno proporcionalna ukupnom izmjerenom vremenu. To znači da povećavanje vremena izvršavanja n puta treba da prouzrokuje i da se srednja vrijednost poveća n puta. Suprotno tome, srednja vrijednost dobijena sumiranjem odnosa (broj operacija u jedinici vremena) treba da bude inverzno proporcionalna ukupnom izmjerenom vremenu. To znači da povećavanje vremena izvršavanja n puta treba da prouzrokuje da se srednja vrijednost smanji n puta.

Kroz narednu diskusiju koristiće se sljedeće pretpostavke. Mjeri se vrijeme izvršavanja n benčmark programa na istom sistemu. Dobijena vremena se označavaju sa T_i , gdje je $1 \leq i \leq n$. Dalje, pretpostavlja se da je ukupan posao koji odradi svaki od benčmark programa konstantan. Specificira se da svaki od benčmark programa izvršava F *floating-point* operacija. Na osnovu toga prosječna brzina benčmark programa i iznosi $M_i = F / T_i$ *floating-point* operacija u sekundi.

5.1.1. Aritmetička sredina

Aritmetička sredina se definiše kao što je prethodno navedeno:

$$\bar{x}_A = \frac{1}{n} \sum_{i=1}^n x_i.$$

U gornjoj formuli vrijednosti x_i predstavljaju pojedinačna mjerenja. Po zadatim pretpostavkama x_i predstavlja T_i , pa slijedi da je prosječno vrijeme izvršavanja:

$$\bar{T}_A = \frac{1}{n} \sum_{i=1}^n T_i.$$

Zadnja jednačina daje vrijednost koja je direktno proporcionalna ukupnom vremenu izvršavanja. Na osnovu toga slijedi da je aritmetička sredina pravo sredstvo za sumiranje vremena izvršavanja. Međutim, ako se aritmetička sredina koristi za sumiranje odnosnih vrijednosti dobija se:

$$\bar{M}_A = \frac{1}{n} \sum_{i=1}^n M_i = \sum_{i=1}^n \frac{F/T_i}{n} = \frac{F}{n} \sum_{i=1}^n \frac{1}{T_i}.$$

Zadnja jednačina govori da je prosječna vrijednost inverzno proporcionalna sumi vremena izvršavanja što se kosi sa logikom ove vrijednosti. Zaključak je da aritmetička sredina nije pogodna za sumiranje odnosnih veličina.

5.1.2. Harmonijska sredina

Harmonijska sredina se definiše kao:

$$\bar{x}_H = \frac{n}{\sum_{i=1}^n 1/x_i}$$

Vrijednosti x_i imaju isto značenje kao kod aritmetičke sredine. Ako se ova vrijednost koristi za sumiranje vremena izvršavanja ($x_i = T_i$), dobija se sljedeći izraz:

$$\bar{T}_H = \frac{n}{\sum_{i=1}^n 1/T_i}.$$

Dobijena vrijednost očigledno nije direktno proporcionalna ukupnom vremenu izvršavanja, što se kosi sa svojstvima dobrih metrika. Lako je zaključiti da harmonijska sredina nije pogodna za sumiranje vremena izvršavanja.

Sa druge strane, dobija se da je harmonijska sredina pogodna za sumiranje odnosnih veličina, gdje je $x_i = M_i = F / T_i$:

$$\bar{M}_H = \frac{n}{\sum_{i=1}^n 1/M_i} = \frac{n}{\sum_{i=1}^n T_i/F} = \frac{Fn}{\sum_{i=1}^n T_i}.$$

Dobijena vrijednost, koja predstavlja ukupan broj operacija izvedenih u svim benčmark programima podijeljen sa sumom svih vremena izvršavanja, očigledno je inverzno proporcionalna ukupnom vremenu izvršavanja. Na osnovu toga slijedi da je harmonijska sredina pogodna za sumiranje mjerenja odnosnih veličina.

Primjer izračunavanja aritmetičke i harmonijske sredine dat je u tabeli 5.1:

Tabela 5.1 – Izračunavanje aritmetičke i harmonijske sredine

Mjerenje (i)	T_i (s)	F (10^9 FLOP)	M_i (MFLOPS)
1	321	130	405
2	436	160	367
3	284	115	405
4	601	252	419
5	482	187	388
$\sum_{i=1}^5 x_i$	2124	844	
$\overline{T}_A$	425		
$\overline{M}_H$			396

5.1.3. Geometrijska sredina

Geometrijska sredina se definiše kao n -ti korijen proizvoda n pojedinačnih x_i vrijednosti:

$$\bar{x}_G = \sqrt[n]{x_1 x_2 \cdots x_i \cdots x_n} = \left(\prod_{i=1}^n x_i \right)^{1/n}.$$

Geometrijska sredina, kao što će se pokazati u nastavku teksta, nije pogodna za sumiranje ni vremena ni odnosnih veličina.

Za glavnu prednost geometrijske sredine se smatra održavanje konzistentnih odnosa između mjerenih alternativa bez uticaja od izbora baze koja se koristi za normalizaciju. U tabeli 5.2 je prikazan primjer mjerenja tri različita sistema uz prikaz odgovarajućih geometrijskih sredina. Na osnovu njih se zaključuje da sistem S_3 pokazuje najbolje performanse, slijedi ga sistem S_2 , a najslabiji je sistem S_1 . U tabeli 5.3 su prikazani isti sistemi, ali normalizovani tako da je sistem S_1 poslužio kao baza, a u tabeli 5.4 sistem S_2 je poslužio kao baza. Očigledno je da je rangiranje sistema po performansama stvarno konzistentno.

Tabela 5.2 – Izračunavanje geometrijske sredine na tri različita sistema

Program	S_1	S_2	S_3
1	417	244	134
2	83	70	70
3	66	153	135
4	39,449	33,527	66,000
5	772	368	369
Geometrijska sredina	587	503	499
Poredak	3	2	1

Tabela 5.3 – Prikaz geometrijskih sredina normalizovanih prema sistemu S_1

Program	S_1	S_2	S_3
1	1.0	0.59	0.32
2	1.0	0.84	0.85
3	1.0	2.32	2.05
4	1.0	0.85	1.67
5	1.0	0.48	0.45
Geom. sredina	1.0	0.86	0.84
Poredak	3	2	1

Tabela 5.4 – Prikaz geometrijskih sredina normalizovanih prema sistemu S_2

Program	S_1	S_2	S_3
1	1.71	1.00	0.55
2	1.19	1.00	1.00
3	0.43	1.00	0.88
4	1.18	1.00	1.97
5	2.10	1.00	1.00
Geom. sredina	1.17	1.00	0.99
Poredak	3	2	1

Nažalost, iako korištenje geometrijske sredine rezultuje konzistentnim rangiranjem poređenih sistema, samo rangiranje je pogrešno. Ako se primjeni aritmetička sredina na istim sistemima, dobija se potpuno drugačiji poredak, što je uočljivo iz tabele 5.5. Aritmetičkoj sredini treba vjerovati u ovom slučaju zbog filozofije sistema. Zaključak je da je geometrijska sredina konzistentno pogrešna.

Tabela 5.5 – Prikaz aritmetičkih sredina u cilju pokazivanja da geometrijska sredina nije odgovarajuća za sumarizaciju ni vremena ni odnosnih veličina

Program	S_1	S_2	S_3
1	417	244	134
2	83	70	70
3	66	153	135
4	39,449	33,527	66,000
5	772	368	369
Ukupno vrijeme	40,787	34,362	66,798
Aritm. sredina	8157	6872	13,342
Poredak	2	1	3

Lako je i formalno pokazati zbog čega geometrijska sredina rezultuje pogrešnim poretom sistema kada se koristi za izvođenje prosječnih vremena izvršavanja. U ovom slučaju, kada je $x_i = T_i$, dobija se:

$$\overline{T}_G = \left(\prod_{i=1}^n T_i \right)^{1/n}.$$

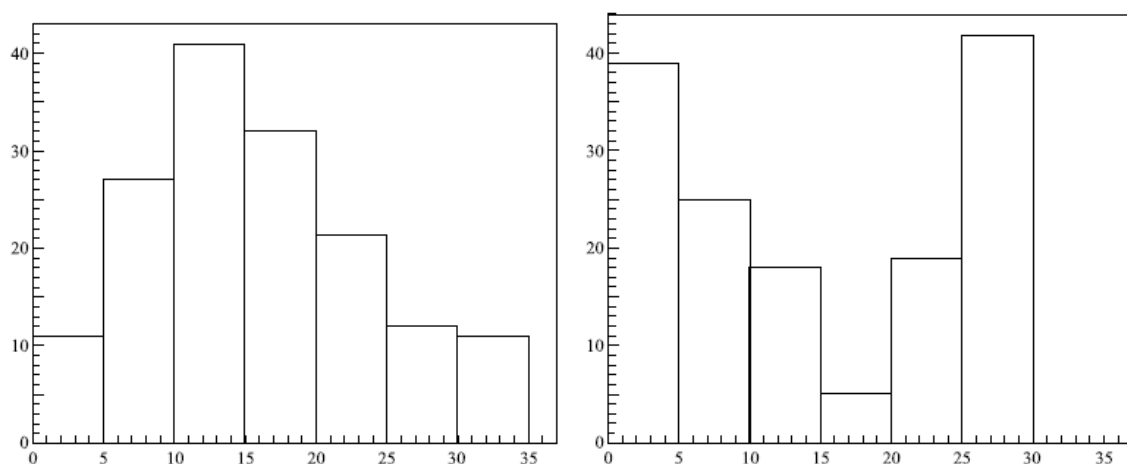
Vrijednost očigledno nije direktno proporcionalna ukupnom vremenu izvršavanja. Slično, izvođenjem prosječne vrijednosti za odnosne veličine dobija se:

$$\overline{M}_G = \left(\prod_{i=1}^n M_i \right)^{1/n} = \left(\prod_{i=1}^n \frac{F}{T_i} \right)^{1/n}$$

Ovaj izraz nije inverzno proporcionalan ukupnom vremenu izvršavanja. Očigledno je da obe dobijene vrijednosti krše karakteristike dobrih metrika.

5.1.4. Kvantifikovanje varijabilnosti

Iako su srednje vrijednosti korisne u smislu da sumiraju velike količine podataka jednim brojem, one ipak kriju detalje o raspodjeli sumiranih podataka. Često je ta raspodjela, popularno nazvana varijabilnost, interesantnija od srednje vrijednosti. Na primjer, podaci sa dva dijagrama na slici 5.1 imaju približno istu srednju vrijednost iako im se raspodjele u potpunosti razlikuju.



Slika 5.1 – Problem kvantifikovanja varijabilnosti

Najbolji način za predstavljanje varijabilnosti je varijansa. Varijansa se definiše kao:

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}$$

Ova vrijednost zavisi od udaljenosti svakog mjerenja od srednje vrijednosti. Suma ovih udaljenosti se dijeli sa $n - 1$ zato što je samo $n - 1$ svih mjerenja nezavisno. To znači

da se rezultat n -tog mjerenja može izračunati ako su poznati rezultati ostalih $n - 1$ mjerenja i srednja vrijednost. Kaže se da je broj stepeni slobode u prethodnoj jednačini $n - 1$. Međutim, problem u izračunavanju varijanse pomoću prethodne jednačine je što se mora dva puta proći kroz skup vrijednosti pošto je neophodno poznavati srednju vrijednost da bi se izračunala varijansa. Zbog toga se može koristiti i sljedeći oblik jednačine za računanje varijanse:

$$\begin{aligned}s^2 &= \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} = \frac{1}{n - 1} \sum_{i=1}^n (x_i^2 - 2\bar{x}x_i + \bar{x}^2) \\ &= \frac{n \sum_{i=1}^n x_i^2 - (\sum_{i=1}^n x_i)^2}{n(n - 1)}.\end{aligned}$$

Problem korištenja varijanse može da predstavlja nemogućnost poređenja sa srednjom vrijednošću pošto vrijednost varijanse predstavlja kvadrat jedinice kojom se vrši mjerenje. Za navedeno poređenje je pogodna standardna devijacija, koja zapravo predstavlja kvadratni korijen varijanse:

$$s = \sqrt{s^2} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}.$$

Konačno, koeficijent varijacije (eng. *coefficient of variation*, COV) eliminiše problem pojedinih jedinica normalizujući standardnu devijaciju sa srednjom vrijednošću. COV predstavlja bezdimenzionalnu veličinu i definiše se kao:

$$COV = s/\bar{x}$$

5.2. Greške u eksperimentalnim mjerenjima

U pokušaju mjerenja i razumijevanja performansi računarskih sistema, postoji konstantna suočenost sa suštinskim detaljima stvarnog svijeta koji unose nesigurnost u izvršena mjerenja. Te nesigurnosti se popularno nazivaju greške ili šum. Da bi se odredilo koliko nesigurnosti postoji u izvršenim mjerenjima, neophodno je da se koriste statistički alati i tehnike radi kvantifikovanja grešaka.

Svaki alat za mjerenje (na primjer, tajmer za mjerenje vremenskih intervala) ima tri bitne karakteristike:

- *tačnost*

Tačnost predstavlja razliku između izmjerene vrijednosti i odgovarajuće referentne vrijednosti.

- *preciznost*

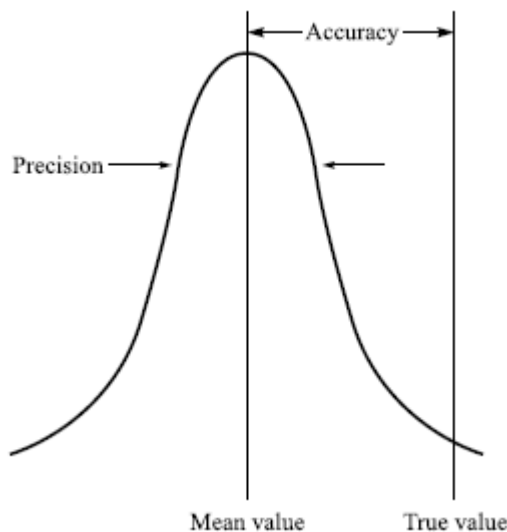
Preciznost se odnosi na ponavljajući efekat mjerenja. Može se i obrnuto definisati. Nepreciznost predstavlja koliko su široko razbacane vrijednosti

mjerjenja određene karakteristike sistema koji se mjeri. Slijedi da je veoma precizan skup mjerjenja grupisan oko jedne tačke, a da je nepreciznost veća što su pojedinačna mjerjenja više raširena.

- *rezolucija*

Rezolucija predstavlja najmanju inkrementalnu promjenu koja može da se detektuje i prikaže. Konačna rezolucija mjernog alata unosi kvantizacijsku grešku u izmjerene vrijednosti.

Kao uvid u razliku između tačnosti i preciznosti, može se posmatrati slika 5.2. Preciznost skupa mjerjenja se ogleda u raširenosti mjerjenja oko dobijene srednje vrijednosti. Tačnost, sa druge strane, predstavlja razliku između dobijene srednje vrijednosti i nepoznate "prave" srednje vrijednosti. Lako je primjetiti da netačna mjerjenja mogu biti savršeno precizna u slučaju male raširenosti izmjerenih vrijednosti od dobijene srednje vrijednosti koja je znatno udaljena od "prave" srednje vrijednosti.



Slika 5.2 – Razlika između tačnosti i preciznosti

Za kvantifikovanje preciznosti skupa mjerjenja uglavnom se koristi varijansa. Kvantifikovanje tačnosti mjerjenja nije toliko jednostavno. Na primjer, kvantifikovanje tačnosti tajmera bi zahtijevalo kalibraciju sa standardnim mjerenjem vremena. Umjesto toga, jednostavno se vjeruje tačnosti alata, a preciznost se kvantifikuje pomoću intervala povjerenja, koji će biti opisani kasnije u ovoj glavi.

Kao posljedica mnogih faktora, kako determinističkih tako i neterminističkih, različita mjerjenja istog događaja mogu da produkuju različite rezultate mjerjenja. Sa tačke gledišta analitičara performansi, sva različita mjerjenja predstavljaju procjenu prave srednje vrijednosti. Razlike između tih mjerjenja se ne mogu predvidjeti i moraju se tretirati kao greške u mjerenjima.

Korisno je klasifikovati sve različite izvore grešaka u dvije velike klase – sistematske i slučajne greške. Sistematske greške su posljedica neke "greške" u eksperimentu (na

primjer, promjena u eksperimentalnom okruženju ili nekorektna procedura). Ova klasa grešaka je uglavnom konstantna kroz sva mjerenja ili sporo varira u vremenu. Ove greške utiču na tačnost izvršenih mjerenja. Kontrolisanje i eliminisanje sistematskih grešaka je zadatak izvodilaca eksperimenta i većinom zavisi od njihovih vještina. Sa druge strane, slučajne greške su u potpunosti nepredvidive, nedeterminističke i ne mogu se kontrolisati. Takođe, slučajna greška može sa jednakom vjerovatnoćom i povećati i smanjiti rezultat mjerenja.

Iako je nemoguće predvidjeti efekat pojedinih izvora slučajnih grešaka, moguće je razviti statistički model pomoću kojeg bi se opisao njihov ukupan efekat na eksperimentalne rezultate, odnosno pomoću kojeg bi se kvantifikovala preciznost izvršenih mjerenja.

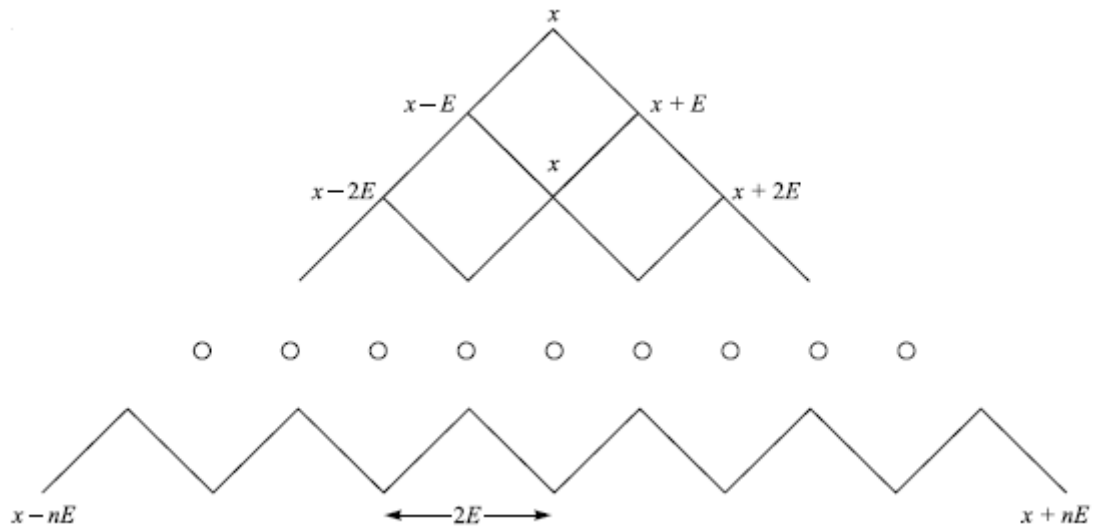
Za slučajne greške se pretpostavlja da prate Gausovu raspodjelu. To znači da, ako se izvrši više mjerenja iste veličine, ova mjerenja će da budu centrirana oko srednje vrijednosti. U nastavku teksta je predstavljen jednostavan model grešaka pomoću kojeg bi se na neki način dokazalo zbog čega je razumno pretpostaviti da mjerenja prate Gausovu raspodjelu.

Na početku, pretpostavka je da jedan izvor slučajnih grešaka može da promijeni mjerenu vrijednost za $+E$ ili $-E$ sa jednakom vjerovatnoćom. Ovo znači da će taj izvor slučajnih grešaka prouzrokovati da se vrijednost $x + E$ izmjeri u pola mjerenja, a vrijednost $x - E$ u drugoj polovini. Ako postoje dva takva izvora slučajnih grešaka, onda postoje četiri moguće kombinacije djelovanja greške na izmjerenu vrijednost. Kao što je prikazano u tabeli 5.6, ove četiri kombinacije rezultuju u tri moguća ishoda pošto se neke greške međusobno poništavaju.

Tabela 5.6 – Dva izvora slučajnih grešaka

Greška 1	Greška 2	Izmjerena vrijednost	Vjerovatnoća
$-E$	$-E$	$x - 2E$	$\frac{1}{4}$
$-E$	$+E$	x	$\frac{1}{4}$
$+E$	$-E$	x	$\frac{1}{4}$
$+E$	$+E$	$x + 2E$	$\frac{1}{4}$

Ako se sad razmatranje generalizuje sa dva na n izvora slučajnih grešaka, dobija se dijagram na slici 5.3. Pošto postoji n izvora slučajnih grešaka, potencijalan opseg vrijednosti koje mogu da se izmjere je $[x - nE, x + nE]$, i to sa korakom od $2E$ između svake od mogućih $n + 1$ vrijednosti. Takođe, pošto se do nekih ishoda može doći različitim putanjama, nisu sve moguće vrijednosti jednako vjerovatne. Iako je pokazati da ova raspodjela predstavlja binomnu. Dalje, kako n više raste, binomna raspodjela se približava Gausovoj raspodjeli.



Slika 5.3 – Model slučajnih grešaka

5.2.1. Kvantifikovanje grešaka

Kao što je ranije navedeno, veoma je teško kvantifikovati tačnost mjerenja. Međutim, kvantifikovanje preciznosti se može vršiti korištenjem intervala povjerenja.

Intervali povjerenja za srednju vrijednost

Ako se raspodjela slučajnih grešaka u vršenim mjerenjima može aproksimirati Gausovom raspodjelom, mogu se iskoristiti jedinstvene karakteristike te raspodjele radi utvrđivanja koliko je blizu dobijena procijenjena srednja vrijednost u odnosu na pravu srednju vrijednost koja se želi utvrditi. U ovu svrhu se koriste statistički intervali povjerenja da bi se našao opseg vrijednosti koji ima određenu vjerovatnoću da se u njemu nalazi prava srednja vrijednost.

Prvi slučaj : broj mjerenja je velik ($n \geq 30$)

Aritmetička sredina n mjerenja ($\bar{x}$) se uglavnom koristi kao najbolja aproksimacija prave srednje vrijednosti x . Ako su sva mjerenja ($x_1, x_2, \dots, x_n$) nezavisna i dolaze iz iste populacije sa srednjom vrijednošću μ i standardnom devijacijom σ , centralna granična teorema garantuje da za dovoljno velike vrijednosti n (uglavnom se pretpostavlja da je broj mjerenja dovoljno velik ako je veći ili jednak od 30) aritmetička sredina $\bar{x}$ prati Gausovu raspodjelu sa srednjom vrijednošću μ i standardnom devijacijom $\sigma / \sqrt{n}$. U ovom slučaju se pretpostavlja da μ predstavlja pravu vrijednost x koja je cilj mjerenja.

Da bi se kvantifikovala preciznost skupa mjerenja, cilj je pronalaženje dvije vrijednosti, c_1 i c_2 , takvih da se sa vjerovatnoćom od $1 - \alpha$ može reći da se prava srednja

vrijednost koja se želi izmjeriti nalazi između ove dvije vrijednosti. Ova vjerovatnoća predstavlja površinu ispod krive između vrijednosti c_1 i c_2 sa slike 5.4. To se može iskazati sljedećom jednačinom:

$$\Pr[c_1 \leq x \leq c_2] = 1 - \alpha.$$

Tipično se vrijednosti za c_1 i c_2 uzimaju tako da formiraju simetričan interval tako da je:

$$\Pr[x < c_1] = \Pr[x > c_2] = \frac{\alpha}{2}.$$

Interval $[c_1, c_2]$ se naziva interval povjerenja za srednju vrijednost $\bar{x}$, a $(1 - \alpha) \cdot 100$ se naziva nivo povjerenja. Sljedeća normalizacija transformiše $\bar{x}$ da prati Gausovu raspodjelu sa srednjom vrijednošću $\mu = 0$ i varijansom $\sigma^2 = 1$:

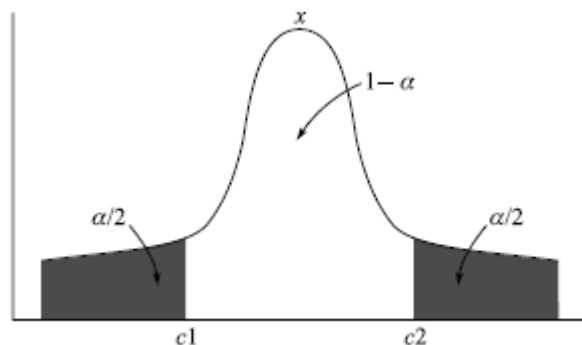
$$z = \frac{\bar{x} - x}{\sigma/\sqrt{n}}$$

Primjenom centralne granične teoreme dobija se da je:

$$c_1 = \bar{x} - z_{1-\alpha/2} \frac{s}{\sqrt{n}}$$

$$c_2 = \bar{x} + z_{1-\alpha/2} \frac{s}{\sqrt{n}}$$

U gornjim jednačinama $\bar{x}$ predstavlja aritmetičku sredinu, s standardnu devijaciju, n broj mjerenja, a $z_{1-\alpha/2}$ je vrijednost koja se iščitava iz tabele u dodatku na kraju skripte.



Slika 5.4 – Interval povjerenja

Drugi slučaj : broj mjerenja je malen ($n < 30$)

Kada je broj mjerenja veći od 30, varijansa s^2 predstavlja dobru procjenu prave varijanse raspodjele σ^2 . Međutim, ovo ne važi kada je broj mjerenja manji od 30. Pokazuje se da raspodjela normalizovane vrijednosti t sada prati t distribuciju sa $n - 1$ stepeni slobode. U tom slučaju formule za c_1 i c_2 su:

$$c_1 = \bar{x} - t_{1-\alpha/2; n-1} \frac{s}{\sqrt{n}}$$

$$c_2 = \bar{x} + t_{1-\alpha/2; n-1} \frac{s}{\sqrt{n}}$$

Jedina razlika u odnosu na formule dobijene kada je $n \geq 30$ je što se sad uzimaju vrijednosti iz tabele t raspodjele u prilogu A. t raspodjela je slična Gausovoj raspodjeli i one postaju identične kada je broj mjerenja velik. Iz tog razloga, zadnji red u tabeli t raspodjele označen sa ∞ predstavlja vrijednosti Gausove raspodjele za odgovarajuću vjerovatnoću.

Primjer. Treba odrediti prosječno vrijeme upisivanja fajla određene veličine na disk. Osam mjerenja su produkovala sljedeće vrijednosti: 8.0, 7.0, 5.0, 9.0, 9.5, 11.3, 5.2 i 8.5. Takođe treba odrediti interval povjerenja za dobijenu srednju vrijednost da bi se kvantifikovala preciznost mjerenja.

Suma svih mjerenja i aritmetička sredina su jednaki:

$$\sum x_i = 8 + 7 + \dots + 8.5 = 63.5$$

$$\bar{x} = \frac{\sum x_i}{n} = \frac{63.5}{8} = 7.94.$$

Vrijednost sume kvadrata mjerenja koja je potrebna za računanje standardne devijacije je:

$$\sum x_i^2 = 8^2 + 7^2 + \dots + 8.5^2 = 536.23.$$

Na osnovu toga dobija se da je standardna devijacija:

$$s = \sqrt{\frac{8(536.23) - 63.5^2}{8(7)}} = 2.14.$$

Ako se izabere 90%-tni interval povjerenja, slijedi da je $\alpha = 0.10$. Pošto je izvršeno 8 mjerenja, mora se koristiti t raspodjela sa 7 stepeni slobode. Iz tabele t raspodjele dobija se da je:

$$t_{1-\alpha/2; n-1} = t_{0.95; 7} = 1.895.$$

Ako se dobijene vrijednosti uvrste u jednačine za granice intervala povjerenja, za c_1 i c_2 dobija se:

$$c_1 = 7.94 - \frac{1.895(2.14)}{\sqrt{8}} = 6.5$$

$$c_2 = 7.94 + \frac{1.895(2.14)}{\sqrt{8}} = 9.4.$$

Na sličan način se mogu dobiti vrijednosti i za 95%-tni interval povjerenja:

$$c_1 = \bar{x} - t_{0.975;7} \frac{s}{\sqrt{n}} = 7.94 - \frac{2.365(2.14)}{\sqrt{8}} = 6.1$$

$$c_2 = \bar{x} + t_{0.975;7} \frac{s}{\sqrt{n}} = 7.94 + \frac{2.365(2.14)}{\sqrt{8}} = 9.7.$$

Zaključak je da postoji vjerovatnoća od 90% da se prava srednja vrijednost upisivanja fajla na disk nalazi u intervalu (6.5, 9.4). Takođe, postoji vjerovatnoća od 95% da se prava srednja vrijednost upisivanja fajla na disk nalazi u intervalu (6.1, 9.7). Konačno, ako se izračunaju vrijednosti za 99%-tni interval povjerenja, dobija se interval povjerenja (5.3, 10.6).

Neophodno je razriješiti konfuziju koja može da nastane kao posljedica nerazumijevanja šta zapravo pokazuje interval povjerenja. Iz prethodnog primjera je jasno da sa povećavanjem nivoa povjerenja interval povjerenja postaje širi! Širi interval na prvi pogled implicira da se sada posjeduje manje znanja o pravoj srednjoj vrijednosti nego što se posjedovalo sa manjim nivoom povjerenja. Sa povećavanjem nivoa povjerenja, preciznost se očigledno smanjuje.

Kontradikcija dolazi kao posljedica nerazumijevanja šta zapravo pokazuje interval povjerenja. 90%-tni interval povjerenja, na primjer, znači da postoji 90%-tna vjerovatnoća da se prava srednja vrijednost nalazi unutar dobijenog intervala. Povećavanje nivoa na 95% znači da se povećava vjerovatnoća da se prava srednja vrijednost zapravo nalazi u dobijenom intervalu. Pošto se vrijednosti mjerenja na osnovu kojih se došlo do intervala nisu mijenjale, iste informacije ne mogu da doprinesu većoj preciznosti. Na osnovu toga, jedini način da se poveća vjerovatnoća da se prava srednja vrijednost nalazi u novom intervalu je da se on proširi. Interval se dalje mora proširivati ako se želi da se sa vjerovatnoćom od 99% prava srednja vrijednost nalazi u dobijenom intervalu. Drugim riječima, interval se mora proširiti ako postoji želja da se osigura da sa 1%-tnom vjerovatnoćom može reći da prava srednja vrijednost nije u intervalu.

Primjer. Pravi se eksperiment u kojem se nasumično generiše broj između 0 i 1000 trideset puta i pravi se interval povjerenja sa nivoom povjerenja od 90% za dobijenu srednju vrijednost. Ako se eksperiment ponavlja 2000 puta, koliko će puta interval povjerenja sadržavati pravu srednju vrijednost (500)? Koliko intervala povjerenja će obuhvatati pravu srednju vrijednost ako smanjimo povjerenje na 60%?

Interval povjerenja će sadržavati pravu srednju vrijednost 1800 puta. Očigledno je da interval povjerenja zapravo pokazuje u kolikom će procentu od ukupnog broja izvođenja eksperimenta prava srednja vrijednost biti u intervalu povjerenja. U slučaju smanjivanja nivoa povjerenja na 60%, suziće se i interval povjerenja i prava srednja vrijednost će se u ovom slučaju naći 1200 puta u dobijenom intervalu povjerenja.

Određivanje broja potrebnih mjerenja

Iz formule za izračunavanje intervala povjerenja može se vidjeti da je širina intervala inverzno proporcionalna kvadratnom korjenu broja izvršenih mjerenja. Pošto se obično želi redukovati broj potrebnih mjerenja, navedena formula se može iskoristiti za utvrđivanje koliko mjerenja je potrebno da bi se dobio interval povjerenja specifične širine.

Pretpostavka je da se želi utvrditi interval povjerenja za $\bar{x}$ tako da postoji vjerovatnoća od $1 - \alpha$ da se prava srednja vrijednost x nalazi unutar intervala $(c_1, c_2) = ((1 - e)\bar{x}, (1 + e)\bar{x})$. Tada se dobija da je:

$$c_1 = (1 - e)\bar{x} = \bar{x} - z_{1-\alpha/2} \frac{s}{\sqrt{n}}$$

$$c_2 = (1 + e)\bar{x} = \bar{x} + z_{1-\alpha/2} \frac{s}{\sqrt{n}}.$$

Pošto c_1 i c_2 formiraju simetričan interval, može se iskoristiti bilo koja od gornjih jednačina da se dobije da je:

$$z_{1-\alpha/2} \frac{s}{\sqrt{n}} = \bar{x}e.$$

Ako se iz gornje jednačine izrazi n , dobija se:

$$n = \left(\frac{z_{1-\alpha/2} s}{e\bar{x}} \right)^2.$$

Primjetno je da je preduslov za korištenje dobijene jednačine poznavanje standardne devijacije, s . Međutim, da bi se s odredilo mora se prvo izvršiti nekoliko mjerenja. Na osnovu toga, procedura je da se prvo izvrši nekoliko mjerenja radi određivanja približne vrijednosti s . Zatim se dobijeni rezultat koristi u gornjoj jednačini.

Primjer. U primjeru sa upisivanjem fajla na disk, nađeno je da je prosječno vrijeme upisivanja 7.94s sa standardnom devijacijom od 2.14s. Potrebno je odrediti koliko mjerenja je potrebno ako je potrebna uvjerenost od 90% da je prava srednja vrijednost unutar 7% dobijene srednje vrijednosti.

Za 90%-tni interval povjerenja, poznato je da je $\alpha = 0.10$, pa je $1 - \alpha/2 = 0.95$. Dodatno, ako prava srednja vrijednost treba da bude unutar 7% dobijene, slijedi da je

dozvoljena greška od $\pm 3.5\%$. Iz toga slijedi da je $e = 0.035$. Na osnovu jednačine za broj potrebnih mjerenja dobija se:

$$n = \left(\frac{z_{1-\alpha/2} s}{e\bar{x}} \right)^2 = \left(\frac{1.895(2.14)}{0.035(7.94)} \right)^2 = 212.95.$$

Iz dobijenog rezultata zaključuje se da je potrebno približno 213 mjerenja da bi postojala 90%-tna sigurnost da je prava vrijednost u $\pm 3.5\%$ -tnom intervalu. Primijetiti da je u zadnjem računanju umjesto vrijednosti z raspodjele uzeta vrijednost t raspodjele sa $n-1$ stepeni slobode pošto su se koristili aritmetička sredina i devijacija koji su dobijeni na osnovu samo 8 mjerenja.

Intervali povjerenja za proporcije

Često se u računarskim sistemima koriste vrijednosti koje daju informaciju o procentu od ukupnog vremena u kojem se dešavao neki događaj. U ovom slučaju je neophodno utvrditi interval povjerenja za proporciju p , gdje p predstavlja odnos broja koliko se puta desio željeni događaj (m) i ukupnog broja zabilježenih događaja (n):

$$p = m / n.$$

Binomna raspodjela sa parametrima p i n ima srednju vrijednost pn i varijansu $p(1 - p)n$. Ako je $np \geq 10$, binomna raspodjela se može aproksimirati Gausovom sa srednjom vrijednošću $\bar{p}n$ i varijansom $\bar{p}(1 - \bar{p})n$. Sličnim izvođenjem kao u jednoj od prethodnih sekcija dobija se da je:

$$c_1 = \bar{p} - z_{1-\alpha/2} \sqrt{\frac{\bar{p}(1 - \bar{p})}{n}}$$

$$c_2 = \bar{p} + z_{1-\alpha/2} \sqrt{\frac{\bar{p}(1 - \bar{p})}{n}}$$

Primjer. Računarski sistem izvršava nekoliko korisničkih programa i instrukcije operativnog sistema. Treba da se odredi koliko vremena procesor provede izvršavajući instrukcije operativnog sistema u odnosu na vrijeme koje provede izvršavajući korisničke programe. Da bi se izvršilo mjerenje, sistem se konfiguriše da šalje prekid procesoru svakih 10 ms. Rutina za servisiranje prekida održava dva brojača. Jedan od njih, n , inkrementuje se svaki put kada se izvrši prekid. Rutina svaki put kada se prekid desi na osnovu adrese na steku određuje da li se izvršavao operativni sistem kada se desio prekid. Ako je odgovor potvrđan, inkrementuje se drugi brojač, m . Nakon izvršavanja opisanog eksperimenta u intervalu od 1 minute dobijene su vrijednosti $m = 658$ i $n = 6000$.

Na osnovu dobijenih podataka jednostavno je odrediti da je $\bar{p} = m / n = 0.1097$. Ako se za ovu vrijednost primjeni 95%-tni interval povjerenja, dobija se:

$$z = z_{1-\alpha/2} \sqrt{\frac{\bar{p}(1-\bar{p})}{n}} = 1.96 \sqrt{\frac{0.1097(1-0.1097)}{6000}} = 0.0079.$$

Slijedi da je interval povjerenja (0.1018, 0.1176). Znači, sa 5%-tnom vjerovatnoćom da se griješi, može se reći da procesor provede 10.2 – 11.8% svog vremena izvršavajući instrukcije operativnog sistema.

Određivanje broja potrebnih mjerenja

Iz istih razloga kao i kod prošlog određivanja broja potrebnih mjerenja, taj broj se može odrediti i u ovom slučaju:

$$(1 - e)\bar{p} = \bar{p} - z_{1-\alpha/2} \sqrt{\frac{\bar{p}(1-\bar{p})}{n}}.$$

Rearanžiranjem prethodne jednakosti dobija se:

$$e\bar{p} = z_{1-\alpha/2} \sqrt{\frac{\bar{p}(1-\bar{p})}{n}}.$$

Rješavanjem po n dobija se:

$$n = \frac{z_{1-\alpha/2}^2 \bar{p}(1-\bar{p})}{(e\bar{p})^2}.$$

Kao i ranije, i ovdje je neophodno izvršiti nekoliko mjerenja na početku radi procjene vrijednosti $\bar{p}$.

Primjer. Koliko dugo treba izvoditi prethodni eksperiment sa operativnim sistemom, da bi se sa 95%-tnom vjerovatnoćom moglo reći da procesor izvršava instrukcije operativnog sistema u intervalu $\pm 0.5\%$?

Interval od $\pm 0.5\%$ daje da je $e = 0.005$. Ako se iskoristi dobijena vrijednost $\bar{p} = 0.1097$, korištenjem izvedene jednačine dobija se da je:

$$n = \frac{z_{1-\alpha/2}^2 \bar{p}(1-\bar{p})}{(e\bar{p})^2} = \frac{(1.960)^2 (0.1097)(1-0.1097)}{[0.005(0.1097)]^2} = 1,247,102.$$

Pošto se prekid dešava svakih 10 ms, za 1,247,102 uzoraka bi bilo potrebno otprilike vrijeme izvršavanja od 3.46 h.

5.3. Poređenje alternativa

Ultimativna želja je iskorištavanje dobijenih rezultata mjerenja u svrhu poređenja komponenti dva različita sistema ili u svrhu utvrđivanja koliko je neka promjena na jednom sistemu unaprijedila sistem. Pošto šum postoji u svakom mjerenju koje se izvrši,

potrebna je tehnika za utvrđivanje da li razlika između poređenih alternativa postoji usljed slučajnih grešaka u mjerenjima ili su te razlike značajne u statističkom smislu. Navedene tehnike se zasnivaju na intervalima povjerenja.

5.3.1. Poređenje dvije alternative

Najjednostavniji pristup u korištenju intervala povjerenja je utvrđivanje da li se intervali povjerenja za dva skupa mjerenja koji se porede preklapaju. Ako se poklapaju, nemoguće je tvrditi da bilo koja razlika između skupova nije zbog slučajnih grešaka. Međutim, ako se ne preklapaju, može se zaključiti da nema dokaza koji pokazuje da ne postoji statistički značajna razlika. Naročito je zanimljiva zadnja konstrukcija. Kada se intervali povjerenja ne preklapaju, ne može se tvrditi sa potpunom sigurnošću da postoji prava razlika između alternativa. Može se jedino tvrditi da ne postoji razlog zbog kojeg bi se vjerovalo da ne postoji razlika. Uvijek postoji vjerovatnoća α da razlike postoje zbog slučajnih grešaka. Iako ovaj dvosmisleni zaključak nije baš uvijek zadovoljavajući, na nesreću je nešto najbolje do čega se može doći u ovom slučaju.

U ovoj sekciji se razmatraju tri različita slučaja – *before-and-after* analiza, poređenje nekorespondirajućih ili neuparenih mjerenja i poređenje proporcija.

Before-and-after analiza

Before-and-after poređenja se uglavnom koriste za utvrđivanje da li neka promjena na sistemu ima statistički značajan učinak na njegove performanse (na primjer, da li se isplati nadogradnja operativnog sistema na najnoviju verziju). Takođe, može se koristiti za poređenje brzina dva različita sistema.

U svim ovim tipovima situacija, *before-and-after* mjerenja nisu nezavisna i varijanse dva skupa mjerenja nisu uvijek jednake. U stvari, mjerenje iz svakog od dva skupa je povezano sa odgovarajućim mjerenjem iz drugog skupa i s njim čini korespondirajući par. Da bi se utvrdilo da li postoji statistički značajna razlika između dva skupa mjerenja mora se pronaći interval povjerenja za srednju vrijednost razlika posmatranih parova. Ako ovaj interval uključuje nulu, može se zaključiti da izmjerene razlike nisu statistički značajne.

Neka $b_1, b_2, \dots, b_n$ predstavlja skup mjerenja prije (eng. *before*) promjene, a $a_1, a_2, \dots, a_n$ skup mjerenja poslije (eng. *after*) promjene. Onda je jedino potrebno naći interval povjerenja za srednju vrijednost n razlika (eng. *differences*), $d_1 = a_1 - b_1, d_2 = a_2 - b_2, \dots, d_n = a_n - b_n$. Korištenjem ranije izvedenih relacija pod pretpostavkom da je $n \geq 30$, dobija se da je:

$$c_1 = \bar{d} - z_{1-\alpha/2} \frac{s_d}{\sqrt{n}}$$

$$c_2 = \bar{d} + z_{1-\alpha/2} \frac{s_d}{\sqrt{n}}$$

U gornjim formulama $\bar{d}$ predstavlja aritmetičku sredinu vrijednosti d_i , a s_d odgovarajuću standardnu devijaciju. Ako je broj mjerenja manji od 30, $z_{1-\alpha/2}$ vrijednosti treba da se zamijene sa odgovarajućim t vrijednostima sa $n - 1$ stepeni slobode.

Primjer. Zadatak je poređenje novog poboljšanog mrežnog protokola u odnosu na postojeći protokol. Poređenje se vrši na osnovu mjerenja potrebnog za slanje 6 poruka različitih veličina, prvo preko originalnog protokola, a zatim preko novog. Pošto se iste poruke šalju preko oba protokola, očigledno postoji korespondencija između parova mjerenja. Ova mjerenja zajedno sa razlikama između parova su prikazani u tabeli 5.7.

Tabela 5.7 – Before-and-after analiza

Redni broj mjerenja (i)	Originalni protokol (b_i)	Novi protokol (a_i)	Razlika ($d_i = b_i - a_i$)
1	85	86	-1
2	83	88	-5
3	94	90	4
4	90	95	-5
5	88	91	-3
6	87	83	4

Nakon izračunavanja srednje vrijednosti razlika (zadnja kolona) nalazi se da je $\bar{d} = -1$. Standardna devijacija za skup mjerenja je $s_d = 4.15$. Pošto je izvršeno manje od 30 mjerenja, mora se koristiti t raspodjela. Za nivo povjerenja od 95% iz tabele t raspodjele nalazi se da je $t_{1-\alpha/2; n-1} = t_{0.975; 5} = 2.571$. Na osnovu svih tih vrijednosti, pomoću formule za izračunavanje intervala povjerenja dobija se da je interval za srednju vrijednost razlika jednak $(-5.36, 3.36)$.

Na osnovu dobijene vrijednosti za d od -1 na prvi pogled može se zaključiti da je novi protokol nešto slabiji od originalnog. Međutim, pošto interval povjerenja uključuje nulu, može se reći sa nivoom povjerenja od 95% da je izmjerena razlika posljedica slučajnih grešaka u mjerenju. Znači da se može reći sa 95%-tnom vjerovatnoćom da ne postoji razlika između dva komunikaciona protokola.

Nekorespondirajuća mjerenja

U mnogim situacijama ne postoji direktna korespondencija između parova mjerenja. Taj broj mjerenja čak ne mora da bude ni jednak na različitim sistemima. U ovom slučaju mjerenja se nazivaju nekorespondirajuća ili neuparena. Kao i ranije, moraju se razmotriti dvije moguće situacije, zavisno da li je broj mjerenja manji od 30 ili ne.

Pošto se više ne može tražiti srednja vrijednost razlika između parova mjerenja, traže se srednje vrijednosti, $\bar{x}_1$ i $\bar{x}_2$, i standardne devijacije, s_1 i s_2 , za svaki skup mjerenja ponaosob. Onda se izračuna razlika srednjih vrijednosti kao $\bar{x} = \bar{x}_1 - \bar{x}_2$. Može se pokazati da se standardna devijacija ove razlike može izračunati kao:

$$s_x = \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}$$

Sada se može primjeniti formula za izračunavanje granica intervala povjerenja. Njenom primjenom se dobija:

$$c_1 = \bar{x} - z_{1-\alpha/2} s_x$$

$$c_2 = \bar{x} + z_{1-\alpha/2} s_x$$

Ako interval povjerenja uključuje nulu, može se zaključiti da nema značajne razlike između dva skupa mjerenja.

Gornje relacije važe ako je broj mjerenja u oba skupa veći ili jednak od 30. U suprotnom slučaju $z_{1-\alpha/2}$ vrijednosti treba da se zamijene sa odgovarajućim t vrijednostima. Međutim, u tom slučaju broj stepena slobode nije jednostavno $n_1 + n_2 - 2$, kao što se možda moglo očekivati. Umjesto toga, broj stepena slobode se u ovom slučaju dobija po sljedećoj formuli:

$$n_{df} = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{(n_1 - 1)} + \frac{(s_2^2/n_2)^2}{(n_2 - 1)}}$$

Ova vrijednost vjerovatno neće biti cijeli broj pa je treba zaokružiti na najbliži takav.

Primjer. Porede se performanse dva računarska sistema korištenjem standardnih benčmark programa. Izvršeno je $n_1 = 8$ mjerenja na prvom sistemu i $n_2 = 5$ mjerenja na drugom sistemu. Mjerenja su prikazana u tabeli 5.8, kao i srednja vrijednost i standardna devijacija za oba skupa mjerenja.

Tabela 5.8– Nekorespondirajuća mjerenja

Redni broj mjerenja	Vremena	
	Sistem 1	Sistem 2
1	1,011	894
2	998	963
3	1,113	1,098
4	1,008	982
5	1,100	1,046
6	1,039	
7	1,003	
8	1,098	
n_i	8	5
$\bar{x}_i$	1,046.25	996.6
s_i	49.25	78.41

Na prvi pogled se čini da je sistem 2 nešto brži od sistema 1. Međutim standardna devijacija mjerenja je relativno velika u odnosu na srednje vrijednosti. Dalje, standardna devijacija sistema 2 je veća od standardne devijacije sistema 1. Kao posljedica, moguće je da je razlika između srednjih vrijednosti zbog grešaka u mjerenju, a ne zbog pravih razlika između sistema. Da bi se karakterizovala razlika, odlučeno je da se konstruiše interval povjerenja sa nivoom povjerenja od 90% za razliku srednjih vrijednosti vremena izvršavanja dva sistema.

Prvo je potrebno izračunati broj stepeni slobode, n_{df} , za ovu razliku korištenjem odgovarajuće jednačine:

$$n_{df} = \frac{\left(\frac{49.25^2}{8} + \frac{78.41^2}{5} \right)^2}{\frac{(49.25^2/8)^2}{(8-1)} + \frac{(78.41^2/5)^2}{(5-1)}} = 6.01$$

Ova vrijednost se zaokružuje na 6. Nakon toga se nalazi odgovarajuća vrijednost iz t raspodjele, pa je $t_{0.95;6} = 1.943$. Razlika srednjih vrijednosti se dobija kao:

$$\bar{x} = 1046.25 - 996.60 = 49.65.$$

Odgovarajuća standardna devijacija se dobija kao:

$$s_x = \sqrt{\frac{49.25^2}{8} + \frac{78.41^2}{5}} = 39.15.$$

Na osnovu svih navedenih vrijednosti može se izračunati da su granice intervala povjerenja:

$$c_1 = 49.65 - 1.943(39.15) = -26.42$$

$$c_2 = 49.65 + 1.943(39.15) = 125.7.$$

Pošto izračunati interval povjerenja uključuje nulu, može se zaključiti da ne postoji statistički značajna razlika između dva sistema sa nivoom povjerenja od 90%.

Poređenje proporcija

Prilikom utvrđivanja da li je razlika između dvije proporcije statistički značajna, eksploatiše se činjenica da mjerenja proporcija prate binomnu raspodjelu. Neka m_1 bude broj koji pokazuje koliko puta se desio odgovarajući događaj u sistemu 1, a n_1 ukupan broj mjerenja. Iz toga slijedi da je $p_1 = m_1 / n_1$. Slično se dobija i za sistem 2, $p_2 = m_2 / n_2$. Na osnovu karakteristika binomne raspodjele, dobija se da je srednja vrijednost sistema i $p_i n$, a varijansa $p_i(1 - p_i)n_i$.

Ako su m_1 i m_2 veći od 10, raspodjela ovih proporcija se može aproksimirati na Gausovu sa srednjom vrijednošću $p_i n_i$ i varijansom $p_i(1 - p_i)n_i$. Ova aproksimacija dozvoljava korištenje istog pristupa za izvođenje intervala povjerenja za razliku srednjih vrijednosti p_1 i p_2 kao kod nekorespondirajućih mjerenja. Dobija se da je:

$$c_1 = p - z_{1-\alpha/2} s_p$$

$$c_2 = p + z_{1-\alpha/2} s_p$$

U gornjim jednačinama je $p = p_1 - p_2$, a s_p predstavlja standardnu devijaciju navedene razlike. Formula za računanje s_p i u ovom slučaju predstavlja sumu pojedinačnih devijacija normalizovanih sa brojem mjerenja:

$$s_p = \sqrt{\frac{p_1(1-p_1)}{n_1} + \frac{p_2(1-p_2)}{n_2}}.$$

Primjer. U primjeru navedenom kod izračunavanja intervala povjerenja za proporcije na osnovu tehnike uzorkovanja se izračunao procenat vremena koje procesor izvede izvršavajući instrukcije operativnog sistema. Konkretno, pretpostavka je bila da se prekid vrši svakih 10 ms. Održavala su se dva brojača, jedan koji je brojao ukupan broj prekida i drugi, koji je brojao koliko su se puta izvršavale instrukcije operativnog sistema kada se desio prekid. Nakon što je test izvršavan 3.5h, dobilo se da je ukupan broj prekida $n_1 = 1,300,203$, a broj prekida u kojima se izvršavao operativni sistem je bio $m_1 = 142,892$. Odnos ove dvije vrijednosti, $p_1 = m_1 / n_1 = 0.1099$ je procjena ukupnog dijela vremena koje procesor provede izvršavajući operativni sistem.

Sada je izvršen *upgrade* operativnog sistema. Korištenjem tehnike poređenja proporcija treba odrediti da li je *upgrade*-ovani operativni sistem efikasniji od starog. Nakon izvršavanja istog testa na novom sistemu 2h i 45 minuta dobilo se da je $m_2 = 84,876$ i $n_2 = 999,382$. Na osnovu toga, dobija se da je $p_2 = m_2 / n_2 = 0.0849$. Slijedi da novi sistem provodi približno 8.5% vremena izvršavajući operativni sistem.

Razlika dvije proporcije je $p = p_1 - p_2 = 0.0250$, a standardna devijacija je:

$$s_p = \sqrt{\frac{0.1099(1-0.1099)}{1300203} + \frac{0.0849(1-0.0849)}{999382}} = 3.911 \times 10^{-4}.$$

Za granice 95%-tnog intervala povjerenja dobija se da je:

$$c_1 = 0.0250 - 1.960(3.911 \times 10^{-4}) = 0.0242$$

$$c_2 = 0.0250 + 1.960(3.911 \times 10^{-4}) = 0.0257.$$

Pošto interval ne uključuje nulu, može se zaključiti sa 90%-tnim povjerenjem da ne postoji dokaz koji govori da ne postoji statistički značajna razlika između performansi dva

operativna sistema. Treba obratiti pažnju na prethodnu konstrukciju, a pogotovo kada interval ne uključuje nulu. Kao što je pomenuto ranije u glavi, ne može se tvrditi da postoji prava razlika između alternativa. Zato je bolje reći da nema razloga da se vjeruje u suprotno.

5.3.2. Poređenje više od dvije alternative

Načini poređenja opisani u sekciji 5.3.1. su jednostavni za razumijevanje, ali se među statističarima smatraju relativno slabim metodama poređenja. Robusniji pristup predstavlja generalna tehnika pod nazivom analiza varijanse (eng. *analysis of variance*, ANOVA). ANOVA dijeli ukupnu varijaciju između skupova mjerenja koji se porede u komponentu nastalu usljed slučajnih grešaka i komponentu nastalu usljed pravih razlika između alternativa. Korištenje tehnike ANOVA u svrhu poređenja više različitih alternativa se naziva jednosmjerna klasifikacija ili analiza jednofaktorskih eksperimenata.

ANOVA

ANOVA predstavlja generalnu tehniku za dijeljenje ukupne varijacije između skupova mjerenja u smislene komponente. U ovoj analizi se pretpostavlja da su greške u mjerenjima različitih alternativa nezavisne i da prate Gausovu raspodjelu. Dalje se pretpostavlja da su varijanse grešaka mjerenja iste u svim alternativama. ANOVA razdvaja ukupnu varijaciju u dvije komponente:

1. varijaciju koja se posmatra unutar svakog pojedinačnog sistema, za koju se pretpostavlja da je izazvana isključivo greškama u mjerenjima,
2. varijaciju između različitih sistema.

Cilj je utvrditi da li je magnituda druge komponente značajno veća od magnitude prve komponente u statističkom smislu. Drugim riječima, cilj je utvrditi da li razlika između alternativa potiče od pravih razlika između njih ili zbog grešaka u mjerenju.

Da bi se odgovorilo na postavljeno pitanje, pravi se n mjerenja svake od k alternativa. Pogodno je organizovati svih kn mjerenja na način kao u tabeli 5.9, gdje y_{ij} predstavlja i -to mjerenje izvršeno na j -toj alternativu. Srednje vrijednosti kolona u tabeli $(\bar{y}_{.1}, \bar{y}_{.2}, \dots, \bar{y}_{.j}, \dots, \bar{y}_{.k})$ predstavljaju prosječne vrijednosti svih mjerenja unutar jedne alternative. To se može zapisati kao:

$$\bar{y}_j = \frac{\sum_{i=1}^n y_{ij}}{n}.$$

Tabela 5.9 – ANOVA

Mjerenja	Alternative						Ukupna srednja vrijednost
	1	2	...	j	...	k	
1	y_{11}	y_{12}	...	y_{1j}	...	y_{1k}	
2	y_{21}	y_{22}	...	y_{2j}	...	y_{2k}	
...	...	...	...	...	...	...	
i	y_{i1}	y_{i2}	...	y_{ij}	...	y_{ik}	
...	...	...	...	...	...	...	
n	y_{n1}	y_{n2}	...	y_{nj}	...	y_{nk}	
Sred. vrij. kolona	$\bar{y}_{.1}$	$\bar{y}_{.2}$	...	$\bar{y}_j$	...	$\bar{y}_k$	$\bar{y}_{..}$
Efekti	α_1	α_2	...	α_j	...	α_k	

Ukupna srednja vrijednost, $\bar{y}_{..}$, predstavlja srednju vrijednost svih mjerenja izvršenih na svim alternativama:

$$\bar{y}_{..} = \frac{\sum_{j=1}^k \sum_{i=1}^n y_{ij}}{kn}.$$

Sada je korisno zapisati svako mjerenje, y_{ij} , kao sumu srednje vrijednosti svih mjerenja izvršenih na alternativu j , $\bar{y}_{.j}$, i vrijednosti e_{ij} , koja predstavlja razliku tog mjerenja od srednje vrijednosti kolone:

$$y_{ij} = \bar{y}_{.j} + e_{ij}.$$

Dalje, ova ideja se može proširiti da bi se svaka srednja vrijednost kolone reprezentovala kao suma ukupne srednje vrijednosti, $\bar{y}_{..}$, i razlike srednje vrijednosti kolone i ukupne srednje vrijednosti. Ako se ta razlika označi sa α_j , dobija se:

$$\bar{y}_{.j} = \bar{y}_{..} + \alpha_j.$$

Ako se prošla jednačina uvrsti u prethodnu dobija se:

$$y_{ij} = \bar{y}_{..} + \alpha_j + e_{ij}.$$

Intuitivno, ovaj izraz se može shvatiti kao pokazatelj koliko je svako mjerenje udaljeno od ukupne srednje vrijednosti svih mjerenja. Ta udaljenost se ogleda u horizontalnom smislu preko ostalih alternativa korištenjem vrijednosti α_j , i u vertikalnom smislu preko mjerenja iste alternative korištenjem vrijednosti e_{ij} .

Vrijednosti α_j se nazivaju efektima odgovarajuće alternative. Ove vrijednosti moraju da ispunjavaju sljedeću osobinu:

$$\sum_{j=1}^k \alpha_j = 0.$$

Izražavanje individualnih mjerenja na ovaj način omogućava da se ukupna varijacija svih mjerenja podijeli u dvije odvojene komponente: varijaciju usljed efekata alternativa i varijaciju usljed grešaka u mjerenju.

Varijacija usljed efekata alternativa, koju ne treba miješati sa varijansom, definiše se kao suma kvadrata razlika između srednjih vrijednosti mjerenja svake alternative (srednjih vrijednosti kolona) i ukupne srednje vrijednosti svih mjerenja. Ova vrijednost, nazvana SSA, računa se po sljedećoj formuli:

$$SSA = n \sum_{j=1}^k (\bar{y}_j - \bar{y}_{..})^2.$$

Slično prethodnom, varijacija usljed grešaka u mjerenju, označena SSE, računa se kao suma kvadrata razlika između individualnih mjerenja i njihovih odgovarajućih srednjih vrijednosti kolona:

$$SSE = \sum_{j=1}^k \sum_{i=1}^n (y_{ij} - \bar{y}_j)^2.$$

Konačno, suma kvadrata razlika između svakog individualnog mjerenja i ukupne srednje vrijednosti svih mjerenja, označena kao SST, računa se kao:

$$SST = \sum_{j=1}^k \sum_{i=1}^n (y_{ij} - \bar{y}_{..})^2.$$

Može se pokazati da je:

$$SST = SSA + SSE.$$

Ovaj izraz pokazuje da se ukupna varijacija između svih izmjerenih vrijednosti može podijeliti u dvije komponente – SSA i SSE. Ako su razlike između alternativa posljedica pravih razlika, a ne grešaka u mjerenjima, realno je očekivati da SSA bude veće u statističkom smislu od SSE. Da bi se odredio odnos ove dvije veličine, potrebna je neka vrsta statističkog testa.

Statistički test koji je pogodan za primjenu u ovom slučaju naziva se F-test. Ovaj test je baziran na F raspodjeli i bazira se na poređenju varijansi. Pošto se F-test zasniva na odnosu dvije varijanse, izračunate vrijednosti bliske jedinici će pokazivati da vjerovatno ne postoji značajna razlika.

Približne varijanse veličina SSA i SSE se dobija dijeljenjem tih veličina sa brojem stepena slobode odgovarajuće veličine. Pošto se poredi k alternativa, broj stepeni slobode za SSA je k – 1. Na osnovu ovoga, estimat varijanse veličine SSA je:

$$s_a^2 = \frac{SSA}{k - 1}.$$

Slično tome, estimat varijanse veličine SSE se dobija dijeljenjem vrijednosti SSE sa brojem stepeni slobode ove komponente. Pošto se na svakoj alternativni vrši po n mjerenja, svaka alternativa ima $n - 1$ stepeni slobode. Na osnovu toga, ukupan broj stepeni slobode za SSE je $k(n - 1)$, pošto postoji k alternativa. Estimat varijanse je na kraju:

$$s_e^2 = \frac{SSE}{k(n - 1)}.$$

Nije loše napomenuti da je ukupan broj stepeni slobode veličine SST $kn - 1$ pošto je ukupno izvršeno kn mjerenja. Dalje, ako se sabere broj stepeni slobode SSA i broj stepeni slobode SSE, dobije se baš $kn - 1$. Slijedi da je:

$$df(SST) = df(SSA) + df(SSE)$$

U gornjoj relaciji $df(.)$ predstavlja skraćenicu za broj stepeni slobodi (eng. *degrees of freedom*).

Konačno, F-test se računa kao:

$$F = \frac{s_a^2}{s_e^2}.$$

Ako je izračunata vrijednost veća od vrijednosti dobijene iz tabele koja se nalazi u prilogu B, onda se može reći da je varijacija usljed pravih razlika između alternativa, SSA, znatno veća od varijacije usljed grešaka, SSE za nivo povjerenja od $1 - \alpha$.

Radi lakšeg računanja³, korisno je napisati gornje izraze na sljedeći način:

$$SST = \sum_{j=1}^k \sum_{i=1}^n y_{ij}^2 - \frac{y_{..}^2}{kn}$$

$$SSA = \frac{\sum_{j=1}^k (\sum_{i=1}^n y_{ij})^2}{n} - \frac{y_{..}^2}{kn}$$

$$SSE = SST - SSA$$

gdje je:

$$y_{..} = \sum_{j=1}^k \sum_{i=1}^n y_{ij}.$$

Komponente ANOVA testa su sumirane u tabeli 5.10.

³ Postoji i veliki broj *online* ANOVA kalkulatora

Tabela 5.10 – Sumirani parametri tehnike ANOVA

Izvor varijacije	Alternativa	Greška	Ukupno
Zbir kvadrata	SSA	SSE	SST
Broj stepeni slobode	$k - 1$	$k(n - 1)$	$kn - 1$
Varijansa	$s_a^2 = SSA/(k - 1)$	$s_e^2 = SSE/[k(n - 1)]$	
Izračunata F vrijednost	s_a^2/s_e^2		
F vrijednost iz tabele	$F_{[1-\alpha; (k-1), k(n-1)]}$		

Primjer. U svrhu poređenja $k = 3$ različita računarska sistema, mjerena su vremena potrebna da bi svaki sistem izvršio *call* i *return* instrukcije. Koristi se ANOVA za poređenje sistema, pa je $n = 5$ mjerenja svakog sistema organizovano na način prikazan u tabeli 5.11.

Tabela 5.11 – ANOVA primjer

Mjerenja	Alternative			Ukupna srednja vrijednost
	1	2	3	
1	0.0972	0.1382	0.7966	
2	0.0971	0.1432	0.5300	
3	0.0969	0.1382	0.5152	
4	0.1954	0.1730	0.6675	
5	0.0974	0.1383	0.5298	
Srednje vrijednosti kolona	0.1168	0.1462	0.6078	0.2903
Efeki	-0.1735	-0.1441	0.3175	

Prvi korak u analizi je da se izračunaju sume kvadrata:

$$SST = [(0.0972^2 + 0.0971^2 + \dots + 0.5298^2) - \frac{(0.0972 + 0.0971 + \dots + 0.5298)^2}{3(5)}]$$

$$= 0.8270$$

$$SSA = [(0.0972 + \dots + 0.0974)^2 + (0.1382 + \dots + 0.1383)^2 + (0.7966 + \dots + 0.5298)^2]/5 - \frac{(0.0972 + 0.0971 + \dots + 0.5298)^2}{3(5)}$$

$$= 0.7585$$

$$SSE = SST - SSA = 0.8270 - 0.7585 = 0.0685.$$

Nakon toga se računaju dijelovi ukupne varijacije objašnjeni od strane svake od komponenata:

$$\frac{SSA}{SST} = \frac{0.7585}{0.8270} = 0.917$$

$$\frac{SSE}{SST} = \frac{0.0685}{0.8270} = 0.083.$$

Na osnovu dobijenih vrijednosti može se reći da je 91.7% ukupne varijacije zbog razlika između sistema, dok je 8.3% ukupne varijacije zbog grešaka u mjerenju.

Iako ovakva raspodjela varijacije sugerise da postoji znatna razlika između sistema, sljedeći korak je da se primjeni F-test radi utvrđivanja da li je ova razlika statistički značajna. Rezultati ovog testa su prikazani u tabeli 5.12. Pošto je izračunata F vrijednost veća od F vrijednosti dobijene iz tabele može se zaključiti da sa nivoom povjerenja od 95% postoji statistički značajna razlika između sistema.

Tabela 5.12 – ANOVA primjer (2)

Izvor varijacije	Alternative	Greška	Ukupno
Suma kvadrata	$SSA = 0.7585$	$SSE = 0.0685$	$SST = 0.8270$
Broj stepeni slobode	$k - 1 = 2$	$k(n - 1) = 12$	$kn - 1 = 14$
Varijanse	$s_a^2 = 0.7585/2 = 0.3793$	$s_e^2 = 0.0685/12 = 0.0057$	
Izračunata F vrijed	$0.3793/0.0057 = 66.4$		
Tabelarna F vrijed	$F_{10.95, 2, 12} = 3.89$		

Kontrasti

Upotrebom F-testa može se otkriti da li postoji statistički značajna razlika između alternativa. Međutim, ovaj test ne pokazuje gdje su te razlike ispoljene.

Radi detaljnijeg poređenja između pojedinih alternativa koristi se metoda kontrasta. Kontrast, c , predstavlja linearnu kombinaciju efekata alternativa, α_j :

$$c = \sum_{j=1}^k w_j \alpha_j.$$

Težinski faktori u gornjoj jednačini moraju biti izabrani tako da je:

$$\sum_{j=1}^k w_j = 0.$$

Kontrasti se mogu iskoristiti za poređenje efekata bilo kojeg podskupa alternativa. Na primjer, ako postoje $k = 3$ alternative u eksperimentu, biranjem faktora tako da je $w_1 = 1$, $w_2 = -1$, $w_3 = 0$ dobija se: $c = (1)\alpha_1 + (-1)\alpha_2 + (0)\alpha_3 = \alpha_1 - \alpha_2$. Iako je primjetiti da je i zbir faktora jednak 0. Konstruisanje odgovarajućeg intervala povjerenja za dobijeni kontrast će da odgovori na pitanje da li je efekat alternative 1 statistički različit od efekta alternative 2.

Težinski faktori se mogu izabrati na bilo koji način, sve dok je zadovoljen uslov da njihov zbir mora biti jednak 0. Međutim, najčešće se faktori biraju na način da se na 1 i -1 postave faktori alternativa koje se porede, a ostali faktori se postavljaju na 0.

Formiranje intervala povjerenja se obavlja na identičan način kao kod procedure koja je ranije pokazana. Potrebna varijansa za računanje intervala povjerenja dobija se korištenjem sljedeće formule:

$$s_e^2 = \frac{\sum_{j=1}^k (w_j^2 s_e^2)}{kn}$$

U gornjem izrazu vrijednost s_e^2 predstavlja srednjekvadratnu grešku i računa se pomoću formule predstavljene u sekciji posvećenoj tehnici ANOVA.

Interval povjerenja (c_1 , c_2) za kontrast c dobija se korištenjem sljedećih formula:

$$c_1 = c - t_{[1-\alpha/2; k(n-1)]} s_e$$

$$c_2 = c + t_{[1-\alpha/2; k(n-1)]} s_e$$

Kao i u prethodnim situacijama, vrijednosti t raspodjele se mogu zamijeniti sa z vrijednostima kada je broj stepeni slobode, $k(n-1)$, veći od 30.

Primjer. Na osnovu podataka iz tabele 5.11, vidi se da su efekti tri sistema $\alpha_1 = -0.1735$, $\alpha_2 = -0.1441$, $\alpha_3 = 0.3175$. Ako se težinski faktori izaberu tako da je $w_1 = 1$, $w_2 = -1$ i $w_3 = 0$, dobija se kontrast za razliku između efekata sistema 1 i 2, gdje je:

$$c_{[1-2]} = -0.1735 - (-0.1441) = -0.0294.$$

Standardna devijacija za sve kontraste korištenjem ovog skupa mjerenja je:

$$s_e = \sqrt{\frac{\sum_{j=1}^k (w_j^2 s_e^2)}{kn}} = s_e \sqrt{\frac{\sum (1^2 + (-1)^2 + 0^2)}{3(5)}} = 0.0754(0.3651) = 0.0275.$$

Za nivo povjerenja od 90% nalazi se da je $t_{1-\alpha/2; k(n-1)} = 1.782$, pa je interval povjerenja:

$$[-0.0294 \mp (1.782)(0.0275)] = [-0.0784, 0.0196].$$

Pošto ovaj interval povjerenja uključuje nulu, zaključuje se da ne postoji statistički značajna razlika između vremena potrebnih za izvršavanje instrukcija *call* i *return* na sistemima 1 i 2.

Na sličan način može se konstruisati kontrast za razliku između efekata sistema 1 i 3 postavljajući faktore na $w_1 = 1$, $w_2 = 0$ i $w_3 = -1$. Dobija se da je $c_{[1-3]} = -0.4910$. Za odgovarajući 90%-tni interval povjerenja za ovaj kontrast se dobije:

$$[-0.4910 \mp (1.782)(0.0275)] = [-0.540, -0.442]$$

Pošto ovaj interval ne uključuje nulu, može se zaključiti da se ne može reći da ne postoji statistički značajna razlika između ova dva sistema.

5.4. Linearni regresioni model

Mjerenje performansi računarskog sistema za sve moguće ulazne kombinacije bi dalo potpun odgovor na pitanje kako će se sistem ponašati pod određenim uslovima. Međutim, često je preskupo, ako ne i nemoguće, načiniti sva ta mjerenja. Umjesto toga se došlo na ideju da se izmjere performanse sistema za ograničen broj ulaza i da se na osnovu dobijenih rezultata razvije matematički model koji opisuje ponašanje sistema za određeni opseg ulaznih vrijednosti. Nakon toga, razvijeni model se može koristiti za predviđanje kako će se sistem ponašati za neku ulaznu vrijednost koja nije direktno izmjerena. Razvijeni model je poznat pod nazivom regresioni model.

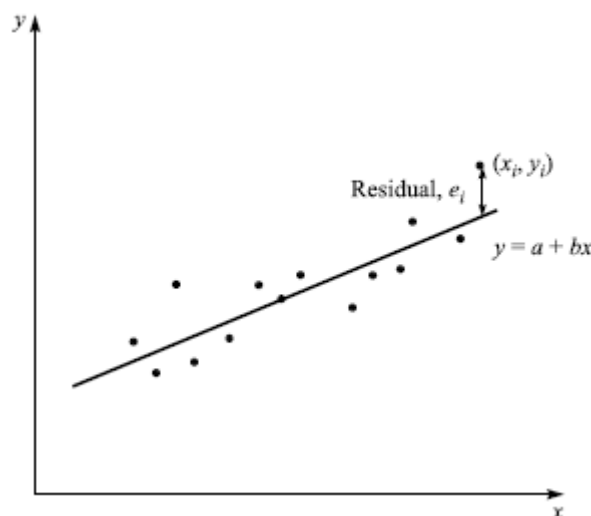
U slučaju da se izmjereni skup vrijednosti približno nalazi na istoj pravoj, može se razviti linearni regresioni model. U ovoj vrsti modela se može primijeniti metoda minimizacije najmanjih kvadrata u svrhu dobijanja linearno-regresione jednačine koja će da modeluje sistem. Oblik navedene jednačine je:

$$y = a + bx$$

U navedenoj jednačini x predstavlja ulaznu varijablu, y predviđeni izlaz, a a i b su regresioni parametri koji se moraju procijeniti na osnovu skupa izvršenih mjerenja. Ako y_i predstavlja vrijednost koja se izmjeri za ulazni parametar x_i , onda svaki od parova (x_i, y_i) može da se zapiše na sljedeći način:

$$y_i = a + bx_i + e_i$$

U gornjoj jednačini e_i predstavlja razliku između izmjerene vrijednosti y_i i vrijednosti koju predviđa regresioni model za isti ulazni parametar. Značenje vrijednosti e_i je prikazano na slici 5.5. Ova razlika se naziva rezidualna vrijednost mjerenja.



Slika 5.5 – Linearni regresioni model

Da bi se izračunali regresioni parametri a i b , vrši se minimizacija sume kvadrata reziduala. Suma se označava sa SSE . Jednačina koja se minimizuje je:

$$SSE = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - a - bx_i)^2.$$

Minimizacija ove jednačine se vrši tako što se posebno diferencira po a i po b , pa se dobijene jednačine izjednačavaju sa nulom i rješava se dobijeni sistem. Nakon završetka opisanog postupka dobijaju se sljedeće vrijednosti za parametre a i b :

$$b = \frac{n \sum_{i=1}^n x_i y_i - (\sum_{i=1}^n x_i)(\sum_{i=1}^n y_i)}{n \sum_{i=1}^n x_i^2 - (\sum_{i=1}^n x_i)^2}$$

$$a = \bar{y} - b\bar{x}.$$

U jednačini za dobijanje parametra a koristi se srednja vrijednost skupa ulaznih parametara i srednja vrijednost dobijenih rezultata mjerenja.

Na dobijene vrijednosti se dodatno mogu primjeniti intervali povjerenja, ali se taj dio neće obrađivati na ovom kursu.

Primjer. Treba razviti regresioni model koji će povezivati vrijeme potrebno za operaciju čitanja fajlova i broj pročitanih bajtova. Izvodi se eksperiment u kojem se varira broj pročitanih bajtova od 10 do 10,000 i mjeri odgovarajuće vrijeme čitanja. Dobijeni rezultati mjerenja su prikazani u tabeli 5.13.

Tabela 5.13 – *Primjer linearnog regresionog modela*

File size (bytes) (x_i)	Time (μ s) (y_i)
10	3.8
50	8.1
100	11.9
500	55.6
1,000	99.6
5,000	500.2
10,000	1,006.1

Na osnovu prikazanih podataka mogu se izračunati sljedeće vrijednosti:

$$\sum_{i=1}^7 x_i = 16,660.0$$

$$\sum_{i=1}^7 y_i = 1,685.3$$

$$\sum_{i=1}^7 x_i y_i = 12,691,033.0$$

$$\sum_{i=1}^7 x_i^2 = 126,262,600.0$$

$$\bar{x} = 2,380$$

$$\bar{y} = 240.76.$$

Uvrštavanje dobijenih vrijednosti u jednačine za parametre a i b rezultira:

$$b = \frac{(7)(12,691,033.0) - (16,660.0)(1,685.3)}{(7)(126,262,600.0) - (16,660.0)^2} = 0.1002$$

$$a = 240.76 - (0.1002)(2,380) = 2.24.$$

Korištenjem ovih parametara formira se jednačina za linearni regresioni model posmatranog sistema:

$$y = 2.24 + 0.1002x.$$

Iz dobijene jednačine može se zaključiti da je vrijeme potrebno za čitanje fajla veličine od, na primjer, 3,000 bajtova otprilike $0.1002(3,000) = 303 \mu s$.

Bitno je napomenuti da se preporučuje primjena linearnog regresionog modela samo za predviđanje opsega vrijednosti za ulazne parametre između najmanjeg i najvećeg za koje su izmjereni rezultati. Za ulazne vrijednosti izvan tog opsega postoji velika vjerovatnoća da model neće raditi korektno.

5.4.1. Korelacija

Nakon razvijanja linearnog regresionog modela korisno je znati koliko dobro dobijena jednačina modeluje izmjerene podatke. Drugim riječima, postoji zainteresovanost za poznavanje koliko je jaka linearna korelacija između ulaza i izlaza. Koeficijent determinacije i njegov kvadratni korijen, koeficijent korelacije, predstavljaju kvantitativne mjere za linearnost.

Ukupna varijacija između izmjerenih vrijednosti i regresionog modela se može podijeliti u dvije komponente. Prva komponenta predstavlja dio varijacije koju objašnjava regresioni model. Druga komponenta je posljedica grešaka u mjerenjima. Koeficijent determinacije se definiše kao dio varijacije koju objašnjava regresioni model. Njegova oznaka je r^2 i moguće vrijednosti su mu u opsegu $0 \leq r^2 \leq 1$. Ako postoji savršena linearna veza između ulaza i izlaza, onda je cijela varijacija u izmjerenim izlazima objašnjena regresionim modelom. U ovom slučaju, sve izmjerene vrijednosti padaju direktno na regresionu liniju pa su sve rezidualne vrijednosti jednake nuli ($r^2 = 1$). Sa druge strane, ako je $r^2 = 0$, varijacija izlaznih vrijednosti nije uopšte objašnjena linearnim regresionim modelom.

Na osnovu prethodnog razmatranja može se zaključiti da koeficijent determinacije pruža uvid koliko dobro regresioni model odgovara podacima. Vrijednosti bliske jedinici upućuju na blisku linearnu vezu između ulaznih i izlaznih vrijednosti. Suprotno tome, vrijednosti blizu nuli upućuju da ne postoji linearna veza. U zadnjem slučaju, poznavanje ulaznih vrijednosti ne pomaže u predviđanju izlaznih vrijednosti. Slično tome, ako čak i postoji funkcionalna veza između ulaza i izlaza, a da nije linearna, opet bi vrijednost koeficijenta determinacije bila bliska nuli.

Kvadratni korijen koeficijenta determinacije se naziva koeficijent korelacije, koji se označava sa r . Opseg vrijednosti koeficijenta korelacije je $-1 \leq r \leq 1$. Vrijednost jednaka jedinici označava savršen linearan odnos između ulaznih i izlaznih vrijednosti. U ovom slučaju, svaka promjena ulazne vrijednosti će prouzrokovati odgovarajuću promjenu izlazne vrijednosti. Sa druge strane, vrijednost jednaka -1 označava da će bilo koja promjena ulazne vrijednosti izazvati suprotnu promjenu izlazne vrijednosti.

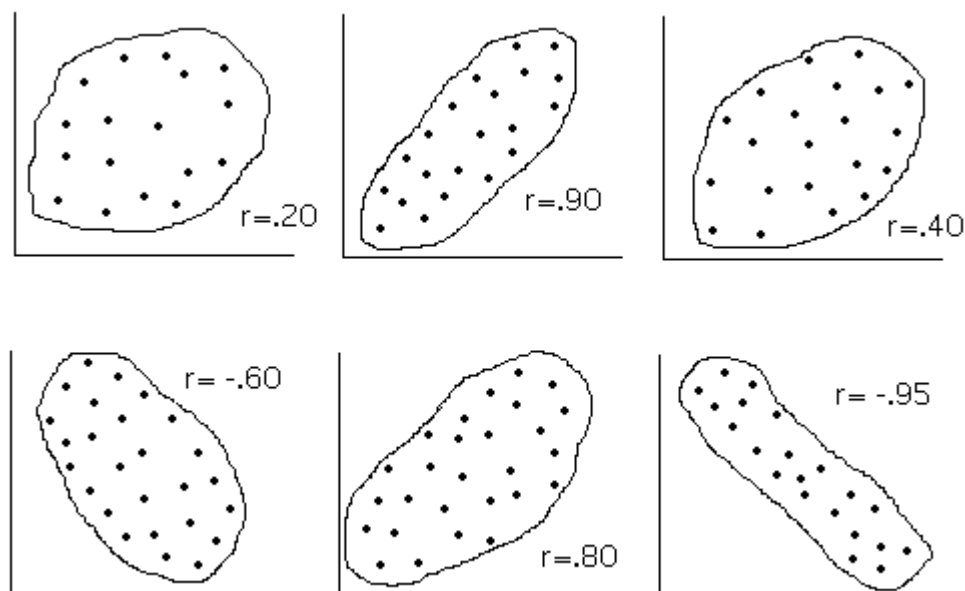
Bitno je naglasiti razliku između opisana dva koeficijenta. Recimo da su za dva različita sistema razvijeni linearni regresioni modeli i da su odgovarajući koeficijenti korelacije sistema $r_1 = 0.6$ i $r_2 = -0.9$. Na osnovu ovih vrijednosti se ne može zaključiti da je linearna veza drugog sistema 50% "bolja" od prvog. Jedini zaključak koji se može donijeti je da linearna veza drugog sistema izgleda snažnija.

Sa druge strane, ako se posmatraju koeficijenti determinacije, nalazi se da je $r_1^2 = 0.36$ i $r_2^2 = 0.81$. Sada je očigledno da je kod prvog sistema samo 36% varijacije objašnjeno linearnim regresionim modelom, a kod drugog 81%. Na osnovu ovih vrijednosti je vidljivo da je linearna veza drugog sistema mnogo snažnija od linearne veze prvog sistema.

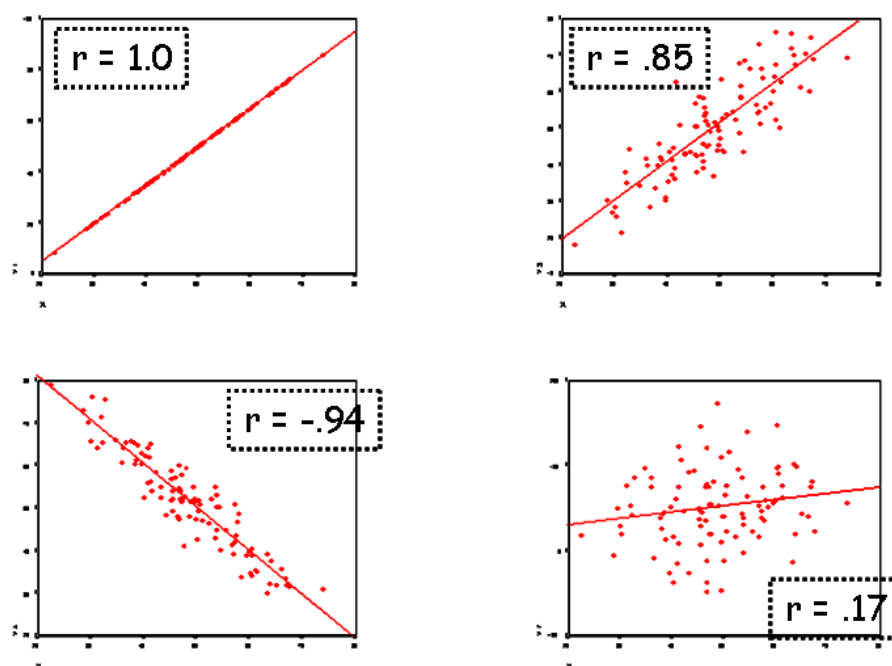
Na kraju ove sekcije bitno je naglasiti razliku između korelacije i kauzacije. Kauzacija govori da je promjena izlazne vrijednosti direktna posljedica promjene ulazne vrijednosti. Drugim riječima, postoji proces unutar sistema koji na određen način povezuje ulaz i izlaz. Međutim, moguće je da je izlaz u jakoj korelaciji sa ulazom koji direktno ne uzrokuje promjenu izlaza. Na primjer, postoji jaka korelacija između vremena potrebnog za slanje poruke na dijeljenoj mreži sa vremenom dana kad se poruka šalje (na primjer, LAN je opterećeniji za vrijeme radnog vremena nego poslije njega). Poznavanje vremena dana omogućava predikciju koliko će vremena biti potrebno

za slanje poruke. Međutim, vrijeme dana ne prouzrokuje vrijeme potrebno za slanje poruke. Pravi uzrok predstavlja broj ljudi koji u tom trenutku koriste usluge dijeljene mreže. Iako postoji jaka korelacija između ove dvije veličine, jedna ne prouzrokuje drugu. Zaključak se može svesti na rečenicu da korelacija ne implicira kauzaciju.

Primjer. Vizuelni primjeri korelacije sa skupovima mjerenja i odgovarajućim koeficijentima korelacije dati su na slikama 5.6 i 5.7.



Slika 5.6 – Primjeri korelacije (1)



Slika 5.7 – Primjeri korelacije (2)

Očigledno je da znak ispred koeficijenta korelacije zavisi od uticaja ulaza na izlaz.

5.4.2. Verifikovanje linearnosti

Važno je razmotriti da li je veza koja se modeluje u razvijanju linearnog regresionog modela uopšte linearna. Čak i ako veza nije linearna, još uvijek se mogu mehanički primjeniti formule radi izračunavanja vrijednosti regresionih parametara. Međutim, tako dobijeni model će biti jako slab pošto će davati pogrešne predikcije o izlaznim vrijednostima.

Dalje, primjena linearnog modela na nelinearne sisteme će dati pogrešne interpretacije sveukupnog ponašanja sistema. Zato je veoma važno verifikovati da li su ulazi i izlazi linearno povezani. Jednostavan nacrt izlaznih vrijednosti u funkciji od ulaznih je često dovoljan za verifikaciju pretpostavke o linearnosti. Konstruisanje ovog nacrta prije bilo kakvog regresionog računa treba da se smatra obaveznim.

5.4.3. Nelinearni modeli

Nekad je sisteme sa nelinearnim ulazno–izlaznim vezama moguće transformisati u linearnu formu. Zatim se može primjeniti linearno–regresiona jednačina na transformisane podatke.

Na primjer, pretpostavka je da je odnos ulaznih i izlaznih vrijednosti eksponencijalan, tako da je:

$$y = ab^x.$$

Ako se primjeni operacija logaritmovanja na obe strane jednakosti, dobija se:

$$\ln y = \ln a + (\ln b)x.$$

Ako izvršimo sljedeće zamjene:

$$y' = \ln y, a' = \ln a, \text{ and } b' = \ln b,$$

prethodna jednačina se može zapisati na sljedeći način koji se podudara sa standardnom formom linearno-regresione jednačine:

$$y' = a' + b'x.$$

Regresione formule se sada mogu primjeniti radi dobijanja vrijednosti a' i b' . Zatim se te vrijednosti mogu iskoristiti za dobijanje vrijednosti a i b iz nelinearnog modela.

Prethodni postupak transformacije nelinearne u linearnu funkciju se popularno naziva kurvilinearna regresija.

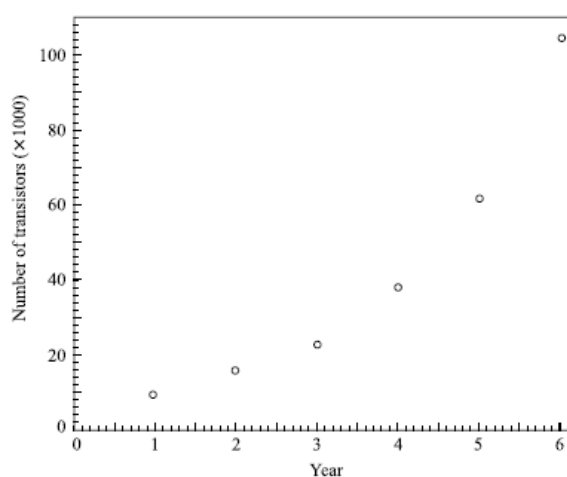
Primjer. Zadatak je predvidjeti rast broja tranzistora na određenom integrisanom čipu. Konkretno, potrebno je procijeniti koliko će tranzistora biti integrisano na čipu 4

godine od danas ako se pretpostavi da će broj rasti istim intenzitetom kao i zadnjih nekoliko godina. Podaci iz prethodnih šest godina su prikazani u tabeli 5.14.

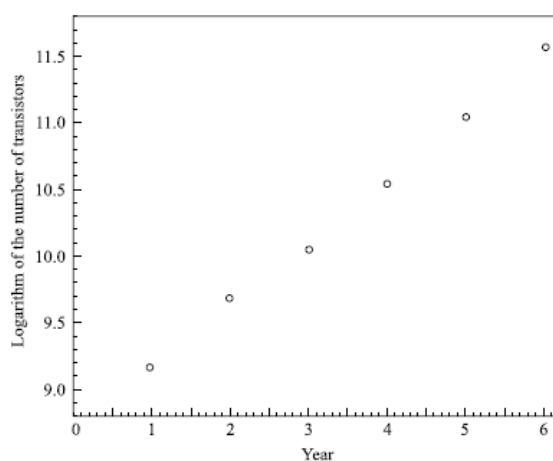
Tabela 5.14 – Primjer nelinearnog modela

Godina (x_i)	Broj tranzistora (y_i)	Transformisani podaci ($y'_i = \ln y_i$)
1	9,500	9.1590
2	16,000	9.6803
3	23,000	10.0432
4	38,000	10.5453
5	62,000	11.0349
6	105,000	11.5617

Nakon skiciranja vrijednosti iz tabele, što je prikazano na slici 5.8, očigledno je da je rast broja tranzistora u vremenu nelinearan. Zapravo, izgleda da rast prati eksponencijalnu raspodjelu. Nakon postupka linearizacije podataka dobijene su vrijednosti u zadnjoj koloni tabele 5.14, koje su skicirane na slici 5.9.



Slika 5.8 – Nelinearni model



Slika 5.9 – Linearizacija nelinearnog modela

Sada se mogu primjeniti linearne regresione jednačine. Prije njihovog korištenja, prvo je neophodno izračunati vrijednosti za izračunavanje vrijednosti regresionih parametara:

$$\sum_{i=1}^6 x_i = 21$$

$$\sum_{i=1}^6 x_i^2 = 91$$

$$\sum_{i=1}^6 y'_i = 62.0246$$

$$\sum_{i=1}^6 (y'_i)^2 = 645.1103$$

$$\sum_{i=1}^6 x_i y'_i = 225.3756$$

$$S_{xx} = 17.5$$

$$S_{yy} = 3.9354$$

$$S_{xy} = 8.2895.$$

Za regresione parametre se dobije da je $a' = 8.68$ i $b' = 0.474$. Sada se može izračunati da je:

$$a = e^{a'} = 5,881 \text{ and } b = e^{b'} = 1.61.$$

Sada se može formirati jednačina regresionog modela za posmatrani nelinearni sistem:

$$y = (5,881)1.61^x.$$

5.5. Verifikacija slučajnih brojeva

Problem verifikacije niza slučajnih brojeva se može podijeliti u dvije komponente. Prvo se koristi statistički test *goodness-of-fit* radi utvrđivanja da li je oblik generisane sekvence statistički blizak očekivanoj raspodjeli. Zatim se koriste ekstenzije tih

standardnih testova i neki dodatni testovi da se verifikuje da je redoslijed uzoraka takav da je svaki uzorak statistički nezavisan od drugih uzoraka. Ove dvije komponente su opisane u narednim sekcijama.

5.5.1. Goodness-of-fit testiranje

Goodness-of-fit testiranje je generalna tehnika koja se koristi radi upoređivanja generisanih ili izmjerenih vrijednosti sa idealnom ili očekivanom raspodjelom. Drugim riječima, cilj testiranja je da se odredi koliko su dobijene vrijednosti blizu očekivanim vrijednostima određene raspodjele. *Goodness-of-fit* test kvantifikuje kolika je devijacija dobijenih od očekivanih vrijednosti.

Na primjer, pretpostavka je da se vrši simulacija u kojoj treba generisati vrijednosti koje treba da prate binomnu raspodjelu sa parametrima $n = 5$ i $p = 0.2$. Ova raspodjela je definisana sljedećom funkcijom:

$$f(x) = \binom{n}{x} p^x (1-p)^{n-x}$$

U gornjoj jednačini p uzima vrijednosti između 0 i 1, n je pozitivan cijeli broj, a $x = 0, 1, 2, \dots, n$.

Da bi se utvrdilo da li generisane vrijednosti prate željenu raspodjelu, generiše se 100 uzoraka i broji se koliko se puta svaki od šest mogućih slučajeva desio, gdje x može biti 0, 1, 2, 3, 4 i 5. Rezultati mjerenja su prikazani u prvom redu tabele 5.15. Sljedeći korak je da se odredi koliko puta se očekuje od svake vrijednosti da se pojavi ako je raspodjela stvarno binomna. Za $x = 0$, dobija se da je:

$$f(0) = \binom{5}{0} (0.2)^0 (1-0.2)^5 = 0.328.$$

Pošto je izmjereno 100 uzoraka, dobijena vjerovatnoća se mora pomnožiti sa 100. Ovim se dobija da je predviđeni broj ishoda za $x = 0$ jednak 32.8. Na sličan način se dobijaju i vrijednosti za $f(1)$, $f(2)$, $\dots$, $f(5)$. Te vrijednosti su prikazane u drugom redu tabele 5.15.

Tabela 5.15 – Primjer eksperimenta sa binomnom raspodjelom

	0	1	2	3	4	5
Observed (O_i)	36	40	18	6	0	0
Predicted (P_i)	32.8	40.9	20.5	5.1	0.7	≈ 0

Primjetno je da vrijednosti u redovima tabele nisu identične, što nije nimalo iznenađujuće. Pitanje je koliko je vjerovatno da su uzrok dobijene varijacije između izmjerenih i očekivanih vrijednosti normalne slučajne fluktuacije.

U ovu svrhu se koristi hi-kvadrat test. Hi-kvadrat vrijednost za posmatrani skup mjerenja se dobija po formuli:

$$\chi_{m-1}^2 = \sum_{i=1}^m \frac{(O_i - P_i)^2}{P_i}.$$

Oslanjajući se na centralnu graničnu teoremu statističari su utvrdili da broj izračunat pomoću prethodne formule prati hi-kvadrat raspodjelu sa $m - 1$ stepeni slobode, gdje m predstavlja broj različitih mogućih ishoda. Kritične vrijednosti ove raspodjele date su u tabeli u prilogu C. Ove kritične vrijednosti se zatim mogu iskoristiti da bi se utvrdilo da li su devijacije između dobijenih i očekivanih vrijednosti usljed slučajnih fluktuacija. Ako nisu, zaključak je da je malo vjerovatno da dobijene vrijednosti prate očekivanu raspodjelu.

Pošto se hi-kvadrat raspodjela zasniva na centralnoj graničnoj teoremi, neophodno je da svaki mogući ishod ima dovoljno velik broj očekivanih vrijednosti. Taj broj mora da bude minimalno 5 ili 6. Ako ovaj uslov nije ispunjen, susjedni mogući ishodi se moraju posmatrati kao jedan dok rezultujući novi ishod nije dovoljno velik da bi se uslov ispunio.

Ako se posmatraju vrijednosti iz tabele 5.15, vidljivo je da se ishodi 4 i 5 pojavljuju suviše rijetko da bi se zadovoljio prethodno navedeni uslov. Kao posljedica toga, ishodi 3, 4 i 5 se kombinuju u jedan ishod čija je očekivana vrijednost jednaka zbiru konkateniranih, tj. 5.8. Dobijena vrijednost novog objedinjenog ishoda je 6. Korespondirajuća hi-kvadrat vrijednost za ovaj primjer sada se računa kao:

$$\chi_{4-1}^2 = \frac{(36 - 32.8)^2}{32.8} + \dots + \frac{(6 - 5.8)^2}{5.8} = 0.6438.$$

Kritične vrijednosti iz tabele u prilogu C pokazuju vjerovatnoću da će se tabelarna vrijednost desiti zbog slučajnih fluktuacija, sa određenim brojem stepeni slobode. U ovom primjeru taj broj je $m - 1 = 3$. Iz tabele se nalazi da postoji vjerovatnoća od 95% da će hi-kvadrat vrijednost biti manja od 7.815 sa 3 stepena slobode. Pošto je izračunata vrijednost u ovom primjeru manja od kritične vrijednosti iz tabele, zaključuje se sa 95%-tnom sigurnošću da nema razloga da se sumnja da dobijene vrijednosti ne dolaze iz očekivane raspodjele. Opet je primjetno da se ne može tvrditi da dobijene vrijednosti dolaze iz očekivane raspodjele, već da ne postoje dokazi koji uvjeravaju u suprotno tvrđenje.

Pošto se u jednačini za računanje vrijednosti vrši normalizacija sa vrijednostima P_i , slijedi da će manje vrijednosti P_i više uticati na dobijenu hi-kvadrat vrijednost od većih vrijednosti P_i . Da bi se ublažio ovaj uticaj, poželjno je kombinovati susjedne moguće ishode tako da su vrijednosti P_i za svaki mogući ishod približno jednake. Ova agregacija vrijednosti smanjuje dostupne informacije, ali s njom hi-kvadrat test bolje radi u praksi.

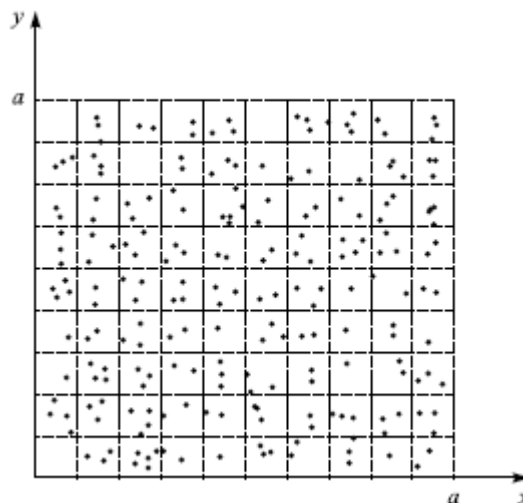
Postoje i drugi tipovi *goodness-of-fit* testova koji su pogodni za specifične situacije. Kolmogorov-Smirnov test je, na primjer, napravljen da bi poredio mali broj opservacija sa očekivanom kontinualnom raspodjelom. Ovo je suprotno od hi-kvadrat testa, koji pretpostavlja diskretnu raspodjelu sa velikim brojem uzoraka.

5.5.2. Testovi nezavisnosti

Činjenica da je sekvenca vrijednosti prošla uspješno kroz *goodness-of-fit* test nije garancija da je sekvenca uistinu slučajna. Na primjer, sekvenca (0, 1, 2, ..., 19, 0, 1, 2, ..., 19) savršeno prati *unif* (0, 19) raspodjelu i uspješno će proći bilo koji *goodness-of-fit* test. Međutim, za ovu sekvencu je očigledno da uopšte nije slučajna. *Goodness-of-fit* test kvantifikuje da li je broj jedinstvenih vrijednosti u sekvenci predaleko od očekivane vrijednosti svake od njih. Ali on daje informaciju o redoslijedu tih vrijednosti. Određivanje da li je redoslijed vrijednosti dovoljno slučajan predstavlja cilj testova nezavisnosti. Jedan od najpoznatijih testova nezavisnosti je serijski test.

Serijski test pokušava da verifikuje uniformnost sekvence *unif* (0, 1) vrijednosti u k dimenzija. Bazična ideja u dvije dimenzije je da se iscrtaju tačke u dvodimenzionalnom prostoru koristeći nepreklapajuće parove vrijednosti generisane sekvence. Vrijednosti se grupišu u parove tako da svaki par (x, y) predstavlja koordinate tačke u 2D prostoru.

Prvi korak u verifikovanju uniformnosti predstavlja jednostavna vizuelna inspekcija. Pokušava se odgovoriti na pitanje da li se tačke čine uniformno raspoređenima kada se nacrtaju. Ako iscrtane tačke nisu nezavisne, tačke će težiti tome da formiraju linije ili će biti grupisane u klastere. U sljedećem koraku 2D prostor treba podijeliti u grid sa $a \times a$ ćelija, kao što je prikazano na slici 5.10. Ako je raspodjela tačaka u potpunosti uniformna, svaka ćelija treba da sadrži $n/2a^2$ tačaka, gdje n predstavlja ukupan broj generisanih slučajnih vrijednosti. Hi-kvadrat test sa $a^2 - 1$ stepeni slobode može da se iskoristi za precizniju verifikaciju uniformnosti umjesto jednostavnog vizuelnog testa.



Slika 5.10 – Serijski test nezavisnosti

Serijski test se može proširiti na k dimenzija ako se dobijene vrijednosti grupišu u k -torke i predstave u prostoru sa k dimenzija.

Ostali testovi nezavisnosti uključuju serijsko-korelacione testove, spektralne testove i *runs-up* testove. Svi navedeni testovi, uključujući i serijski, ipak daju samo dovoljan, ali ne i potreban uslov statističke nezavisnosti.

6. SAOPŠTAVANJE REZULTATA

Važan dio svake studije evaluacije performansi je prezentacija završnih rezultata. Konačan cilj svake analize performansi je pomoć u donošenju odluka. Analiza čiji rezultati ne mogu da se protumače od strane ljudi koji donose odluke je jednako vrijedna kao analiza koja nije ni izvršena. Samim tim, na analitičaru performansi je odgovornost da rezultate saopšti na što jednostavniji način. Ovo zahtijeva korištenje riječi, slika i grafova radi objašnjavanja rezultata i analize.

Postoji velik skup grafova koji se koriste za prezentovanje dobijenih rezultata performansi računarskih sistema. U taj skup spadaju linijski, *bar* i *pie* grafovi, kao i histogrami.

Postoji veliki broj razloga zbog čega se grafovi koriste kao zamjena za tekstualni prikaz dobijenih rezultata mjerenja. Poznato je da slika predstavlja hiljadu riječi. Grafici smanjuju potrebno vrijeme za tumačenje izloženih informacija, a istovremeno se informacije predstavljaju na koncizniji način. Takođe, grafik će prije da zainteresuje čitaoca od tekstualnog prikaza podataka. Grafički prikaz podataka je takođe pogodan za naglašavanje neke činjenice i za sumiranje rezultata izvršene studije.

6.1. Tipovi varijabli

Postoje dva tipa varijabli koje se prikazuju na graficima:

1. *kvalitativne*
2. *kvantitativne*

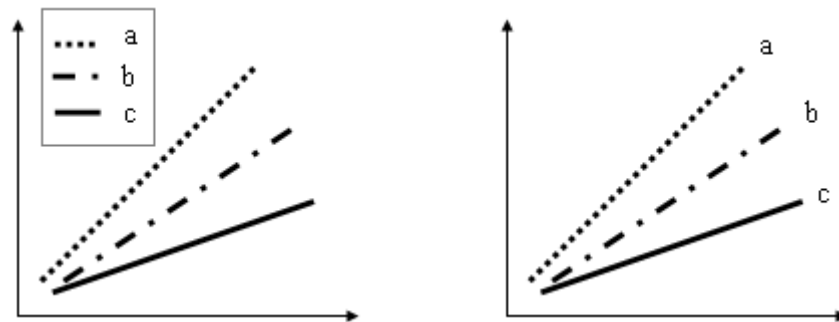
Navedeni tipovi su prethodno opisani u skripti.

6.2. Preporuke za pravljenje dobrih grafičkih dijagrama

Sljedeće preporuke pomažu u poboljšavanju grafičkih prezentacija. Navedene preporuke su uglavnom generalne i primjena svake od njih predstavlja vještinu koja se stiče sa iskustvom. Bitno je napomenuti da sve napomene koje će biti navedene predstavljaju preporuke, a ne pravila. Generalne preporuke su:

1. *Graf treba da zahtijeva minimalan napor od strane njegovog posmatrača.*

Predstavlja vjerovatno najvažniju metriku u mjerenju koliko je određen graf dobar. Preporuka se može demonstrirati pomoću korištenja legende na dijagramu. Postoje dva načina označavanja različitih linija na linijskom dijagramu – pomoću *legend box*-a i pomoću direktnog labeliranja. Na osnovu primjera sa slike 6.2 očigledno je da je direktno labeliranje pogodnije, a to pogotovo važi ako je broj linija velik.



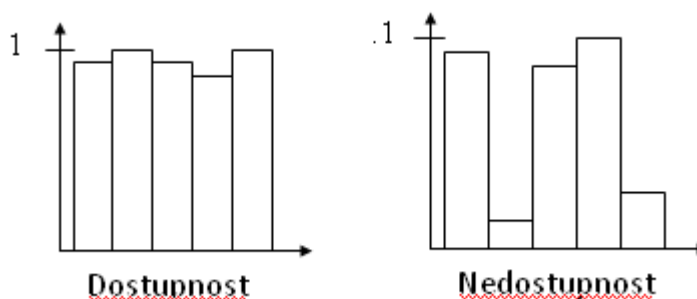
Slika 6.2 – Minimalan napor od strane posmatrača

2. Maksimiziranje količine informacija.

Na grafu je neophodno da postoji dovoljno informacija da bi on sam sebi bio dovoljan. Postoji više načina kako da se ovo ostvari. Na primjer, treba koristiti pune riječi umjesto simbola pošto simboli zahtjevaju dodatan napor od posmatrača. Ose na grafu treba da nose što je moguće više informacija i da uključuju jedinice. Na primjer, “CPU vrijeme u sekundama” nosi više informacija od “CPU vrijeme”.

3. Minimiziranje tinte.

Treba predstaviti što više informacija sa što manje pisanja (tinte). Previše nepotrebnih informacija na grafu ga čini zbrkanim i neinteresantnim. Na primjer, grid linije ne treba da se prikazuju ako nisu neophodne da bi se tačno pročitale vrijednosti. Grafički primjer ove preporuke je prikazan na slici 6.3. Na prvom grafu je prikazana dostupnost sistema, a na drugom nedostupnost koja je predstavljena kao 1 – dostupnost, a i skala je izmjenjena. Očigledno je da je drugi graf smisleniji i da daje više informacija od prvog.



Slika 6.3 – Minimiziranje tinte

4. Koristiti široko prihvaćene običaje.

Treba da se predstavi ono što ljudi očekuju. Na primjer, koordinatni početak treba da bude u tački (0, 0), uzrok (nezavisna veličina) treba da bude na x-osi, a posljedica (zavisna veličina) na y-osi, skale treba da budu linearne i da se povećavaju slijeva u desno i odozdo prema gore. Odstupanja od ovih preporuka

su dozvoljena, ali zatijevaju dodatan napor za čitanje informacija pa se preporučuju samo u rijetkim situacijama.

5. Izbjegavati dvosmislenosti.

Treba prikazati strelice na osama, podjele skala i koordinatni početak, kao i identifikovati pojedine krive i *bar*-ove. Graf treba lako da se čita. Ne treba prikazivati više varijabli na istom dijagramu.

6.3. Česte greške na dijagramima

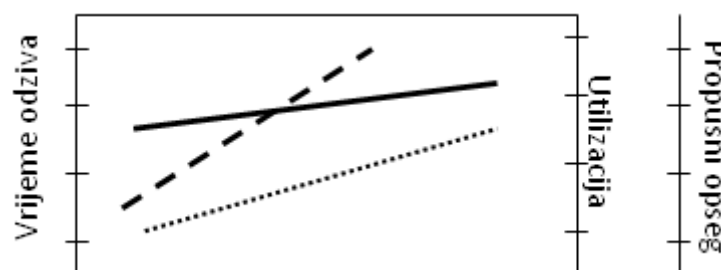
U ovoj sekciji su navedene neke greške koje se najčešće javljaju pri grafičkoj prezentaciji podataka:

1. Predstavljanje previše alternativa na jednom dijagramu.

Bez obzira od tipa grafa koji se koristi, ključni cilj predstavljanja podataka je da se njihovo čitanje napravi što lakšim. Većina posmatrača ne može da istovremeno poima više od 5 do 7 poruka, što je povezano sa limitima ljudske kratkoročne memorije. Na osnovu toga, dijagram treba da bude ograničen na 6 krivih linija, *bar* i kolonski dijagrami na 10 različitih vrijednosti, a *pie* dijagrami na 8 komponenata. Svaki mogući ishod na histogramu mora da ima najmanje 5 vrijednosti.

2. Predstavljanje previše y-varijabli na jednom dijagramu.

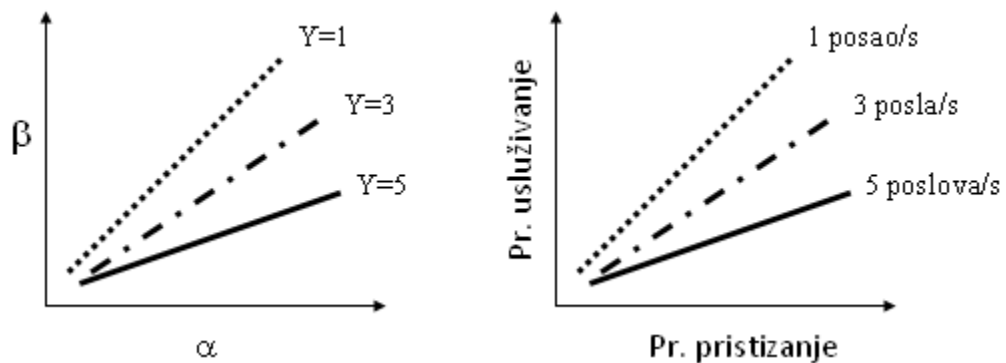
Na slici 6.4 su na istom dijagramu predstavljeni vrijeme odziva, utilizacija i propusni opseg sistema u funkciji od broja korisnika. Predstavljanje tri y-varijable istovremeno štedi prostor, ali posmatraču ostavlja problem asociiranja krivih sa odgovarajućom skalom što mu znatno otežava da dobije željenu poruku. Samim tim, prezentovanje svake y-varijable na zasebnom dijagramu je mnogo logičnije i bolje rješenje.



Slika 6.4 – Predstavljanje previše y-varijabli na jednom dijagramu

3. Korištenje simbola umjesto teksta.

Slika 6.5 prikazuje linijski dijagram koji je težak za razumijevanje pošto na dijagramu ne postoji tekst. Posmatrač mora dodatno da prođe kroz tekst da bi otkrio šta svaki od navedenih simbola zapravo predstavlja.



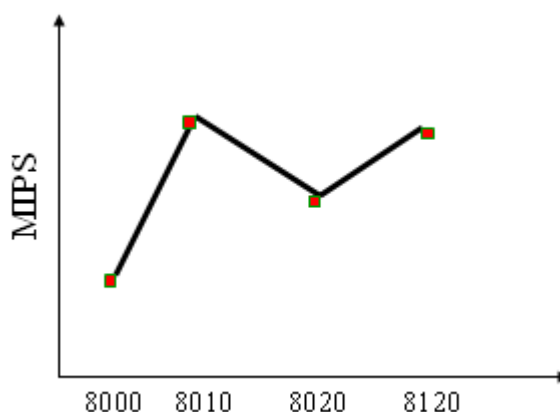
Slika 6.5 – Korištenje simbola i teksta za označavanje dijagrama

4. Postavljanje dodatnih informacija na dijagram.

Cilj svakog dijagrama je da prenese određenu poruku svom posmatraču. Bilo koja informacija koja odvlači pažnju posmatrača od te poruke je suvišna i treba je izbjegavati.

5. Korištenje linijskog umjesto kolonskog dijagrama.

Linije koje povezuju uzastopne tačke na grafu naglašavaju činjenicu da se vrijednosti između tačaka mogu približno interpolirati. Slika 6.6 pokazuje primjer pogrešne upotrebe linijskog dijagrama. U ovom primjeru su prikazane performanse izražene u MIPS-u za različite tipove CPU-ova. Pošto frakcionalni dijelovi CPU tipova nemaju smisla, u ovom slučaju treba koristiti kolonske ili *pie* dijagrame.



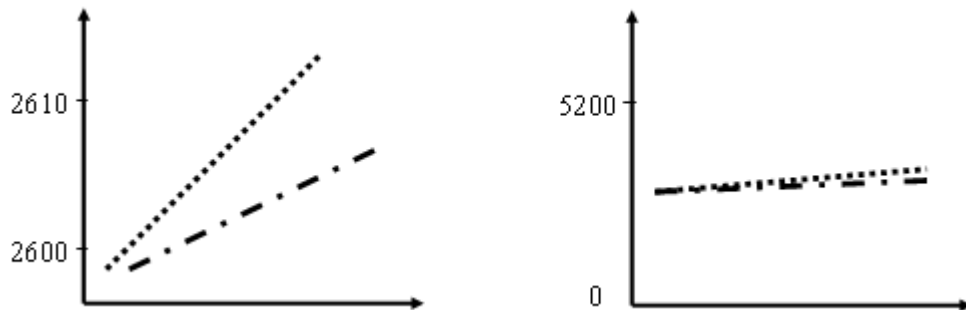
Slika 6.6 – Pogrešna upotreba linijskog dijagrama

6.4. Grafičke varke

U ovoj sekciji su predstavljene tehnike koje analitičari koriste da namjerno sakriju istinu i prevare posmatrača grafa. Neke od takvih varki su:

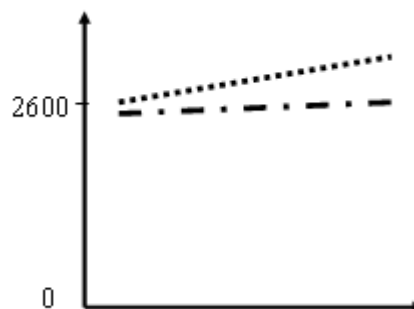
1. Korištenje nenultog koordinatnog početka.

U normalnoj situaciji ose grafa se sijeku u koordinatnom početku koji se označava sa tačkom (0, 0). Pomjeranjem koordinatnog počeka i odgovarajućim skaliranjem grafa, moguće je uvećati ili smanjiti percepciju razlike u performansama. Na slici 6.7, iste performanse su prikazane na dva različita grafa korištenjem različitih skala i različitih koordinatnih početaka. Dva grafa sada dovode do izvođenja dva različita zaključka iako se radi o istim podacima. Po prvom ispada da je jedna alternativa mnogo bolja od druge, a po drugom ispada da su performanse alternativa skoro jednake.



Slika 6.7 – Iste performanse predstavljene na dva različita načina

Međutim, nijedan od dva grafa nije pravilan. Pravi način za skaliranje grafa je da se skala izabere tako da je vertikalna visina najviše tačke najmanje $\frac{3}{4}$ horizontalnog odmaka najdesnije tačke. Takođe, koordinatni početak treba da se predstavi tačkom (0, 0) osim ako ne postoji neki opravdan razlog da se tako ne uradi. Performanse sistema sa slike 6.7 su na pravilan način predstavljene na slici 6.8.

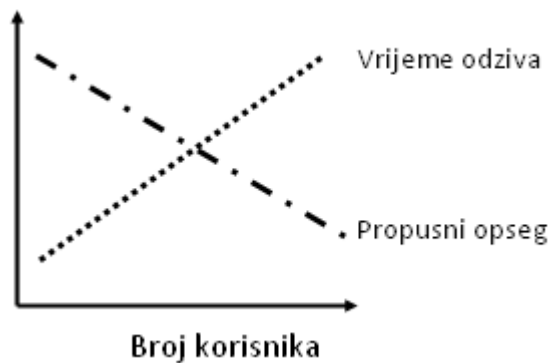


Slika 6.8 – Pravilo tri-četvrtine i nulti koordinatni početak

2. Korištenje double-whammy grafova radi dramatiziranja.

Dvije krive na istom grafu mogu imati dvostruki učinak u odnosu na samo jednu. Ovo se može iskoristiti u svrhu naglašavanja informacije, bez obzira da li je ona dobra ili loša. Ako postoje dvije metrike koje su povezane, poznavanjem jedne može se automatski zaključiti druga. Propusni opseg i vrijeme odziva su primjer povezanih metrika. Međutim, ako se obe ove metrike predstave na istom grafu, kao što je urađeno na slici 6.9, uticaj na nestručnog posmatrača

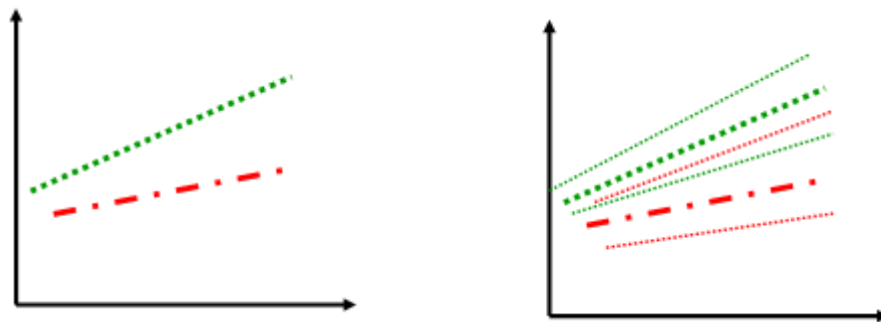
može da bude dvostruko veći u odnosu da je na grafu predstavljena samo jedna metrika.



Slika 6.9 – Double-whammy grafovi

3. Predstavljanje brojeva bez intervala povjerenja.

Mnogi rezultati mjerenja predstavljaju slučajnu kvantifikaciju performansi. Ako se postupak mjerenja ponovi, moguće je da rezultati ne bi bili isti. Međutim, i kada se izvrši nekoliko mjerenja i predstavi njihova srednja vrijednost, opet ne mora da znači da se performanse statistički razlikuju, što je i opisano u prethodnoj glavi. Na slici 6.10 su prikazane krive koje prikazuju performanse dva sistema sa i bez intervala povjerenja. Na lijevom dijagramu izgleda da je jedan sistem bolji po performansama od drugog. Međutim, desni dijagram pokazuje da razlika između dva sistema možda i ne postoji.



Slika 6.10 – Predstavljanje performansi bez i sa intervalom povjerenja

4. Piktogrami skalirani po visini.

Jedan od načina za predstavljanje razlike u performansama je crtanje odgovarajuće skaliranih slika sistema. Primjer je prikazan na slici 6.11. Performanse lijevog sistema su duplo bolje od performansi desnog sistema. Taj odnos je na lijevom dijelu slike prikazan na način da su performanse lijevog sistema prikazane duplo većim i po širini i po visini. Samim tim, površina lijevog pravougaonika postaje četiri puta veća od površine desnog. Pravilan način za skaliranje piktograma je prikazan na desnom dijelu slike, gdje je skaliranje

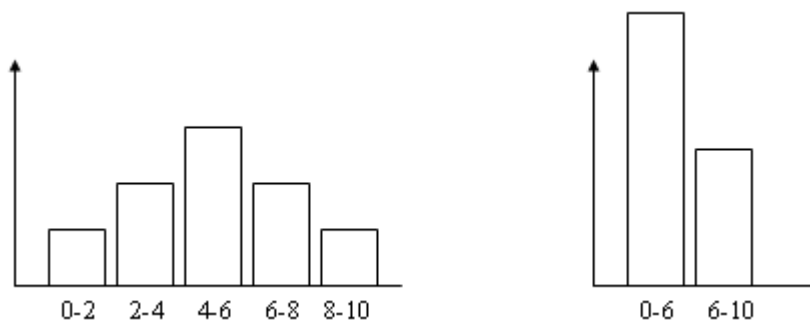
urađeno po površini. Po tom skaliranju, širina i visina desnog pravougaonika treba da budu 1.41 puta veće od širine i visine lijevog pravougaonika.



Slika 6.11 – Piktogrami skalirani po visini

5. Korištenje neodgovarajućih veličina ćelija u histogramima

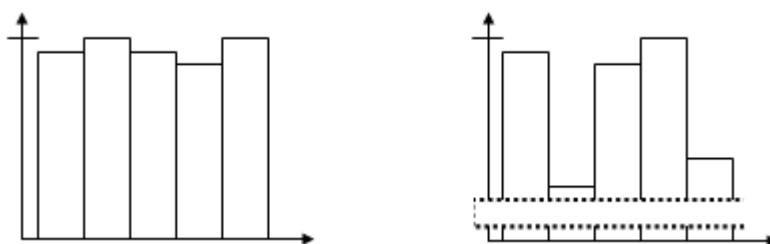
Odabir širina ćelija uvijek predstavlja izazov. Uglavnom zahtijeva više od jednog pokušaja. Ako su ćelije preširoke, svi podaci padaju u samo nekoliko ćelija. Sa druge strane, ako su ćelije preuske, na histogramu se gubi osjećaj. Odgovarajućim izborom širine ćelije, podaci se mogu predstaviti kao da pripadaju odgovarajućoj raspodjeli. Na slici 6.12 su prikazana dva histograma pomoću kojih se predstavljeni isti podaci. Na prvom su podaci grupisani tako da izgleda da oni prate Gausovu raspodjelu, a na drugom da prate eksponencijalnu raspodjelu.



Slika 6.12 – Isti podaci prikazani tako da prate različite raspodjele

6. Korištenje prelomljenih skala u kolonskim dijagramima.

Efekat sličan onom koji izaziva izbor nenultog koordinatnog početka se može postići prelamanjem skale po sredini, kao što je prikazano na slici 6.13. Ako se posmatra desni dio slike, može se zaključiti da postoje znatne razlike između sistema, dok lijevi dio slike daje suprotnu poruku.



Slika 6.13 – Korištenje prelomljenih skala u kolonskim dijagramima.

PRILOG A – Kritične vrijednosti Studentove t raspodjele

<i>n</i>	<i>a</i>						
	0.7000	0.8000	0.9000	0.9500	0.9750	0.9950	0.9995
1	0.727	1.376	3.078	6.314	12.706	63.657	636.619
2	0.617	1.061	1.886	2.920	4.303	9.925	31.599
3	0.584	0.979	1.638	2.354	3.183	5.841	12.924
4	0.569	0.941	1.533	2.132	2.777	4.604	8.610
5	0.559	0.920	1.476	2.015	2.571	4.032	6.869
6	0.553	0.906	1.440	1.943	2.447	3.708	5.959
7	0.549	0.896	1.415	1.895	2.365	3.500	5.408
8	0.546	0.889	1.397	1.860	2.306	3.356	5.041
9	0.544	0.883	1.383	1.833	2.262	3.250	4.781
10	0.542	0.879	1.372	1.813	2.228	3.169	4.587
11	0.540	0.876	1.363	1.796	2.201	3.106	4.437
12	0.539	0.873	1.356	1.782	2.179	3.055	4.318
13	0.538	0.870	1.350	1.771	2.161	3.012	4.221
14	0.537	0.868	1.345	1.761	2.145	2.977	4.141
15	0.536	0.866	1.341	1.753	2.132	2.947	4.073
16	0.535	0.865	1.337	1.746	2.120	2.921	4.015
17	0.534	0.863	1.333	1.740	2.110	2.898	3.965
18	0.534	0.862	1.330	1.734	2.101	2.879	3.922
19	0.533	0.861	1.328	1.729	2.093	2.861	3.884
20	0.533	0.860	1.325	1.725	2.086	2.845	3.850
21	0.533	0.859	1.323	1.721	2.080	2.832	3.819
22	0.532	0.858	1.321	1.717	2.074	2.819	3.792
23	0.532	0.858	1.320	1.714	2.069	2.807	3.768
24	0.531	0.857	1.318	1.711	2.064	2.797	3.746
25	0.531	0.856	1.316	1.708	2.060	2.788	3.725
26	0.531	0.856	1.315	1.706	2.056	2.779	3.707
27	0.531	0.855	1.314	1.703	2.052	2.771	3.690
28	0.530	0.855	1.313	1.701	2.048	2.763	3.674
29	0.530	0.854	1.311	1.699	2.045	2.757	3.660
30	0.530	0.854	1.310	1.697	2.042	2.750	3.646
40	0.529	0.851	1.303	1.684	2.021	2.705	3.551
50	0.528	0.849	1.299	1.676	2.009	2.678	3.496
60	0.527	0.848	1.296	1.671	2.000	2.660	3.460
70	0.527	0.847	1.294	1.667	1.994	2.648	3.435
80	0.527	0.846	1.292	1.664	1.990	2.639	3.416
90	0.526	0.846	1.291	1.662	1.987	2.632	3.402
100	0.526	0.845	1.290	1.660	1.984	2.626	3.391
110	0.526	0.845	1.289	1.659	1.982	2.621	3.381
120	0.526	0.845	1.289	1.658	1.980	2.618	3.374
130	0.526	0.844	1.288	1.657	1.978	2.614	3.367
140	0.526	0.844	1.288	1.656	1.977	2.612	3.362
150	0.526	0.844	1.287	1.655	1.976	2.609	3.357
∞	0.524	0.842	1.282	1.645	1.960	2.576	3.291

PRILOG B.1 – Kritične vrijednosti F raspodjele (90%)

n_2	n_1						
	1	2	3	4	5	6	10
1	39.863	49.500	53.593	55.833	57.240	58.204	60.195
2	8.526	9.000	9.162	9.243	9.293	9.326	9.392
3	5.538	5.462	5.391	5.343	5.309	5.285	5.230
4	4.545	4.325	4.191	4.107	4.051	4.010	3.920
5	4.060	3.780	3.619	3.520	3.453	3.405	3.297
6	3.776	3.463	3.289	3.181	3.108	3.055	2.937
10	3.285	2.924	2.728	2.605	2.522	2.461	2.323
20	2.975	2.589	2.380	2.249	2.158	2.091	1.937
40	2.835	2.440	2.226	2.091	1.997	1.927	1.763
60	2.791	2.393	2.177	2.041	1.946	1.875	1.707
80	2.769	2.370	2.154	2.016	1.921	1.849	1.680
100	2.756	2.356	2.139	2.002	1.906	1.834	1.663
200	2.731	2.329	2.111	1.973	1.876	1.804	1.631
500	2.716	2.313	2.095	1.956	1.859	1.786	1.612

n_2	n_1						
	20	40	60	80	100	200	500
1	61.740	62.529	62.794	62.927	63.007	63.167	63.264
2	9.441	9.466	9.475	9.479	9.481	9.486	9.489
3	5.184	5.160	5.151	5.147	5.144	5.139	5.136
4	3.844	3.804	3.790	3.782	3.778	3.769	3.764
5	3.207	3.157	3.140	3.132	3.126	3.116	3.109
6	2.836	2.781	2.762	2.752	2.746	2.734	2.727
10	2.201	2.132	2.107	2.095	2.087	2.071	2.062
20	1.794	1.708	1.677	1.660	1.650	1.629	1.616
40	1.605	1.506	1.467	1.447	1.434	1.406	1.389
60	1.543	1.437	1.395	1.372	1.358	1.326	1.306
80	1.513	1.403	1.358	1.334	1.318	1.284	1.261
100	1.494	1.382	1.336	1.310	1.293	1.257	1.232
200	1.458	1.339	1.289	1.261	1.242	1.199	1.168
500	1.435	1.313	1.260	1.229	1.209	1.160	1.122

PRILOG B.2 – Kritične vrijednosti F raspodjele (95%)

n_2	n_1						
	1	2	3	4	5	6	10
1	161.448	199.500	215.707	224.583	230.162	233.986	241.882
2	18.513	19.000	19.164	19.247	19.296	19.330	19.396
3	10.128	9.552	9.277	9.117	9.013	8.941	8.786
4	7.709	6.944	6.591	6.388	6.256	6.163	5.964
5	6.608	5.786	5.409	5.192	5.050	4.950	4.735
6	5.987	5.143	4.757	4.534	4.387	4.284	4.060
10	4.965	4.103	3.708	3.478	3.326	3.217	2.978
20	4.351	3.493	3.098	2.866	2.711	2.599	2.348
40	4.085	3.232	2.839	2.606	2.449	2.336	2.077
60	4.001	3.150	2.758	2.525	2.368	2.254	1.993
80	3.960	3.111	2.719	2.486	2.329	2.214	1.951
100	3.936	3.087	2.696	2.463	2.305	2.191	1.927
200	3.888	3.041	2.650	2.417	2.259	2.144	1.878
500	3.860	3.014	2.623	2.390	2.232	2.117	1.850

n_2	n_1						
	20	40	60	80	100	200	500
1	248.013	251.143	252.196	252.724	253.041	253.677	254.059
2	19.446	19.471	19.479	19.483	19.486	19.491	19.494
3	8.660	8.594	8.572	8.561	8.554	8.540	8.532
4	5.803	5.717	5.688	5.673	5.664	5.646	5.635
5	4.558	4.464	4.431	4.415	4.405	4.385	4.373
6	3.874	3.774	3.740	3.722	3.712	3.690	3.678
10	2.774	2.661	2.621	2.601	2.588	2.563	2.548
20	2.124	1.994	1.946	1.922	1.907	1.875	1.856
40	1.839	1.693	1.637	1.608	1.589	1.551	1.526
60	1.748	1.594	1.534	1.502	1.481	1.438	1.409
80	1.703	1.545	1.482	1.448	1.426	1.379	1.347
100	1.676	1.515	1.450	1.415	1.392	1.342	1.308
200	1.623	1.455	1.386	1.346	1.321	1.263	1.221
500	1.592	1.419	1.345	1.303	1.275	1.210	1.159

PRILOG B.3 – Kritične vrijednosti F raspodjele (99%)

n_2	n_1						
	1	2	3	4	5	6	10
1	4,052.181	4,999.500	5,403.352	5,624.583	5,763.650	5,858.986	6,055.847
2	98.503	99.000	99.166	99.249	99.299	99.333	99.399
3	34.116	30.817	29.457	28.710	28.237	27.911	27.229
4	21.198	18.000	16.694	15.977	15.522	15.207	14.546
5	16.258	13.274	12.060	11.392	10.967	10.672	10.051
6	13.745	10.925	9.780	9.148	8.746	8.466	7.874
10	10.044	7.559	6.552	5.994	5.636	5.386	4.849
20	8.096	5.849	4.938	4.431	4.103	3.871	3.368
40	7.314	5.179	4.313	3.828	3.514	3.291	2.801
60	7.077	4.977	4.126	3.649	3.339	3.119	2.632
80	6.963	4.881	4.036	3.563	3.255	3.036	2.551
100	6.895	4.824	3.984	3.513	3.206	2.988	2.503
200	6.763	4.713	3.881	3.414	3.110	2.893	2.411
500	6.686	4.648	3.821	3.357	3.054	2.838	2.356

n_2	n_1						
	20	40	60	80	100	200	500
1	6,208.730	6,286.782	6,313.030	6,326.197	6,334.110	6,349.967	6,359.501
2	99.449	99.474	99.482	99.487	99.489	99.494	99.497
3	26.690	26.411	26.316	26.269	26.240	26.183	26.148
4	14.020	13.745	13.652	13.605	13.577	13.520	13.486
5	9.553	9.291	9.202	9.157	9.130	9.075	9.042
6	7.396	7.143	7.057	7.013	6.987	6.934	6.902
10	4.405	4.165	4.082	4.039	4.014	3.962	3.930
20	2.938	2.695	2.608	2.563	2.535	2.479	2.445
40	2.369	2.114	2.019	1.969	1.938	1.874	1.833
60	2.198	1.936	1.836	1.783	1.749	1.678	1.633
80	2.115	1.849	1.746	1.690	1.655	1.579	1.530
100	2.067	1.797	1.692	1.634	1.598	1.518	1.466
200	1.971	1.694	1.583	1.521	1.481	1.391	1.328
500	1.915	1.633	1.517	1.452	1.408	1.308	1.232

PRILOG C – Kritične vrijednosti hi-kvadrat raspodjele

<i>n</i>	<i>α</i>						
	0.5000	0.7500	0.8000	0.9000	0.9500	0.9900	0.9950
1	0.455	1.323	1.642	2.706	3.842	6.635	7.879
2	1.386	2.773	3.219	4.605	5.992	9.210	10.597
3	2.366	4.108	4.642	6.251	7.815	11.345	12.838
4	3.357	5.385	5.989	7.779	9.488	13.277	14.860
5	4.352	6.626	7.289	9.236	11.071	15.086	16.750
6	5.348	7.841	8.558	10.645	12.592	16.812	18.548
7	6.346	9.037	9.803	12.017	14.067	18.475	20.278
8	7.344	10.219	11.030	13.362	15.507	20.090	21.955
9	8.343	11.389	12.242	14.684	16.919	21.666	23.589
10	9.342	12.549	13.442	15.987	18.307	23.209	25.188
11	10.341	13.701	14.631	17.275	19.675	24.725	26.757
12	11.340	14.845	15.812	18.549	21.026	26.217	28.300
13	12.340	15.984	16.985	19.812	22.362	27.688	29.820
14	13.339	17.117	18.151	21.064	23.685	29.141	31.319
15	14.339	18.245	19.311	22.307	24.996	30.578	32.801
16	15.339	19.369	20.465	23.542	26.296	32.000	34.267
17	16.338	20.489	21.615	24.769	27.587	33.409	35.719
18	17.338	21.605	22.760	25.989	28.869	34.805	37.157
19	18.338	22.718	23.900	27.204	30.144	36.191	38.582
20	19.337	23.828	25.038	28.412	31.410	37.566	39.997
21	20.337	24.935	26.171	29.615	32.671	38.932	41.401
22	21.337	26.039	27.302	30.813	33.925	40.290	42.796
23	22.337	27.141	28.429	32.007	35.173	41.639	44.181
24	23.337	28.241	29.553	33.196	36.415	42.980	45.559
25	24.337	29.339	30.675	34.382	37.653	44.314	46.928
26	25.337	30.435	31.795	35.563	38.885	45.642	48.290
27	26.336	31.528	32.912	36.741	40.113	46.963	49.645
28	27.336	32.621	34.027	37.916	41.337	48.278	50.994
29	28.336	33.711	35.140	39.088	42.557	49.588	52.336
30	29.336	34.800	36.250	40.256	43.773	50.892	53.672
40	39.335	45.616	47.269	51.805	55.759	63.691	66.766
50	49.335	56.334	58.164	63.167	67.505	76.154	79.490
60	59.335	66.982	68.972	74.397	79.082	88.380	91.952
70	69.335	77.577	79.715	85.527	90.531	100.425	104.215
80	79.334	88.130	90.405	96.578	101.880	112.329	116.321
90	89.334	98.650	101.054	107.565	113.145	124.116	128.299
100	99.334	109.141	111.667	118.498	124.342	135.807	140.170
110	109.334	119.609	122.250	129.385	135.480	147.414	151.949
120	119.334	130.055	132.806	140.233	146.568	158.950	163.648
130	129.334	140.483	143.340	151.045	157.610	170.423	175.278
140	139.334	150.894	153.854	161.827	168.613	181.840	186.847
150	149.334	161.291	164.349	172.581	179.581	193.208	198.360

LITERATURA

1. Paul J. Fortier, Howard E. Michel, *Computer Systems Performance Evaluation and Prediction*, Digital Press, 2003.
2. B. Radenković, M. Stanojević, A. Marković, *Računarska simulacija*, FON Beograd, 1999.
3. David J. Lilja, *Measuring Computer Performance, A practitioner's guide*, Cambridge University Press, 2004.
4. Raj Jain, *The Art of Computer Systems Performance Analysis*, Wiley Computer Publishing, 1991.